

Trabajo de Fin de grado

Arquitecturas neuronales Polinómicas.

Polynomial neural architectures

Código TFG: 25-26-A10

Autor: Nicolás González Sánchez.

Tutor: Jesús S. Aguilar Ruiz

Grado en Ingeniería Informática en Sistemas de Información

Facultad: Escuela Politécnica Superior

Universidad Pablo de Olavide.

Curso académico: 2025 / 2026

Sevilla, mayo de 2026.

Contenido

1. Introducción. 5
   1.1. Contexto. 5
   1.2. Problema. 5
   1.3. Objetivos de la investigación. 6
   1.4. Hipótesis. 6
2. Fundamentos teóricos. 7
   2.1. Arquitectura y Componentes de las Redes Neuronales. 7
      2.1.1. Neuronas artificiales y pesos. 8
      2.1.2. Estructura común de una red neuronal. 9
      2.1.3. Funciones de activación. 10
   2.2. Proceso de Aprendizaje y Optimización. 12
      2.2.1. Función de pérdida. 12
      2.2.2. Retropropagación. 13
      2.2.3. Optimizadores. 13
      2.2.4. Sobreajuste y regularización. 16
   2.3. Polinomios ortogonales. 17
      2.3.1. Utilidades de la Ortogonalidad. 18
      2.3.2. Mejor capacidad de aproximación 18
   2.4. Polinomios ortogonales implementados. 18
      2.4.1. Polinomios de Legendre. 19
      2.4.2. Polinomios de Chebyshev. 19
      2.4.3. Polinomio de Shmaliy. 20
   2.5. Métricas. 22
      2.5.1. Métricas de clasificación. 22
      2.5.2. Métrica de eficiencia. 23
3. Desarrollo de la nueva capa Polinómica. 25
   3.1. Modificación de capas densas clásicas a expansiones Polinómicas. 25
   3.2. Implementación de capas personalizadas con Keras. 26
      3.2.1. Método Init(). 26
      3.2.2. Método Build(). 27
      3.2.3. Método Call(). 27
4. Experimentación. 31
   4.1. Conjuntos de *Dataset*s 31
   4.2. Descripción de los datos. 33
   4.3. Preprocesamiento de los datos. 36
      4.3.1. Partición del conjunto de datos. 36

4.3.2. Normalización. 37
4.3.3. Codificación de etiquetas. 38
4.4. Estructura de los modelos. 38
4.4.1. Explicación del modelo común. 39
4.4.2. Modelo estándar utilizado. 40
4.5. Método de validación. 41
4.5.1. k fold cross validation. 41
Versión estratificada Vs Aleatoria 42
4.5.2. Partición interna (Hold-out 70/30) 42
4.5.3. Prevención de contaminación de datos 42
4.6. Hiperparámetros. 43
5. Tecnologías utilizadas. 45
5.1. Lenguaje de programación y Entorno de Desarrollo 45
5.2. Framework de Aprendizaje Profundo. 45
5.3. Infraestructura de Hardware. 46
5.4. Otras librerías implementadas. 47
6. Planificación 49
6.1. Metodología de trabajo 49
6.1.1. Entendimiento teórico y práctico. 49
6.1.2. Desarrollo de pruebas y modelos base 49
6.1.3. Desarrollo final y entorno de pruebas. 50
6.2. Control de versiones y seguimiento. 50
6.3. Estimación de esfuerzo 50
7. Resultados. 53
7.1. Análisis de los modelos. 53
7.1.1. Análisis del accuracy por grado. 54
- Punto de operación optimo. 54
7.1.2. Fenómeno de degradación por sobreajuste. 54
7.1.3. Análisis del rendimiento según la naturaleza del *dataset*. 55
7.1.4. Conclusión. 56
7.2. Velocidad de convergencia. 56
7.3. Otras métricas. 59
7.3.1. Precision y Recall. 59
7.3.2. F1-score. 59
7.4. Resultados detallados por conjunto de datos. 60
7.4.1. Australian credit. 60
7.4.2. Breast cancer coimbra. 61

7.4.3. Cardiotocography morphologic pattern. 63
7.4.4. customer_churn. 65
7.4.5. Data banknote authentication. 66
7.4.6. Fertility diagnosis. 68
7.4.7. heart. 70
7.4.8. Hepatitis c. 71
7.4.9. Iris. 73
7.4.10. Landsat satellite. 75
7.4.11. Leaf. 76
7.4.12. Magic gamma telescope. 78
7.4.13. Mesothelioma. 80
7.4.14. Movement libras. 81
7.4.15. Musk v1. 83
7.4.16. Musk v2. 85
7.4.17. Parkinson. 86
7.4.18. Phishing websites. 88
7.4.19. Planning relax. 90
7.4.20. Rice cammeo osmancik. 91
7.4.21. Room occupancy. 93
7.4.22. Sonar. 95
7.4.23. Speaker accent recognition. 96
7.4.24. Vertebral column. 98
7.4.25. Vowel. 100
7.4.26. Winequality-red. 101
7.1.1. Winequality-White. 103
8. Trabajos relacionados. 105
8.1. Expansión funcional mediante Redes de Chebyshev (J.C. Patra). 105
8.2. Optimalidad de los modelos polinómicos de Legendre (Milojković et al.) 105
9. Conclusión y cierre. 107
Bibliografía 109

1. Introducción.

1.1. Contexto.

El presente trabajo se enmarca en la investigación y desarrollo de una arquitectura complementaria para las capas ocultas de las Redes Neuronales Artificiales. En los modelos convencionales, las denominadas "capas densas" operan mediante una combinación lineal (transformación afín) entre las variables de entrada y una matriz de pesos entrenables, sobre la cual se aplica posteriormente una función de activación no lineal para extraer patrones de los datos. [Haykin, 1999; Goodfellow, Bengio y Courville, 2016]

La principal contribución de este estudio radica en una modificación estructural de dicha capa: la transición de una entrada lineal hacia una expansión de base polinomial. En lugar de procesar directamente las variables originales del conjunto de datos, la información de entrada se proyecta previamente sobre un espacio de características de mayor dimensión mediante el uso de polinomios ortogonales.

Esta estrategia, conocida en la literatura como *feature map expansion*, ha demostrado ser eficaz en contextos de aprendizaje automático al enriquecer la representación de los datos antes de la proyección lineal. [Goodfellow, Bengio y Courville, 2016; LeCun, Bengio y Hinton, 2015]

Específicamente, se han implementado y evaluado las familias de Legendre, Chebyshev y Shmaliy, seleccionadas por sus propiedades únicas de convergencia, estabilidad numérica y capacidad para minimizar el error de aproximación. Para la ejecución técnica de esta propuesta, se ha utilizado el lenguaje de programación Python, empleando la API Funcional de Keras [Chollet et al., 2015] sobre el motor de TensorFlow [Abadi et al., 2015].

1.2. Problema.

El diseño de las capas ocultas en las redes neuronales ha permanecido prácticamente invariable desde la consolidación del aprendizaje profundo: la transformación afín sigue siendo el componente fundamental en la mayoría de las arquitecturas actuales. Esta formulación presenta una limitación estructural clara: toda la capacidad de representación no lineal recae exclusivamente en la función de activación, lo que obliga a incrementar la profundidad o el ancho de la red para capturar relaciones complejas entre variables.

Esta dependencia de la profundidad como único mecanismo de expresividad conlleva consecuencias prácticas negativas: mayor número de parámetros entrenables, mayor riesgo de sobreajuste, mayor coste computacional y mayor dificultad de interpretación. Surge, por tanto, la necesidad de explorar alternativas que enriquezcan la representación de los datos en la propia etapa de proyección, sin depender exclusivamente de la profundidad del modelo.

1.3. Objetivos de la investigación.

El objetivo principal de este trabajo es diseñar, implementar y evaluar capas neuronales personalizadas basadas en expansiones de bases polinómicas ortogonales, como alternativa a las capas densas convencionales, en tareas de clasificación sobre conjuntos de datos tabulares. De este objetivo principal se derivan los siguientes objetivos específicos:

1. Implementar tres familias de capas polinómicas (Legendre, Chebyshev y Shmaliy) como capas personalizadas en Keras.
2. Comparar su rendimiento frente a una red neuronal densa estándar en términos de exactitud, precisión, recall y F1-score.
3. Analizar el efecto del grado polinómico sobre la capacidad de generalización y el tiempo de convergencia.
4. Determinar el grado óptimo de operación para cada familia polinómica en función de la naturaleza del conjunto de datos.
5. Evaluar la viabilidad computacional de las arquitecturas propuestas en términos de tiempo de entrenamiento.

1.4. Hipótesis.

La hipótesis central de este trabajo sostiene que la sustitución de la transformación lineal en la capa de proyección por una expansión de base polinómica ortogonal incrementará la capacidad de representación del modelo, mejorando la exactitud de clasificación respecto al modelo lineal de referencia sin un incremento proporcional del coste computacional.

En base a esta primera hipótesis, se plantea también las siguientes ideas:

- Existe un grado polinómico óptimo, comprendidos entre 2 y 6, a partir del cual el rendimiento se degrada por sobreajuste estructural.
- Las diferentes familias de polinomios ortogonales presentarán ventajas diferenciadas en función de las características del conjunto de datos (tamaño, dimensionalidad y número de clases).
- Las capas polinómicas convergerán más rápidamente que la capa densa estándar, gracias a la mayor riqueza del flujo de gradientes generado por la expansión polinómica.

2. Fundamentos teóricos.

2.1. Arquitectura y Componentes de las Redes Neuronales.

Según Haykin, "*Una red neuronal artificial puede definirse como un sistema de procesamiento distribuido en paralelo, compuesto por unidades simples de cálculo interconectadas, con capacidad para aprender a partir de ejemplos y almacenar dicho conocimiento en los pesos de sus conexiones*." [Haykin, 1999]

Esta capacidad de aprendizaje convierte a las redes neuronales en una herramienta especialmente adecuada para problemas complejos de clasificación, regresión y reconocimiento de patrones [Goodfellow, Bengio y Courville, 2016].

LeCun et al. destacaron que el aprendizaje profundo permite que los modelos descubran representaciones jerárquicas de los datos de forma automática, lo que ha impulsado su adopción masiva en inteligencia artificial, minería de datos y análisis predictivo [LeCun, Bengio y Hinton, 2015].

El término neuronal proviene de la similitud conceptual con el funcionamiento del cerebro humano, especialmente en dos aspectos fundamentales:

- En primer lugar, la red extrae patrones de su entorno mediante un proceso de aprendizaje, ajustando sus parámetros internos a partir de los datos de entrenamiento.
- En segundo lugar, dicho conocimiento queda almacenado en la intensidad de las conexiones entre neuronas, representadas matemáticamente mediante los pesos sinápticos

El proceso de entrenamiento consiste en la aplicación de un algoritmo que modifica de forma iterativa los parámetros libres del modelo: los pesos (w) y los sesgos o *bias* (b). El objetivo principal es minimizar una función de pérdida (loss function).

El algoritmo de retropropagación del error (*backpropagation*), formulado por Rumelhart et al. [Rumelhart, Hinton y Williams, 1986], constituye el mecanismo fundamental que permite calcular los gradientes necesarios para la actualización de pesos. No obstante, además de la función de pérdida (*loss*), existen diversas métricas de rendimiento, como la precisión (), la sensibilidad o la puntuación F1, que permiten calificar la eficacia del entrenamiento y la capacidad de generalización del modelo desde distintas perspectivas analíticas.

Y por último es necesario definir el concepto de "No linealidad", lo cual indica que una red neuronal debe ser capaz de resolver problemas los cuales no posean relaciones lineales entre sus datos. Hornik et al. [Hornik, Stinchcombe y White, 1989] demostraron formalmente que las redes neuronales con al menos una capa oculta y funciones de activación no lineales son aproximadores universales, es decir, pueden aproximar cualquier función continua con precisión arbitraria dado un número suficiente de neuronas.

En el contexto de este trabajo, las redes neuronales convencionales sirven como base para proponer una innovación en la etapa de proyección de datos. En lugar de aplicar la matriz de pesos directamente sobre las características originales, se introduce una expansión de base polinómica ortogonal, manteniendo la filosofía de funcionamiento de una red neuronal densa convencional, pero incorporando una transformación previa que enriquece la representación del espacio de entrada.

2.1.1. Neuronas artificiales y pesos.

La neurona artificial es la unidad computacional básica de una red neuronal, inspirada funcionalmente en la neurona biológica del sistema nervioso humano. En términos matemáticos, recibe un vector de entradas $X = (x^1, x^2, \dots, x_n)$ correspondientes con las "Features" del conjunto de dato analizado para realizar una combinación lineal ponderada de dichas entradas y produce una salida a través de una función de activación.

Las operaciones de las neuronas se pueden expresar de la siguiente forma:

$$Z = \sum_{i=1}^{d} w_i \cdot x_i + b$$

$$y = f(z)$$

Fórmula 1: Calculo del coste para cada salida de la neurona

Donde:

- $x = (x_1, \dots, x_i)$ corresponde con el vector de entradas recibidas (Features).
- $w = (x_1, \dots, w_i)$ corresponde el vector de pesos entrenables, que representa la importancia relativa de cada entrada.
- *d* : Corresponde con el número total de features
- *b* : Es el sesgo (*bias*), también entrenables, que actúa como umbral desplazando la función de activación de forma independiente a las entradas.
- *f* : Es la función de activación no lineal.
- *y* : Es la salida de la neurona.

Uno de los elementos fundamentales de cualquier red neuronal son los pesos o *weights*. Estos representan los parámetros internos del modelo y determinan la influencia que cada una de las variables de entrada ejerce sobre las neuronas de las capas posteriores. Durante el proceso de entrenamiento, dichos pesos se ajustan automáticamente con el objetivo de minimizar el error cometido por la red, permitiendo así que el modelo aprenda patrones y relaciones presentes en los datos [HAYKIN, 1999].

La salida intermedia de la neurona se obtiene la formula vista anteriormente

$$Z = \sum_{i=1}^{d} w_i \cdot x_i + b$$

Esta operación constituye la transformación lineal básica sobre la que posteriormente se aplica una función de activación para introducir no linealidad en el modelo [GOODFELLOW, BENGIO y COURVILLE, 2016].

El funcionamiento base de los pesos es muy simple: En caso de ser características con gran importancia para el modelo, los pesos correspondientes a dicha entrada aumentarán, dándole un mayor peso a dichas neuronas, mientras que aquellas que posean un peso cercano a 0 o sean bajas indican una menor importancia para el entrenamiento del modelo.

Al inicializar los pesos, no contienen ninguna información útil y se asignan mediante estrategias de inicialización aleatoria cuidadosamente diseñadas. Una mala elección de estos valores iniciales puede provocar problemas de convergencia, como la

desaparición o explosión del gradiente, dificultando considerablemente el aprendizaje del modelo. Por ello, existen métodos específicos de inicialización, como el esquema de Xavier o Glorot, cuyo objetivo es mantener una varianza estable de las activaciones y de los gradientes a lo largo de la red [GLOROT y BENGIO, 2010].

Posteriormente, durante el entrenamiento, los pesos son modificados iterativamente mediante algoritmos de optimización basados en el descenso por gradiente. El procedimiento más extendido para calcular estas actualizaciones es la retropropagación del error (Backpropagation) el cual se explicará en apartados posteriores, el cual permite estimar cómo afecta cada peso al error global del modelo y ajustar sus valores en la dirección que minimiza dicho error [RUMELHART, HINTON y WILLIAMS, 1986].

En el contexto del presente trabajo, el concepto de peso adquiere una relevancia adicional. Mientras que en una capa densa convencional existe un único peso asociado a cada conexión entre neuronas, las capas polinómicas implementadas incorporan una dimensión adicional correspondiente al grado del polinomio. De este modo, la red aprende un conjunto independiente de pesos para cada grado de la expansión polinómica, permitiendo ponderar de forma distinta la contribución de cada componente de la base ortogonal y aumentando así la capacidad de representación del modelo.

2.1.2. Estructura común de una red neuronal.

Las neuronas se organizan en capas, y el conjunto de capas interconectadas forma la arquitectura de la red neuronal. En una arquitectura completamente conectada (*fully connected*), cada neurona de una capa recibe como entrada las salidas de todas las neuronas de la capa anterior. Se distinguen tres tipos fundamentales:

- **Capa de entrada (*Input Layer*):**
  - Constituye el punto de entrada de la información al modelo. Su dimensión coincide con el número de características del vector de entrada y no realiza ninguna transformación sobre los datos; simplemente los distribuye hacia la siguiente capa. En el contexto de este trabajo, la dimensión de la capa de entrada varía dinámicamente en función del *dataset* analizado, determinándose de forma automática en el método build() de cada capa personalizada.

- **Capas ocultas (*Hidden Layers*):**
  - Son las capas intermedias situadas entre la capa de entrada y la de salida. Cada capa oculta aplica una transformación no lineal sobre la representación recibida, extrayendo características de mayor nivel de abstracción. La profundidad de la red (correspondiente al número de capas ocultas totales que se utilicen)y su anchura (correspondiente al número de neurona en cada capa) son hiperparámetros fundamentales que determinan la capacidad representativa del modelo. Según el Teorema de Aproximación Universal [Hornik, Stinchcombe y White, 1989], una red con una única capa oculta con activaciones no lineales puede aproximar cualquier función continua con precisión arbitraria dado un número suficiente de neuronas; sin embargo, en la práctica, arquitecturas más profundas suelen ser más eficientes para representar funciones complejas.

- **Capa de salida (*Output Layer*):**
  - Produce la predicción final del modelo. Su diseño depende de la tarea a resolver. En los problemas de clasificación multiclase abordados en este trabajo, la capa de salida contiene tantas neuronas como clases existen y emplea la función Softmax como activación, transformando las activaciones en una distribución de probabilidad normalizada sobre todas las clases posibles.

2.1.3. Funciones de activación.

Las funciones de activación constituyen uno de los componentes fundamentales dentro de las redes neuronales artificiales, ya que son las encargadas de introducir no linealidad en el proceso de aprendizaje [Goodfellow, Bengio y Courville, 2016]. Estas funciones se aplican sobre la salida de cada neurona tras realizar la combinación lineal entre las entradas, los pesos sinápticos y el término de sesgo.

Sin ellas, una red neuronal profunda se comportaría únicamente como una transformación lineal equivalente, independientemente del número de capas utilizadas, limitando enormemente su capacidad de representación [Goodfellow, Bengio y Courville, 2016; Hornik, Stinchcombe y White, 1989]

A lo largo del desarrollo de este trabajo se estudiaron distintas funciones de activación con el objetivo de analizar cuáles resultaban más adecuadas para redes neuronales basadas en expansiones polinómicas ortogonales. Algunas de las estudiadas fueron las siguientes:

- **Función Sigmoide:**
  - La función sigmoide transforma cualquier valor real al intervalo (0,1) generando una transición suave entre ambos extremos. Siendo su fórmula matemática la siguiente:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Fórmula 2: Calculo de la función de activación Sigmoide

Esta función de activación es ampliamente conocida y utilizada por su interpretación probabilística. Sin embargo, presenta importantes limitaciones relacionadas con la saturación y el desvanecimiento del gradiente (“vanishing gradient”), fenómeno documentado por Bengio et al. [Bengio, Simard y Frasconi, 1994] que dificulta el entrenamiento de redes profundas.

- **Tangente hiperbólica (tanh):**
  - La función tangente hiperbólica es similar a la sigmoide, aunque transforma los valores al intervalo (−1,1) en lugar del dominio (0,1) que tenía la anterior. Su fórmula es la siguiente:

$$\tanh(x) = \frac{e^{X} - e^{-X}}{e^{X} + e^{-X}}$$

Fórmula 3: Calculo de la función de activación de la Tangente hiperbólica

Una de sus ventajas es que se encuentra centrada en cero, lo que puede favorecer la estabilidad durante el entrenamiento y favorecer a nuestra

implementación por el dominio en el que trabaja. No obstante, también presenta problemas de saturación para valores elevados de entrada.

- **ReLU (Rectified Linear Unit):**
  - Debido a su simplicidad computacional y su capacidad para reducir el problema del vanishing gradient. Su comportamiento consiste en anular todos los valores negativos y mantener sin modificar los positivos.

$$Relu(x) = \max(0, X)$$

Esto permite una propagación del gradiente más estable en comparación con funciones saturantes. Aunque también posee diferentes problemas, como por ejemplo el de a veces generar neuronas muertas al reducir demasiado los valores de sus pesos.

- **Swish:**
  - La función Swish fue propuesta por Ramachandran et al. [Ramachandran, Zoph y Le, 2017] de Google Brain como una alternativa suave a ReLU, combinando propiedades no lineales con una transición continua y derivable.

$$Swish(x) = x \cdot \sigma(x) \text{ (Sigmoide)}$$

Fórmula 4: Calculo de la función de activación Swish

En estudios empíricos, Swish mostró mejoras consistentes sobre ReLU en redes profundas [Nair y Hinton, 2010], siendo esta la función de activación empleada nuestras capas polinómicas ya que su forma suave permite preservar la información generada por las transformaciones polinómicas. La elección de Swish está directamente motivada también por la naturaleza oscilatoria de los polinomios ortogonales empleados, que dando un adelanto a lo que se explicará en el análisis de dichos polinomios, estos poseen la siguiente propiedad: todos ellos oscilan simétricamente alrededor del cero, generando activaciones tanto positivas como negativas. Aplicar ReLU truncaría de forma homogénea la mitad negativa, destruyendo información calculada explícitamente por la capa.

- **Softmax:**
  - La función de activación Softmax también se emplea muy habitualmente, en particular en la capa de salida de problemas de clasificación.

$$Softmax(x)_i = \frac{e^{x_i}}{\sum_{j=1}^{K} e^{x_j}}$$

Fórmula 5: Calculo de la función de activación Softmax

Fue introducida en el contexto de redes neuronales por Bridle [Bridle, 1990]. Gracias a ella, cada salida puede interpretarse como la probabilidad estimada de pertenencia a una determinada clase, siendo la suma total igual a 1 [Goodfellow, Bengio y Courville, 2016].

2.2. Proceso de Aprendizaje y Optimización.

Una vez definida la arquitectura y los componentes estructurales de la red, es fundamental comprender el mecanismo mediante el cual el modelo transforma una configuración aleatoria inicial en un sistema capaz de realizar predicciones precisas. Este proceso, conocido como aprendizaje, se basa en la mejora iterativa de los parámetros del modelo a través de la minimización de una función de error. De esta forma veremos la forma que seguiremos para cuantificar el error, al igual que como ese error lo utilizaremos para enseñar a la misma red en que zonas se ha equivocado y en cuales, hasta la actualización final de los pesos mediante algoritmos de optimización, concluyendo con las estrategias necesarias para asegurar que este aprendizaje sea capaz de generalizar sobre datos nuevos.

2.2.1. Función de pérdida.

La función de pérdida (o función de coste) cuantifica la discrepancia entre las predicciones del modelo y las etiquetas reales del conjunto de datos. El proceso de entrenamiento consiste, en última instancia, en minimizar esta función ajustando los parámetros de la red mediante retropropagación y el optimizador.

En este trabajo se ha empleado la función Sparse Categorical Crossentropy, una variante de la entropía cruzada categórica ampliamente utilizada en problemas de clasificación multiclase cuando las etiquetas se encuentran codificadas como valores enteros. Matemáticamente, la pérdida promedio sobre un conjunto de (N) muestras se expresa como:

$$\text{Loss} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{k=1}^{K} y_{i,k} * Log\left(\hat{y}_{i,k}\right)$$

Fórmula 6: Fórmula para calcular la función de perdida

Donde:

- N: Es el número total de muestras.
- K: Es el número total de clases.
- $y_{i,k}$: Es el valor real de la muestra i para la clase k
- $\hat{y}_{i,k}$: Es la probabilidad predicha por el modelo para la muestra i y la clase k

La entropía cruzada penaliza especialmente las predicciones incorrectas realizadas con una confianza elevada. Si el modelo asigna una probabilidad baja a la clase correcta, el término (-$Log\left(\hat{y}_{i,k}\right)$), aumenta considerablemente, generando una pérdida mayor y, por tanto, una corrección más intensa durante el proceso de optimización.

Esta propiedad favorece una convergencia más rápida y estable, motivo por el cual la entropía cruzada se ha convertido en la función de pérdida de referencia para problemas de clasificación multiclase en combinación con la función Softmax [GOODFELLOW, BENGIO y COURVILLE, 2016].

2.2.2. Retropropagación.

El algoritmo de retropropagación del error (backpropagation) [Rumelhart, Hinton y Williams, 1986] es el mecanismo fundamental mediante el cual una red neuronal aprende a partir de los datos. Su objetivo es calcular eficientemente el gradiente de la función de pérdida respecto a cada parámetro entrenable de la red, para que el optimizador pueda actualizarlos. El proceso de entrenamiento se divide en dos fases:

- **Propagación hacia delante (forward pass):**
  - La información fluye desde la capa de entrada hasta la capa de salida, calculando las activaciones de cada capa sucesivamente y produciendo una predicción ŷ. A partir de esta predicción y la etiqueta real y, se calcula el valor de la función de pérdida L.

- **Propagación hacia atrás (backward pass):**
  - El error se propaga desde la capa de salida hacia la capa de entrada aplicando la regla de la cadena del cálculo diferencial, lo que permite computar el gradiente de L respecto a cada peso w_{ij} y sesgo b_j de la red:

$$\frac{\partial L}{\partial w_{\{ij\}}} = \delta_{\mathrm{j}} \cdot a_{\mathrm{i}}$$

Fórmula 7: Fórmula para aplicar la técnica de propagación hacia atrás

Donde:

- δ_j: Es el término de error de la neurona j, calculado recursivamente desde la capa de salida.
- a_i: Es la activación de la neurona i de la capa anterior.

Estos gradientes son utilizados por el optimizador para actualizar los parámetros en la dirección que minimiza la función de pérdida. En las capas polinómicas implementadas en este trabajo el proceso de retropropagación es análogo al de una capa densa convencional, dado que las recurrencias polinómicas son operaciones diferenciables definidas sobre tensores, y TensorFlow calcula los gradientes automáticamente mediante diferenciación automática [Abadi et al., 2015].

2.2.3. Optimizadores.

Un optimizador es el algoritmo encargado de actualizar los parámetros entrenables de una red neuronal (pesos y sesgos) con el objetivo de minimizar la función de pérdida. Para ello, utiliza los gradientes calculados mediante el algoritmo de retropropagación (backpropagation), ajustando iterativamente los parámetros hasta encontrar una configuración que reduzca el error del modelo sobre los datos de entrenamiento [RUMELHART, HINTON y WILLIAMS, 1986].

De forma general, la actualización matemática de un parámetro genérico θ en un instante de tiempo t se define mediante la ecuación:

$$\theta_t = \theta_{t-1} - \alpha \nabla_\theta L(\theta_{t-1})$$

Fórmula 8: Fórmula base de la función del optimizador.

Donde:

- α representa la tasa de aprendizaje conocido como "*learning rate*"
- $\nabla_\theta L$ es el gradiente de la función de pérdida con respecto al parámetro θ

La tasa de aprendizaje es un hiperparámetro crítico: valores demasiado elevados pueden provocar divergencia en la optimización, mientras que valores excesivamente pequeños ralentizan drásticamente la convergencia hacia el mínimo global. teniendo que buscar un valor adecuado para este hiperparametro. [GOODFELLOW, BENGIO y COURVILLE, 2016].

Con el avance del aprendizaje profundo en redes neuronales, han surgido diversas familias de algoritmos de optimización para superar las limitaciones de esta fórmula básica. A continuación, veremos algunas de las más potentes y utilizadas actualmente:

- **Descenso de Gradiente Estocástico (SGD):**

Es la variante más clásica y utilizada en redes simples. En lugar de calcular el gradiente sobre todo el conjunto de datos, SGD calcula la actualización utilizando un único ejemplo o un pequeño subconjunto de muestras al que se denomina *mini-batch* [BOTTOU, 2010]. Aunque es la mayoría de casos es eficiente, presenta una convergencia lenta en áreas donde la superficie de pérdida es mucho más pronunciada en una dimensión que en otra, quedando frecuentemente atrapado en mínimos locales sin alcanzar la solución óptima.

- **SGD con Momento (*Momentum*):**

Para mitigar los problemas comunes del optimizados SGD, se introdujo el concepto de "Momentum" [POLYAK, 1964]. Este método acumula una media móvil exponencial de los gradientes pasados para determinar la dirección actual, acelerando el descenso en las dimensiones cuyos gradientes apuntan consistentemente en la misma dirección y atenuando las oscilaciones, permitiendo de esta forma dar soluciones más optimas al no quedarse estancado en áreas estechas.

- **Algoritmos Adaptativos (Adagrad y RMSprop):**

La principal limitación de SGD y Momentum es que aplican la misma tasa de aprendizaje global "α" a todos los parámetros. Para evitar esto se desarrollaron algoritmos como RMSprop [TIELEMAN y HINTON, 2012], que resuelven esto dividiendo la tasa de aprendizaje de cada peso por una media móvil exponencial de la magnitud de sus gradientes recientes. Esto permite que los parámetros con gradientes grandes tengan actualizaciones más conservadoras y aquellos con gradientes pequeños se actualicen de forma más agresiva.

- **Estimación de Momento Adaptativo (Adam):**

Este optimizador fue propuesto por primera vez por Kingma y Ba [Kingma y Ba, 2015], al cual denominaron como “Adam”, y es actualmente el optimizador estándar en el que se utiliza constantemente en el entrenamiento de redes neuronales profundas, y es el optimizador que se ha empleado en este trabajo. Adam combina las ventajas computacionales de Momentum y de RMSprop. Para ello, el algoritmo calcula de forma iterativa dos estimaciones estadísticas independientes para cada parámetro:

El primer Momentum m_t que estima la media de los gradientes:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)\nabla_\theta L$$

Formula 9: Fórmula del optimizador Adam para obtener el primer gradiente.

Y después un segundo momento V_t , que estima la varianza no centrada de dichos gradientes:

$$V_t = \beta_2 V_{t-1} + (1 - \beta_2)(\nabla_\theta L)^2$$

Formula 10: Fórmula del optimizador Adam para obtener el segundo gradiente.

- Quedando la regla para actualizar los pesos de la siguiente forma:

$$\theta_t = \theta_{t-1} - \frac{\alpha}{\sqrt{\widehat{V}_t} + \epsilon}\,\widehat{m}_t$$

Formula 11: Fórmula final de la función de optimización Adam aplicando el primer y segundo gradiente.

Donde:

- α: Es la tasa de aprendizaje base, siendo este el hiperparametro de paso inicial. A diferencia del descenso de gradiente estándar, en Adam este valor no se multiplica directamente por el gradiente bruto, sino que actúa como un límite superior para la magnitud de la actualización.

- $\widehat{m}_t$: Se trata de la estimación corregida del primer Momentum (anteriormente descrita su fórmula para calcularla) el cual actúa como vector de dirección al que se tiene que desplazar. Al basarse en la media móvil de los gradientes pasados, funciona físicamente como el momentum. Esto significa que, si por ejemplo los gradientes anteriores han apuntado consistentemente en una misma dirección, m_t acelera la convergencia hacia ese mínimo; si los gradientes oscilan bruscamente de signo, m_t suaviza dichas oscilaciones reduciendo el impacto.

- $\sqrt{\widehat{V}_t}$: Se trata de la estimación corregida del segundo momentun, el cual en este caso funciona como un factor de escala adaptativa en función de los parámetros que han sufrido cambios: penaliza dinámicamente a los parámetros que han sufrido actualizaciones muy grandes (reduciendo su tasa de aprendizaje efectiva) y recompensa a los parámetros con gradientes históricamente pequeños (aumentando su tasa).

- ϵ: Conocido como “Épsilon”, es un constante escalar minúscula (típicamente configurada en el orden de 10^{-7}o 10^{-8}). Su única función es proporcionar estabilidad numérica, evitando matemáticamente una división por cero en las primeras iteraciones del entrenamiento o en regiones donde el gradiente $\widehat{V}_t$ se desvanece por completo.

La interacción conjunta de los elementos de Adam resulta de extrema necesidad técnica para las capas polinómicas desarrolladas en este trabajo.

En esta arquitectura, todas las variables de entrada deben ser normalizadas estrictamente al intervalo [-1, 1], ya que este constituye el dominio natural de ortogonalidad para las familias polinómicas evaluadas. Si bien esta normalización garantiza que las activaciones polinómicas se mantengan acotadas, introduce un desafío de optimización distinto debido a la disparidad en las frecuencias de oscilación de los polinomios en función de su grado.

Un polinomio ortogonal de grado “n” pequeño presenta una menor cantidad de intercepciones con el 0 en comparación con un grado superior, dentro del intervalo [-1, 1], como se podrá ver al analizar la gráfica de los polinomios más adelante. Esto significa que los grados inferiores (como n=0 o n=1) actúan como funciones de baja frecuencia, capturando tendencias globales con gradientes muy estables y consistentes. Por el contrario, los grados superiores (como n=3 o n=5) actúan como funciones de alta frecuencia espacial detectando tendencias que pueden pasar de forma más discreta. Pequeñas variaciones en los datos de entrada o en los pesos provocan oscilaciones bruscas en la salida de estos grados, lo que se traduce en gradientes altamente volátiles y erráticos durante la retropropagación, generando una superficie difícil de navegar [GOODFELLOW, BENGIO y COURVILLE, 2016; ; DAUPHIN et al., 2014].

Si se utilizara un optimizador estático como SGD, aplicar una misma tasa de aprendizaje global “α” a todas las dimensiones funcionales resultaría ineficaz, ya que un paso lo suficientemente grande para entrenar rápido los grados inferiores causaría inestabilidad en los grados superiores. Aquí es donde el denominador adaptativo de Adam ($\sqrt{\widehat{V}_t}$) resulta vital. Al registrar la varianza histórica, Adam detecta qué parámetros del tensor W, están sufriendo gradientes volátiles (los grados altos) y penaliza su actualización reduciendo dinámicamente su tasa de aprendizaje efectiva. Simultáneamente, acelera el aprendizaje de los pesos asociados a los grados inferiores cuyos gradientes son más consistentes.

De este modo, el optimizador garantiza un entrenamiento armónico de toda la base ortogonal, estabilizando matemáticamente las altas frecuencias sin sacrificar la velocidad de convergencia general del modelo.

2.2.4. Sobreajuste y regularización.

Durante el entrenamiento de una red neuronal pueden producirse dos fenómenos opuestos que limitan la capacidad de generalización del modelo [Goodfellow, Bengio y Courville, 2016]:

Por un lado, el subajuste (*underfitting*) ocurre cuando el modelo carece de la capacidad representativa o flexibilidad matemática necesaria para capturar la

complejidad de los datos, obteniendo un rendimiento deficiente tanto en la fase de entrenamiento como en la de validación.

Por otro lado, el sobreajuste (*overfitting*) se produce cuando el modelo posee una capacidad excesiva y aprende a memorizar los patrones específicos del conjunto de entrenamiento (incluyendo el ruido estadístico y los valores atípicos) en lugar de abstraer las relaciones subyacentes. Como consecuencia, el modelo obtiene un rendimiento muy elevado sobre los datos de entrenamiento, pero falla significativamente al enfrentarse a datos no vistos.

En el contexto de las redes polinómicas abordadas en este trabajo, el sobreajuste es un riesgo crítico al incrementar el grado del polinomio, ya que los términos de orden superior pueden generar oscilaciones de alta frecuencia que distorsionan la frontera de decisión y adaptan el modelo a variaciones irrelevantes del conjunto de datos.

Para mitigar estos efectos y garantizar que la red neuronal generalice correctamente, se emplean técnicas de regularización. Estas estrategias introducen modificaciones en la arquitectura o en el algoritmo de aprendizaje con el fin de reducir el error de prueba sin afectar negativamente al error de entrenamiento. Entre las técnicas más destacadas se encuentran:

- **Parada temprana (*Early Stopping*):**
    - Consiste en monitorizar el error del modelo sobre un conjunto de validación independiente durante el entrenamiento. Si el error de validación deja de disminuir o comienza a aumentar de forma sostenida durante un número determinado de iteraciones (denominado *paciencia*), el entrenamiento se detiene y se restauran los pesos de la época óptima [Prechelt, 1998].
- **Abandono (*Dropout*):**
    - Es una técnica estructural que desactiva aleatoriamente una proporción de neuronas durante cada iteración del paso hacia delante. Esto evita la co-adaptación excesiva entre unidades y fuerza a la red a aprender representaciones más robustas, redundantes y distribuidas [Srivastava et al., 2014].

2.3. Polinomios ortogonales.

Los polinomios ortogonales constituyen una familia de funciones que son altamente usadas en áreas como el análisis numérico y la teoría de aproximación, debido a diversas propiedades matemáticas que poseen [Gautschi, 2004; Szegő, 1939]. Entre ellas, su principal característica es que cada polinomio de la familia es ortogonal respecto a los demás bajo un producto interno definido sobre un intervalo concreto.

Esto significa, de forma más intuitiva, que cada polinomio aporta información nueva e independiente, evitando redundancias entre términos de distintos grados. Esta propiedad puede compararse con el concepto geométrico de vectores perpendiculares: dos vectores ortogonales no comparten dirección y, de manera análoga, dos polinomios ortogonales no comparten la misma información entre ellos. [Gautschi, 2004]

Por lo que una familia de polinomios $\{P_0\ (x), P_1\ (x), \dots, P_n\ (x)\}$ es ortogonal si cumple lo siguiente:

$$\int_a^b P_i(x)\, P_j(x) w(x)\, dx = 0 \qquad \text{para i} \neq \text{j}$$

Fórmula 12: Fórmula que define el comportamiento base de una función denominada como ortogonal

Siendo w(x) una función peso asociada a la familia polinómica correspondiente.

Esta definición es especialmente relevante dentro de este trabajo, ya que permite utilizar dichas bases como mecanismos de transformación estructurada del espacio de características antes de aplicar la matriz de pesos de la red neuronal, constituyendo la base del nuevo enfoque propuesto para el procesamiento de la información dentro de los modelos personalizados propuestos más adelantes.

2.3.1. Utilidades de la Ortogonalidad.

La utilidad principal de los polinomios ortogonales reside en su capacidad para representar funciones complejas mediante combinaciones lineales estables y numéricamente robustas [Quarteroni, Sacco y Saleri, 2006].

A diferencia de la base monomial clásica $(1, x, x^1, x^2, \dots, x^n)$ las bases ortogonales reducen significativamente la correlación entre términos de distintos grados, fenómeno conocido como multicolinealidad, que puede dificultar el proceso de optimización en modelos de aprendizaje automático [Quarteroni, Sacco y Saleri, 2006; Trefethen, 2019].

Además, cada grado del polinomio modela un nivel distinto de complejidad dentro de la información de entrada:

- **Grados bajos:** Capturan tendencias globales y patrones generales.
- **Grados medios:** Permiten modelar mejor curvaturas y relaciones no lineales intermedias.
- **Grados altos:** Ayudan a detectar detalles finos, oscilaciones y pequeñas variaciones en los datos.

Gracias a esta separación natural por niveles de complejidad, la red neuronal puede aprender representaciones más expresivas de las variables de entrada.

2.3.2. Mejor capacidad de aproximación

Los polinomios ortogonales son ampliamente utilizados en teoría de aproximación porque permiten representar funciones no lineales con menor error y mejor convergencia que otras bases clásicas [Trefethen, 2019]. El teorema de aproximación de Weierstrass [Weierstrass, 1885] establece que cualquier función continua en un intervalo cerrado puede aproximarse uniformemente mediante polinomios con precisión arbitraria. Las bases ortogonales, al minimizar la correlación entre términos, ofrecen una convergencia más rápida y estable que la base monomial estándar [Quarteroni, Sacco y Saleri, 2006].

2.4. Polinomios ortogonales implementados.

En este trabajo se han seleccionado tres familias de polinomios ortogonales para sustituir la base de activación tradicional. En este trabajo se han seleccionado tres familias de polinomios ortogonales para sustituir la base de activación tradicional. La

elección de estas familias responde tanto a su gran popularidad en la literatura científica como a las propiedades matemáticas que poseen de forma individual [Gautschi, 2004; Szegő, 1939], ya que cada una presenta propiedades geométricas y numéricas distintas que afectan directamente a la capacidad de generalización del modelo.

2.4.1. Polinomios de Legendre.

Los polinomios de Legendre son quizá la familia más utilizada en el análisis físico-matemático [Szegő, 1939]. Son ortogonales en el intervalo (-1,1) respecto a una función de peso unidad, w(x) = 1. Esto hace que, al poseer un peso uniforme, los polinomios de Legendre no priorizan ninguna región del dominio de entrada, lo que los hace excelentes para aproximaciones generales donde no se conoce a priori la distribución de los datos [Gautschi, 2004].

Para su implementación no se utilizó la formula original de Legendre, sino la fórmula de recurrencia de Bonnet (también conocida como la relación de recurrencia de tres términos para los polinomios de Legendre), siendo esta una expresión fundamental que permite calcular polinomios de orden superior con bajo coste computacional [Gautschi, 2004]:

$$P_n(x) = \frac{(2n-1)xP_{n-1} - (n-1)P_{n-2}(x)}{n}$$

Fórmula 13: Fórmula de recurrencia de Bonnet para los polinomios de Legendre.

Donde:

$$P_0(x) = 1 \quad y \quad P_1(x) = x$$

Dando para los primeros N, la siguiente distribución:

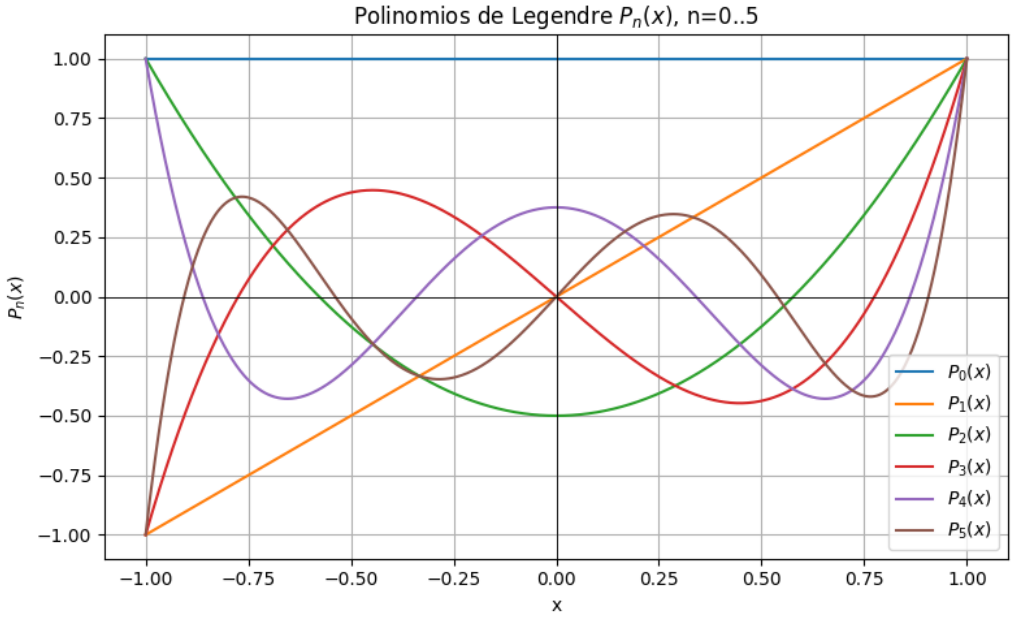


Figura 1: Gráfico de los 6 primeros polinomios de Legendre

2.4.2. Polinomios de Chebyshev.

Los polinomios de Chebyshev son bastantes conocidos y empleados en análisis numérico por la cualidad que tienen llamada “Minimax” la cual permite minimizar el error

máximo de aproximación [Mason y Handscomb, 2002], aunque también son altamente utilizados en procesamiento de imágenes. Este polinomio debe estar acotado entre los intervalos (-1,1) para que se cumpla la ortogonalidad, además de una función de peso tal que:

$$w(x) = (1 - x^2)^{\frac{-1}{2}}$$

La principal fortaleza de los polinomios de Chebyshev reside en su capacidad para evitar el "fenómeno de Runge", proporcionando una aproximación suave que no oscila violentamente en los bordes del dominio. Esto los hace ideales para arquitecturas donde la estabilidad frente a valores extremos en los datos de entrada es crítica.

En esta implementación se utilizarán los denominados de primera especie (Tn), siendo calculadas con la fórmula de recurrencia [Mason y Handscomb, 2002]:

$$T_n(\mathrm{x}) = 2\mathrm{x}T_{n-1}(\mathrm{x}) - T_{n-2}(\mathrm{x})$$

Fórmula 14: Fórmula de recurrencia para los polinomios de Chebyshev de primera especie

Donde:

$$T_0(x) = 1 \; y \; T_1(x) = x$$

Dando para los primeros N, la siguiente distribución:

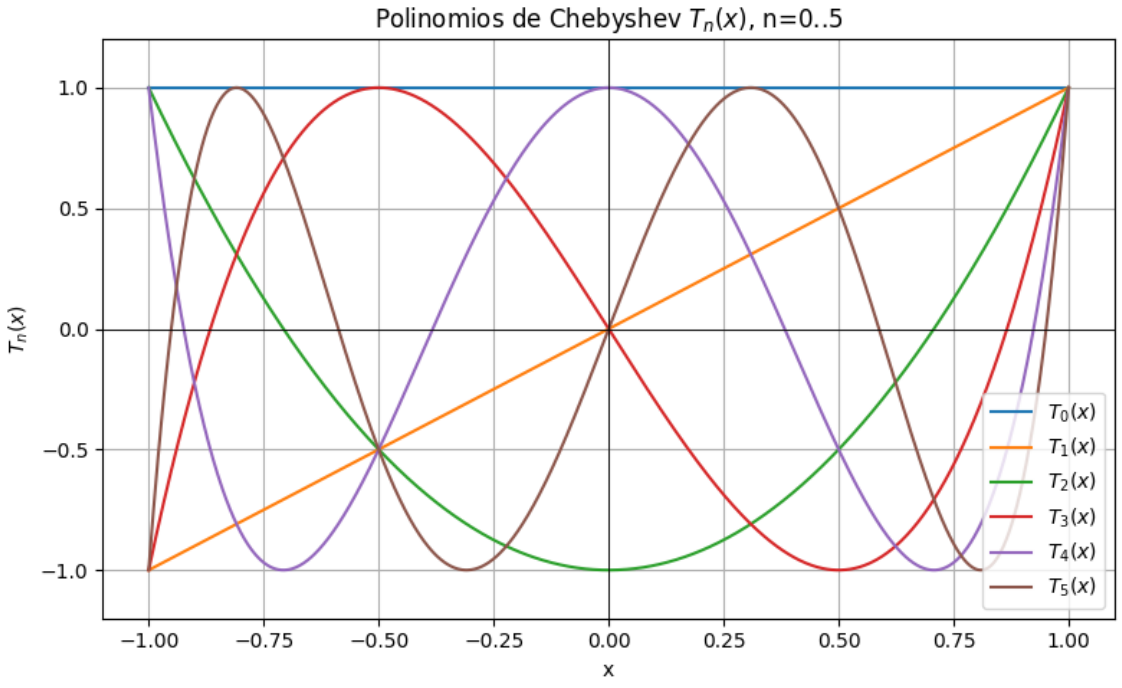


Figura 2: Gráfico de los 6 primeros polinomios de Chebyshev

2.4.3. Polinomio de Shmaliy.

A diferencia de las familias de Legendre o Chebyshev, que operan de forma continua, la integración de los polinomios de Shmaliy en este trabajo responde a la necesidad de gestionar datos que presentan una naturaleza intrínsecamente discreta [Shmaliy, 2006]. Esta familia se define sobre un soporte discreto de N puntos, lo que

introduce una variable crítica en el diseño de la red: el grado de libertad del "muestreo" interno de la neurona.

En la práctica, este parámetro N no es una constante fija, sino que actúa como un hiperparámetro de control de la sensibilidad de la base funcional. Al ajustar el horizonte, permitimos que la neurona adapte su capacidad de respuesta a la densidad y variabilidad del *dataset* analizado. Esta flexibilidad es lo que diferencia a Shmaliy de las bases ortogonales estáticas [Shmaliy, 2006]; siendo especialmente valiosa en análisis de señales discretas.

En esta implementación, para calcular estos polinomios hemos usado la fórmula de recurrencia siguiente:

$$S_{n+1}(x,N) = \frac{(2n+1)(N-1-2x)}{(n+1)(N-1-n)} S_n(x,N) - \frac{n(N+n)}{(n+1)(N-1-n)} S_{n-1}(x,N)$$

Fórmula 15: Fórmula de recurrencia para los polinomios ortogonales discretos de Hahn para calcular Shmaliy

Donde:

$$S_0(x,N) = 1 \quad Y \quad S_1(x,N) = 1 - \frac{2x}{N-1}$$

Dando para los primeros N, la siguiente distribución:

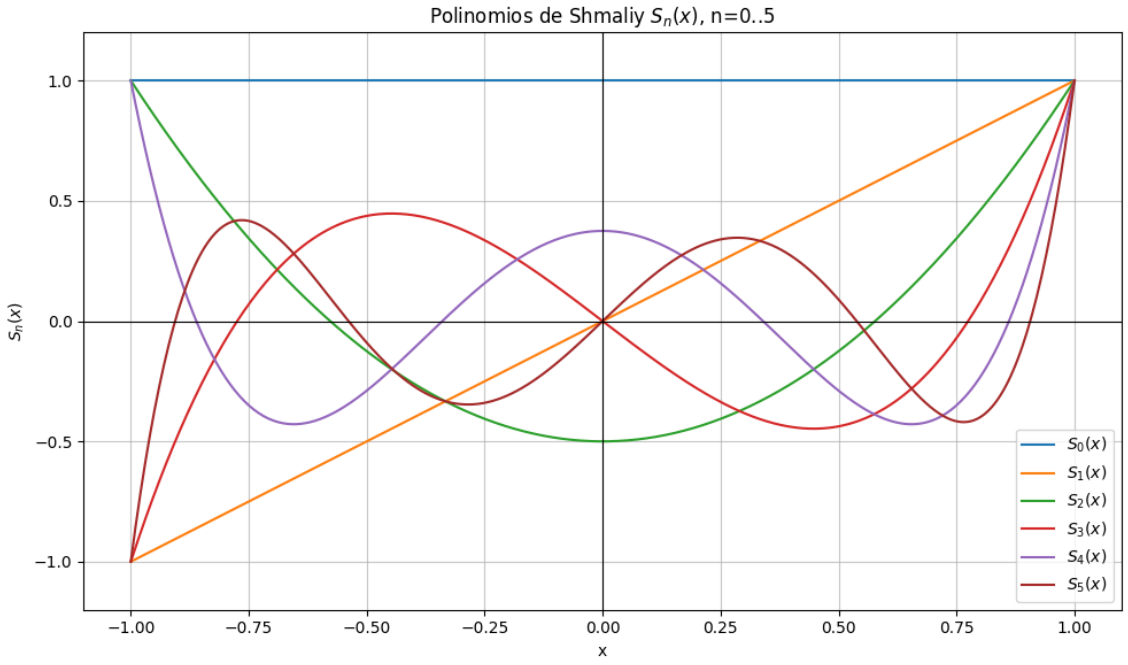


Figura 3: Gráfico de los 6 primeros polinomios de Shmaliy

Dada la sensibilidad de los polinomios de Shmaliy al hiperparámetro N, en este trabajo se aplica una búsqueda ligera de este valor antes del entrenamiento principal, siendo un hiperparámetro que buscamos optimizar [Bergstra y Bengio, 2012].

2.5. Métricas.

Para evaluar la eficacia de las arquitecturas propuestas y establecer una comparativa rigurosa entre las bases de Chebyshev, Legendre y Shmaliy frente al modelo lineal de referencia, se han seleccionado cinco métricas a calcular para cada modelo. El uso de este conjunto de métricas permite analizar el rendimiento de los modelos no solo desde la perspectiva del acierto global, sino también considerando la calidad de la clasificación en cada categoría y la eficiencia computacional.

2.5.1. Métricas de clasificación.

Este tipo de métricas tiene como objetivo cuantificar el desempeño y la capacidad de discriminación de los modelos. Más allá de ofrecer un simple porcentaje de acierto, permiten analizar la fiabilidad del sistema y su capacidad para generalizar sobre datos no vistos.

- **Exactitud (Accuracy):**
  - Se trata de la métrica principal de este estudio, la cual representa la proporción de predicciones correctas sobre el total de casos evaluados. Se ha definido como el objetivo central de mejora al buscar arquitecturas que maximicen el aprendizaje de las fronteras de decisión complejas mediante la expansión polinómica. Esta métrica se calcula con lo que se conoce como "Matriz de confusión" la cual posee los siguientes valores:

    - **TP (Verdaderos Positivos):** Muestras de clase A predichas correctamente como A.
    - **TN (Verdaderos Negativos):** Muestras de clase B predichas correctamente como B.
    - **FP (Falsos Positivos):** Errores de "falsa alarma"; el modelo predice clase A cuando es B.
    - **FN (Falsos Negativos):** Errores de "omisión"; el modelo predice clase B cuando es A.

    Siendo la formula final:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Fórmula 16: Fórmula para el cálculo del Accuracy del modelo.

    Esta clasificación se puede aumentar en función al número de clases que se tenga, para problema de multiclases como los evaluados en este estudio.

- **Precision.**
  - Esta métrica mide la cantidad de casos positivos que el modelo es capaz de identificar del total de casos positivos reales existentes. Lo que quiere decir

es que mide de todos los aciertos de la clase que se tiene, los compara con el total de casos negativos que se habían dado también como positivos. De esta forma, la fórmula para calcularla es la siguiente:

$$Precisión = \frac{TP}{TP + FP}$$

Fórmula 17: Fórmula para el cálculo de la Presicion del modelo.

En los casos de estar ente un problema multiclase, se procede a calcular la media de las precisiones de cada una de ellas, para de esta forma sacar una precisión media entre todas las clases.

$$Precisión_{media} = \frac{1}{n} * \sum_{i=1}^{N} Precisión_i$$

Fórmula 18: Fórmula para el cálculo de la Presicion media del modelo.

- **Recall.**
  - Mide la capacidad del modelo para identificar correctamente todos los casos positivos reales dentro del conjunto de datos. En otras palabras, cuantifica qué proporción de los elementos realmente positivos ha sido detectada por el modelo. Un valor alto de recall indica que el modelo es capaz de capturar la mayor parte de la información relevante de la clase positiva, minimizando los falsos negativos.

- **F1-Score.**
  - Esta métrica que combina la precisión y el recall en una única medida mediante la media armónica de ambas siendo su objetivo ofrecer un equilibrio entre la capacidad del modelo para evitar falsos positivos (precisión) y falsos negativos (recall). Para su cálculo, se utilizó la siguiente formula:

$$F1 = 2 * \frac{Precisión * Recall}{Precisión + Recall}$$

Fórmula 19: Fórmula para el cálculo del F1 Score del modelo.

2.5.2. Métrica de eficiencia.

Más allá del rendimiento predictivo, la viabilidad técnica de una nueva arquitectura de red neuronal depende de su eficiencia computacional. Por ello, se ha integrado el Tiempo Promedio de Entrenamiento (segundos) como una métrica fundamental en la comparativa.

El objetivo de este indicador es evaluar el compromiso entre el coste y el beneficio de las capas personalizadas. Para garantizar una medición justa, se ha registrado el tiempo de ejecución en cada uno de los 10 pliegues de la validación cruzada para cada *dataset*, permitiendo generar así un análisis promedio del tiempo necesario para entrenar los diferentes modelos y de esta forma determinar si estos nuevos modelos son aptos para entornos con recursos limitados.

Asimismo, con el propósito de agilizar la fase de experimentación, optimizar el flujo de trabajo y reducir drásticamente los tiempos globales de cómputo, el entorno de pruebas se diseñó e implementó para permitir la ejecución de los test mediante

multiprocesamiento. Esta paralelización permitió distribuir la carga de las evaluaciones y el entrenamiento de los diferentes pliegues de manera simultánea entre los núcleos del procesador, garantizando una recolección de datos eficiente sin alterar la precisión ni el aislamiento de las mediciones temporales individuales.

3. Desarrollo de la nueva capa Polinómica.

Este capítulo describe en detalle la finalidad central del trabajo que consiste en la implementación de capas neuronales personalizadas cuya transformación interna no sea lineal en el sentido clásico, sino polinómica. Se explicará además cómo se ha logrado implementar dicha solución, al igual que diferentes decisiones de diseño tomadas a la hora de crearlas.

3.1. Modificación de capas densas clásicas a expansiones Polinómicas.

En una capa densa convencional, la transformación que realiza cada neurona sobre el vector de entrada $x = (x_1, \dots, x_i)$ se puede expresar con la siguiente formula, como se explicó en el apartado 2.1.1:

$$Z = \sum_{i=1}^{d} w_i \cdot x_i + b$$

Esta formulación pese a su gran capacidad, presenta una limitación estructural, siendo que la red aprende exclusivamente combinaciones lineales de las entradas originales, desplazando toda la carga de la no linealidad a la función de activación.

La propuesta de esta investigación modifica dicha arquitectura introduciendo una expansión de base polinómica ortogonal previa a la proyección lineal. Matemáticamente, el vector de entrada no se mapea directamente, sino que se proyecta hacia un espacio de características de mayor dimensionalidad $\Phi: \mathbb{R}^d \rightarrow \mathbb{R}^{(n+1)\ d}$, donde *n* es el grado máximo del polinomio. De esta forma, el proceso que quedaría sería el siguiente:

Se realiza una expansión de características, haciendo que cada variable de entrada individual x_i (donde *i* se mueve de 1 hasta *d*, correspondiente con el número de features) se proyecta de forma no lineal hacia una base de características expandidas mediante el cálculo de sus componentes ortogonales hasta el grado *n* del polinomio:

$$\Phi(x_i) = [P_0(x_i), P_1(x_i), \dots, P_n(x_i)]^T$$

Fórmula 21: Fórmula de la nueva implementación para la función de pesos.

Posteriormente se realiza la combinación del tensor mediante la suma ponderada de estas bases expandidas con sus respectivos pesos entrenables (un peso por cada grado y variable):

$$Z_j = \sum_{k=0}^{n} \sum_{i=1}^{d} \Phi(x_i)_k \cdot W_{k,i,j}$$

Donde:

- Z_j: representa la salida escalar calculada por la j-ésima neurona de la capa antes de ser procesada por la función de activación.

- d : es la dimensionalidad de entrada, es decir, el número total de características (*features*) que componen el vector de datos recibido.
- n : indica el grado máximo de la expansión polinómica configurado como hiperparámetro para la capa.
- $\Phi(x_{\mathrm{i}})_k$: corresponde al valor exacto de la base polinómica ortogonal (Legendre, Chebyshev o Shmaliy) de grado K, evaluada individualmente sobre la característica de entrada (x_{i}).
- $W_{k,i,j}$ **:** Es el tensor tridimensional de pesos entrenables. Este coeficiente específico pondera la importancia que tiene el grado polinómico K de la característica i para la predicción de la neurona de destino j.

De esta forma, la doble sumatoria permite a la neurona evaluar y acumular de forma simultánea todas las variables originales tras ser proyectadas en sus múltiples grados. Al sustituir la matriz de pesos bidimensional clásica por este tensor tridimensional W, la red es capaz de capturar e interiorizar relaciones matemáticas de mayor complejidad y curvatura directamente en la etapa de proyección, sin depender exclusivamente de la profundidad del modelo para introducir no linealidad.

Y, por último, se termina aplicándola a Z_j la función de activación correspondiente:

$$y = f(Z)$$

Este diseño permite que la red aprenda relaciones de mayor complejidad directamente en la etapa de proyección, sin depender exclusivamente de la profundidad del modelo para introducir no linealidad. Este principio conecta con las Kolmogorov-Arnold Networks (KAN) propuestas por Liu et al. [Liu et al., 2024], las cuales sugieren sustituir las transformaciones lineales fijas de las capas tradicionales por funciones aprendibles sobre las conexiones, permitiendo una aproximación de funciones más eficiente y expresiva

3.2. Implementación de capas personalizadas con Keras.

Se ha utilizado la API funcional de Keras [Chollet et al., 2015] para la creación de la nueva capa, lo que permite el control completo sobre su funcionamiento interno. Todas las capas desarrolladas (ChebyshevLayer, PolynomialLegendre y ShmaliyLayer) heredan de la clase base keras.layers.Layer, siguiendo el patrón de diseño de capas personalizadas establecido en la documentación oficial de TensorFlow/Keras [Abadi et al., 2015].

3.2.1. Método Init().

Como su propio nombre indica, esta será la función que se encarga de inicializar la capa, en la cual se definirán parámetros que guardará la capa. Un ejemplo de este mismo cogiendo de ejemplo el polinomio de Chebyshev sería el siguiente:

```
def __init__(self, units, degree, **kwargs):
    super().__init__(**kwargs)
    self.units = units
    self.degree = degree
```

En este mismo ejemplo, se puede observar la declaración de 2 propiedades extras en la capa instanciada. La primera de ellas, “units”, corresponde al número de neuronas requeridas; la segunda, “degree”, indica el grado del polinomio a aplicar, siendo de esta forma como las demás funciones van a acceder a dichos valores.

En el caso del polinomio de Shmaliy se añade además el parámetro N, cuyo papel como hiperparámetro de sensibilidad fue descrito en la sección 2.4.3.

3.2.2. Método Build().

El método `build()` es invocado dinámicamente durante la primera propagación hacia delante para reservar el espacio paramétrico exacto. Como se dedujo de la ecuación general en el apartado 3.1, la expansión polinómica requiere que los parámetros adopten la forma de un tensor tridimensional.

```
def build(self, input_shape):
    self.w = self.add_weight(
        shape=(self.degree + 1, input_shape[-1],self.units),
        initializer="glorot_uniform",
        trainable=True,
        name="chebyshev_weights"
    )
```

En términos de álgebra lineal, la dimensión geométrica (self.degree + 1, input_shape[-1], self.units) garantiza que cada neurona asigne un hiperplano de pesos independiente no solo a cada característica de entrada, sino a cada grado funcional ortogonal. Adicionalmente, el inicializador glorot_uniform (también conocido como Xavier initialization) ha sido utilizado para mantener una varianza estable en los gradientes durante las primeras fases del entrenamiento. Glorot y Bengio [Glorot y Bengio, 2010] demostraron que este esquema de inicialización favorece una convergencia más estable en redes profundas al equilibrar la varianza de las activaciones entre capas.

3.2.3. Método Call().

El método call define el comportamiento de la capa durante la propagación hacia delante (forward propagation). Es el encargado de recibir los datos de entrada, generar la expansión polinómica y calcular finalmente la salida de la capa utilizando los pesos entrenables definidos previamente en el método “`build()`”.

Como mencionamos en el apartado anterior, gracias a la formulación recursiva, no es necesario calcular cada polinomio de manera explícita mediante expresiones algebraicas completas, reduciendo así la complejidad computacional y facilitando la escalabilidad a grados superiores.

Gracias a eso, mediante un bucle iterativo, se generan el resto de polinomios utilizando las fórmulas de los respectivos polinomios mostradas en el punto 2.4, para al final, cada nuevo término calculado se multiplica por su conjunto de pesos correspondiente y se añade progresivamente a la salida total de la capa.

A modo de ejemplo, la capa polinómica de Chebyshev sería la siguiente:

```
def call(self, inputs):
    prev_prev = tf.ones_like(inputs)
    prev = inputs
    output = (
        tf.matmul(prev_prev, self.w[0]) +
        tf.matmul(prev, self.w[1])
    )
    for n in range(2, self.degree + 1):
        current = 2.0 * inputs * prev - prev_prev
        output += tf.matmul(current, self.w[n])
        prev_prev = prev
        prev = current
    return output
```

Entrando más en detalle de como este fragmento del código corresponde a la implementación descrita en el apartado 3.1, analizaremos la función que nos porportiona Tensorflow denominada `matmul`. Esta función corresponde con la operación de la aplicación de la formula del polinomio, con la función de pesos correspondiente:

$$\Phi(x_i)_k \cdot W_{k,i,j}$$

Pero al utilizar `tf.matmul`, el código resuelve vectorialmente de un solo golpe el sumatorio interior de la ecuación general de la neurona expuesta en el apartado 3.1:

$$\sum_{i=1}^{d} \Phi(x_i)_k \cdot W_{k,i,j}$$

proyectando las d características sobre las unidades de salida de forma simultánea y posteriormente, al acumular iterativamente estas matrices en la variable output mediante el operador de suma (+=), el bucle `for` resuelve el sumatorio exterior ($\sum_{k=0}^{n}$) correspondiente a los grados funcionales, completando el modelo algebraico

sin recurrir a iteraciones escalares ineficientes y de esta forma aplicar todo lo explicado previamente.

Un aspecto importante es que la salida de la capa no está acotada y puede tomar valores tanto positivos como negativos, dado que los polinomios ortogonales oscilan alrededor del cero por definición [Gautschi, 2004]. Esta propiedad fue decisiva para la elección de Swish como función de activación [Ramachandran, Zoph y Le, 2017].

Aunque las tres capas comparten este esquema general en su método call(), cada familia polinómica introduce variaciones significativas en la recurrencia empleada. En el caso de Shmaliy, la formulación es más compleja porque los polinomios están definidos sobre una secuencia discreta de posiciones y dependen del hiperparámetro adicional N ya mencionado, que controla la longitud efectiva de dicha secuencia. Antes de aplicar la recurrencia tensorial, las entradas normalizadas al intervalo [-1, 1] se mapean a dichas posiciones discretas mediante la fórmula matemática analizada en el punto 2.4.3.

4. Experimentación.

Este capítulo recoge el diseño experimental completo del trabajo. A continuación, se detallan los conjuntos de datos empleados, las técnicas de preprocesamiento aplicadas para adecuar la información, el protocolo de validación adoptado para asegurar la fiabilidad de las métricas y la estructura de los modelos junto con la selección de hiperparámetros que definen el entrenamiento.

4.1. Conjuntos de *Datasets*

Para la evaluación experimental se han seleccionado 27 conjuntos de datos públicos procedentes del repositorio UCI Machine Learning Repository, la cual se trata de una librería publica de *Datasets* pensada para justamente realizar estudios de modelos. La selección de *datasets* que se ha seleccionado cubre un amplio espectro de dominios de aplicación (como medicina, finanzas, biometría, agricultura, etc.) así como distintos rangos de dimensionalidad, tamaño muestral y número de clases. Permitiendo evaluar la robustez y la capacidad de generalización de los modelos propuestos más allá de un entorno específico.

La diversidad de características de los *datasets* seleccionados es en sí misma una garantía metodológica: tal y como señalan Fernández-Delgado et al. (Fernández-Delgado et al. 2014) en su estudio comparativo de 179 clasificadores sobre 121 *datasets* del repositorio UCI, las conclusiones sobre el rendimiento relativo de los métodos solo son fiables cuando se validan sobre un conjunto suficientemente heterogéneo de problemas.

A continuación, se expondrá una tabla con los *datasets* seleccionados, al igual que algunos parámetros fundamentales de cada uno de ellos:

Dataset	Instancias	Atributos	Clases	Dominio	Clasificación
Australian Credit	690	14	2	Finanzas	Binaria
Breast Cancer Coimbra	116	9	2	Medicina	Binaria
Cardiotocography (Morpho.)	2.126	23	10	Medicina	Multiclase
Customer Churn	7.043	20	2	Telecomunicion	Binaria
Banknote Authentication	1.372	4	2	Seguridad	Binaria
Fertility Diagnosis	100	9	2	Medicina	Binaria
hepatitis_c	1385	28	5	Medicina	Multiclase
Heart Disease	303	13	2	Medicina	Binaria

Dataset	Instancias	Atributos	Clases	Dominio	Clasificación
Iris	150	4	3	Botánica	Multiclase
landsat_satellite	6,435	36	7	Clima	Multiclase
leaf	340	16	-	Naturaleza	Multiclase
MAGIC Gamma Telescope	19,020	10	2	Física y Química	Binaria
Mesothelioma	324	34	2	Oncología	Binaria
Movement Libras	360	90	15	Biometría	Multiclase
MUSK v1	476	166	2	Química	Binaria
MUSK v2	6,598	168	2	Biología	Binaria
Parkinson	195	22	2	Neurología	Binaria
Phishing Websites	11,055	31	2	Ciberseguridad	Binaria
Planning Relax	182	12	2	Psicología	Binaria
Rice (Cammeo/Osmancik)	3.810	7	2	Agricultura	Binaria
Room Occupancy	10,129	16	4	Computer Science	Multiclase
Sonar	208	60	2	Geofísica	Binaria
Speaker Accent Recog.	329	12	6	Lingüística	Multiclase
Vertebral Column	310	6	3	Ortopedia	Multiclase
Vowel	990	13	11	Fonética	Multiclase
Wine Quality (Red)	1.599	11	6	Enología	Multiclase
Wine Quality (White)	4.898	11	7	Enología	Multiclase

Tabla 1. Seleccionados del repositorio UCI Machine Learning Repository con los que se ha experimentado.

4.2. Descripción de los datos.

A continuación, se describe brevemente cada conjunto de datos con el objetivo de contextualizar los resultados experimentales. Se destacan aquellas características que resultan especialmente relevantes para el análisis comparativo entre las arquitecturas polinómicas.

- **Australian Credit Approval.**
  - *Dataset* clásico de aprobación de crédito bancario con 690 instancias y 14 atributos (mixtos: continuos y categóricos). El objetivo es predecir si una solicitud de crédito debe aprobarse o rechazarse.

- **Breast Cancer Coimbra.**
  - Conjunto de datos clínico de pequeño tamaño (116 instancias, 9 atributos bioquímicos) orientado a la detección del cáncer de mama. Incluye marcadores como IMC, glucosa, insulina, leptina y adiponectina. Al contar con pocas instancias este *dataset* permite estudiar que tan bien generaliza nuestro modelo los datos.

- **Cardiotocography — Morphologic Pattern.**
  - *Dataset* de monitorización fetal con 2.126 registros y 23 atributos extraídos de cardiotocogramas. La variable objetivo clasifica el estado fetal en 10 estados diferentes. Este *dataset* es muy interesante ya que es muy rico tanto en registros como en clases, por lo que puede dar una vista muy buena del modelo.

- **Customer Churn.**
  - *Dataset* de abandono de clientes en el sector de las telecomunicaciones. Con más de 7.000 instancias y 20 atributo. Este *dataset* presenta un fuerte desequilibrio de clases teniendo muchas más que otras, por lo que pone a prueba la capacidad de las métricas ponderadas.

- **Banknote Authentication.**
  - Conjunto de datos de autenticación de billetes basado en transformadas wavelets de imágenes fotográficas. Con 1.372 instancias y sólo 4 atributos.

- **Fertility Diagnosis.**
  - *Dataset* de diagnóstico de fertilidad masculina con 100 instancias y 9 atributos relativos a hábitos de vida, enfermedades pasadas y datos demográficos. Como sucedía con el *Dataset* “Breast Cancer Coimbra” al tener pocas intancias pone a prueba al *dataset* a ver cómo se comporta ante este tipo de casos.
- **Hepatitis C.**
  - Conjunto de datos médico orientado al diagnóstico de la hepatitis C y el estadio del daño hepático. Contiene 1.385 instancias y 28 atributos clínicos y de laboratorio, clasificando a los pacientes en 5 categorías de progresión de la enfermedad (desde donantes de sangre sanos hasta pacientes con cirrosis)

- **Heart Disease.**
  - Versión procesada del Cleveland Heart Disease *Dataset* con 303 instancias y 13 atributos clínicos (edad, colesterol, frecuencia cardíaca máxima, etc.).

- **Iris.**
  - *Dataset* altamente famoso de clasificación de flores de iris en tres especies (Setosa, Versicolor, Virginica) a partir de 4 atributos morfológicos. Con 150 instancias perfectamente balanceadas, sirve como referencia de sanidad del sistema y permite verificar que los modelos funcionan correctamente en problemas casi triviales.

- **Landsat Satellite.**
  - Dataset basado en imágenes satelitales multiespectrales para la clasificación de tipos de suelo y cobertura terrestre. Consta de 6.435 instancias y 36 atributos numéricos que representan los valores de los píxeles en diferentes bandas espectrales, divididos en 7 clases climáticas.
- **Leaf.**
  - Dataset botánico centrado en la clasificación de especies de plantas a partir del análisis morfológico de sus hojas. Incluye 340 instancias y 16 atributos (tales como la excentricidad, el contraste o la solidez geométrica) para identificar diversas formas naturales.

- **MAGIC Gamma Telescope.**
  - Dataset del ámbito de la física de altas energías diseñado para discriminar señales astronómicas detectadas por telescopios Cherenkov (cascadas de rayos gamma frente a ruido de fondo). Con 19.020 instancias y 10 atributos, su gran volumen de datos resulta fundamental para poner a prueba la eficiencia y escalabilidad computacional de las capas polinómicas.

- **Mesothelioma.**
  - *Dataset* oncológico relativo al diagnóstico de mesotelioma, un cáncer poco frecuente asociado a la exposición al amianto. Contiene 324 instancias y 34 atributos clínicos y de laboratorio.

- **Movement Libras.**
  - *Dataset* de reconocimiento de gestos de la lengua de signos brasileña (LIBRAS). Contiene 360 instancias con 90 atributos (coordenadas de movimiento de manos) y 15 clases. Es uno de los conjuntos multiclase de mayor complejidad de la selección, ideal para evaluar la capacidad de discriminación fina de los modelos.

- **MUSK v1.**
  - *Dataset* de química computacional destinado a identificar moléculas con olor almizclado (musk) frente a moléculas no almizcladas. Con 476 instancias y 166 atributos de conformaciones moleculares.

- **MUSK v2.**
    - Se trata de una extensión y versión ampliada del dataset MUSK v1. Comparte el objetivo de identificar moléculas con olor almizclado, pero incrementa drásticamente la complejidad y el volumen del problema al incorporar 6.598 instancias y 168 atributos de distintas conformaciones moleculares.

- **Parkinson.**
    - *Dataset* de diagnóstico de la enfermedad de Parkinson a partir de medidas biomédicas de la voz (195 instancias, 22 atributos). La variable objetivo indica la presencia o ausencia de la enfermedad.

- **Phishing Websites.**
    - Conjunto de datos de ciberseguridad enfocado en la detección de páginas web fraudulentas (phishing). Cuenta con 11.055 instancias y 31 atributos extraídos de las características de las URLs y metadatos web. Resulta ideal para evaluar el rendimiento de los modelos en la identificación de patrones de seguridad en dominios informáticos.

- **Planning Relax.**
    - Pequeño *dataset* de psicología experimental (182 instancias, 12 atributos de señales EEG) orientado a predecir el estado de relajación de un sujeto.

- **Rice (Cammeo y Osmancik).**
    - *Dataset* de clasificación de variedades de arroz basado en 7 atributos morfológicos extraídos de imágenes (3.810 instancias) en 2 clases as dos clases (Cammeo y Osmancik)

- **Room Occupancy.**
    - Dataset proveniente de redes de sensores IoT (Internet of Things), enfocado en estimar el nivel de ocupación de una habitación. Posee 10.129 instancias temporales y atributos ambientales (sensores de luz, temperatura, CO2, sonido, etc.) para clasificar el espacio en 4 niveles de ocupación multiclase distintos.

- **Sonar.**
    - *Dataset* de detección de objetos subacuáticos (minas o rocas) mediante señales sonar. Con 208 instancias y 60 atributos de amplitud de frecuencia.

- **Speaker Accent Recognition.**
    - *Dataset* de reconocimiento de acento del hablante en seis categorías lingüísticas a partir de 12 atributos prosódicos (329 instancias).

- **Vertebral Column.**
    - *Dataset* ortopédico de clasificación de patologías de columna vertebral (Normal, Hernia, Espondilolisis) a partir de 6 atributos biomecánicos y 310 instancias.

- **Vowel.**
    - *Dataset* de reconocimiento de vocales del inglés (11 clases fonéticas) a partir de 13 atributos semánticos y 990 instancias. Al trabajar con un alto número de clases, se podrá ver el comportamiento que el modelo en estos casos.

- **Wine Quality (Red y White).**
    - Dos *dataset*s de calidad de vino tinto (1.599 instancias) y blanco (4.898 instancias) basados en análisis fisicoquímicos (11 atributos). La variable objetivo es una puntuación de calidad ordinal de 3 a 9, lo que introduce 6 y 7 clases respectivamente. Además, como otro *dataset*s que hemos visto, posee un alto desequilibrio entre sus clases.

4.3. Preprocesamiento de los datos.

Una vez definida la arquitectura de la nueva capa polinómica y el marco teórico que la sustenta, es fundamental explicar el tratamiento que se realiza a los datos antes de su integración en el modelo. La calidad de los resultados no solo reside en la estructura de la red, sino en asegurar que la información de entrada sea homogénea, esté libre de sesgos de escala y se encuentre adaptada a los requisitos matemáticos de las funciones polinómicas empleadas. Como señalan García et al. (García, Luengo y Herrera 2015), el preprocesamiento constituye una etapa crítica en los flujos de trabajo de aprendizaje automático, ya que los modelos son sensibles a la calidad y la forma de los datos de entrada.

En esta fase, el flujo de trabajo se divide en tres pasos principales: La partición de las muestras, la normalización de las variables y la codificación de las etiquetas.

4.3.1. Partición del conjunto de datos.

Antes de aplicar cualquier transformación o ajuste, el primer paso en el flujo de preprocesamiento es la división del *dataset* original en tres subconjuntos independientes: entrenamiento (train), validación (validation) y prueba (test).

Esta separación responde a un principio fundamental de la metodología experimental en aprendizaje automático: evaluar la capacidad de generalización de un modelo requiere datos que no hayan participado en ningún momento del proceso de aprendizaje ni de la selección de hiperparámetros (Goodfellow, Bengio y Courville 2016).

En este proyecto, la herramienta base utilizada para llevar a cabo estas particiones proporcionales es el método train_test_split de la librería sklearn.model_selection [Pedregosa et al., 2011]. Su implementación computacional permite dividir matrices de características y etiquetas de forma automatizada y controlada, tal y como se ilustra a continuación:

```
X_sub, X_test, y_train, y_test= train_test_split( X, y,
test_size=0.3, stratify=y_train, random_state=42)
```

Donde “X” e “y” son los conjuntos de datos separado por características y clase respectivamente. “test_size” indica cuanto del conjunto se asignará al apartado de test

(70 % train, 30% test) y que lo haga de forma estratificada para que haya equilibrio de clases entre las dos partes.

Adicionalmente, para dotar al estudio de la máxima rigurosidad científica, esta función de partición 70/30 no se ejecuta como un corte estático único sobre el dataset global. Como se detallará en profundidad en el apartado 4.5, este método se aplica de manera iterativa e interna dentro del protocolo de validación cruzada estratificada (K-Fold) y de la mecánica de parada temprana (*Early Stopping*). Esta práctica garantiza que las decisiones tomadas durante el entrenamiento no contaminen la estimación final del rendimiento, impidiendo la filtración de información (*data leakage*) tal y como recomiendan Géron [Géron, 2022] y Kohavi [Kohavi, 1995].

4.3.2. Normalización.

Como se ha señalado en apartados anteriores, las funciones polinómicas implementadas están definidas sobre el intervalo [−1, 1] y únicamente pueden aplicarse correctamente dentro de ese dominio. Sin embargo, en la mayoría de escenarios reales, los datos no se presentan directamente en ese rango: dependiendo del dominio del problema y de sus unidades de medida, los valores pueden variar drásticamente. Por ejemplo, en un *dataset* médico, variables como la estatura en centímetros, la edad en años o el peso en kilogramos presentan escalas muy distintas entre sí.

Sin una normalización adecuada, la diferencia de magnitudes entre parámetros provocaría que las variables con rangos mayores tuviesen un peso injustificado en el cálculo de los gradientes, ralentizando la convergencia y sesgando el aprendizaje. Este efecto, conocido en la literatura como problema de la escala de características (*feature scaling*), es especialmente pronunciado en modelos basados en optimización por descenso de gradiente (LeCun, Bottou y Orr 1998). Al normalizar, se busca mitigar estas disparidades y, fundamentalmente, mapear los valores de entrada al dominio correspondiente de los polinomios.

En el código del proyecto, este preprocesamiento se implementa con el método "*MinMaxScaler*" de la librería "`sklearn.preprocessing`" (Pedregosa et al. 2011), configurado para transformar los valores al intervalo [−1, 1]:

```
scaler = MinMaxScaler(feature_range=(-1, 1))
X_sub = scaler.fit_transform(X_sub)
X_val = scaler.transform(X_val)
X_test = scaler.transform(X_test)
```

Aunque existen diversas técnicas de normalización (como la estandarización mediante la media o mediante la desviación típica), para los requisitos de este modelo es suficiente con realizar un reescalado basado en los valores máximos y mínimos, el cual responde a la siguiente fórmula:

$$X_{nom} = \frac{x - x_{min}}{x_{max} - x_{\min}} * \left(x_{maxRange} - x_{minRange}\right) + x_{minRange}$$

Fórmula 22: Fórmula para normalización de los valores.

Recalcar de forma importante que la normalización no se hace sobre todo el conjunto a la vez, sino que se debe hacer una normalización para cada una de las muestras extraídas de X por separado. De esta forma se mezclan valores entre los diferentes conjuntos para un correcto entrenamiento [Géron 2022].

4.3.3. Codificación de etiquetas.

Los modelos de aprendizaje profundo, basados en operaciones algebraicas y optimización de tensores, no son capaces de interpretar directamente variables categóricas en formato de texto. En problemas de clasificación binaria o multiclase, resulta necesario representar las clases mediante identificadores numéricos antes de poder calcular la función de pérdida durante el entrenamiento.

Existen diversos métodos para llevar a cabo esta tarea. El más directo es el denominado "`Label Encoding`", que consiste en asignar un valor entero únicos a cada clase. De este modo, un conjunto de etiquetas original con el formato:

$$[Conejo, Conejo, Perro, Gato, Perro]$$

Pasa a representarse numéricamente como:

$$[0, 0, 1, 2, 1]$$

Esta conversión permite que el modelo identifique cada clase mediante un identificador discreto, facilitando el cálculo de la función de pérdida durante el entrenamiento, en particular la sparse categorical crossentropy empleada en este trabajo. La alternativa más habitual es el One-Hot Encoding, que proyecta cada clase en un vector binario de dimensión igual al número de clases; sin embargo, en combinación con la activación Softmax y la pérdida mencionada, el Label Encoding resulta más eficiente en memoria y equivalente en capacidad expresiva (Goodfellow, Bengio y Courville 2016).

En el código del proyecto, esta operación se implementa mediante la clase "`LabelEncoder()`"de la librería "`sklearn.preprocessing`" (Pedregosa et al. 2011):

```
le = LabelEncoder()
y_train = le.fit_transform(y_train.ravel())
y_test = le.transform(y_test.ravel())
```

4.4. Estructura de los modelos.

Para garantizar la validez científica de los resultados y permitir una comparación equitativa entre las tres familias polinómicas implementadas (Chebyshev, Legendre y Shmaliy) y *para el modelo lineal,* se ha diseñado una arquitectura de red con una topología común.

Este enfoque asegura que las variaciones observadas en el rendimiento se deban exclusivamente a la naturaleza de la transformación polinómica inicial y no a diferencias en la profundidad o configuración de la red.

4.4.1. Explicación del modelo común.

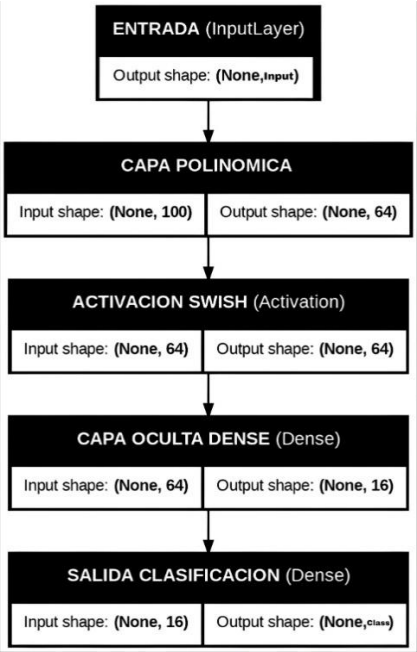


Figura 4: Estructura general de los modelos personalizados que implementan las capas densas personalizadas.

Como se puede apreciar, la estructura implementada en el modelo es bastante sencilla, disponiendo solamente de la capa de entrada, seguida de nuestra capa personalizada y su correspondiente activación, para luego pasar a una capa densa normal, y una última capa de salida.

- **Capa de entrada (*InputLayer*):** Recibe el vector de características del *dataset*, cuya dimensionalidad varía según el conjunto de datos utilizado. Dado que los *dataset*s del repositorio UCI son heterogéneos en cuanto al número de características, la dimensión de entrada se determina dinámicamente dentro del método "`build()`" de cada capa.

- **Capa polinómica personalizada:** Esta es la capa implementada de nuestro estudio. Aquí es donde se aplica la capa personalizada, dependiendo del experimento, con el polinomio correspondiente que implementemos (Chebyshev, Legendre o Shmaliy) con el grado polinómico configurado como hiperparámetro. En todos los casos, la capa proyecta la entrada a un espacio de 64 neuronas de salida. Esta elección de 64 neuronas en la capa polinómica se debe a una búsqueda de equilibrio entre capacidad representativa del modelo y coste computacional, ya que un número de neuronas excesivamente bajo limitaría la capacidad de la capa para capturar patrones complejos, mientras que uno demasiado elevado incrementaría innecesariamente el número de parámetros entrenables y el tiempo de ejecución, especialmente dado que el número de parámetros escala según (degree + 1) × input_dim × units, por lo que

incrementar el grado aumenta la capacidad representativa pero también el coste computacional.

- **Activación Swish:** Tras la expansión polinómica se aplica la función Swish (Ramachandran, Zoph y Le, 2017). Tal y como se justificó en el apartado 3.1, esta función suave y no monótona es ideal para preservar la información de las activaciones negativas generadas por la oscilación natural de los polinomios ortogonales.

- **Capa densa oculta (Dense):** Una capa densa convencional de 16 neuronas con activación ReLU [Glorot, Bordes y Bengio, 2011; Bengio, Simard y Frasconi, 1994] actúa como etapa de compresión y recomposición, aprendiendo combinaciones no lineales sobre las características ya extraídas. En esta etapa ReLU sí resulta adecuada, ya que la información ya ha sido procesada por la capa polinómica anterior.

- **Capa de salida (Dense con Softmax):** La capa de clasificación final es una capa densa cuyo número de unidades corresponde al número de clases del *dataset* con el que estemos trabajando. Además, se le aplica la función de activación Softmax, descrita en el apartado 2.1.4. Gracias a ella, la salida del modelo puede interpretarse directamente como una distribución de probabilidad sobre las clases disponibles, siendo la clase predicha aquella con mayor probabilidad asignada. Además, el modelo se optimiza mediante el algoritmo Adam (Kingma y Ba 2015), con función de pérdida sparse categorical crossentropy, estándar en problemas de clasificación multiclase con etiquetas enteras, los cuales se explicaron en los apartados 2.2.3 y 2.2.1 respectivamente.

4.4.2. Modelo estándar utilizado.

En cuanto al modelo de referencia Lineal, su arquitectura es equivalente, pero reemplaza la capa polinómica por una capa densa convencional de 32 unidades con activación ReLU, seguida igualmente de la capa oculta de 16 unidades y la capa de salida Softmax. Este modelo sirve como línea base para cuantificar el valor añadido que aporta la expansión polinómica descrita en el apartado 2.2 respecto al enfoque de red neuronal estándar.

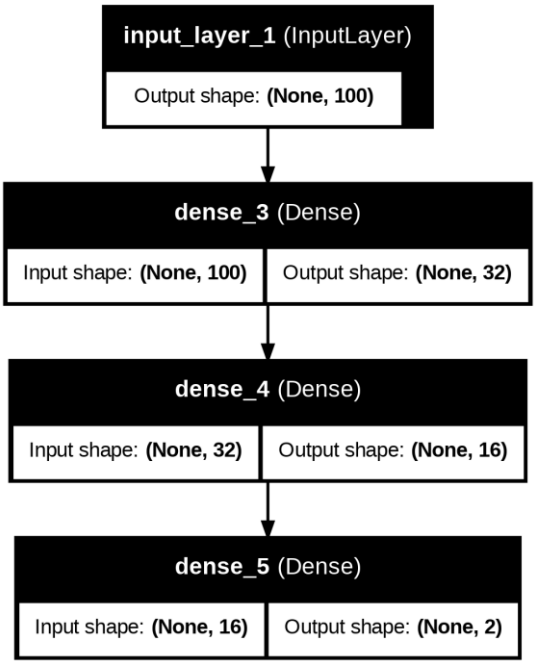


Figura 5: Estructura del modelo estándar utilizando capas densas convencionales

4.5. Método de validación.

Para asegurar la fiabilidad de los resultados y garantizar que las arquitecturas propuestas posean una alta capacidad de generalización, se ha diseñado un protocolo de validación estructurado en dos niveles: un esquema externo de validación cruzada estratificada (K-Fold) y una partición interna de prueba (Hold-out). Este enfoque garantiza la creación de las tres particiones fundamentales del aprendizaje profundo (entrenamiento, validación y prueba) de forma dinámica y sin filtración de información entre ellas.

4.5.1. k fold cross validation.

El primer nivel del proceso que se realiza, consiste en aplicar un procedimiento de K-Fold Cross-Validation sobre la totalidad del conjunto de datos. Mediante el método "`StratifiedKFold`", el *dataset* se divide en K pliegues, garantizando que cada partición mantenga la proporción real de las clases originales al realizarse de forma estratificada, lo cual previene sesgos estadísticos en conjuntos desequilibrados.

El algoritmo itera K veces, donde, en cada iteración, un pliegue distinto (el 10 % de los datos) se aísla por completo para actuar como conjunto de prueba (Test). Este subconjunto se mantiene oculto durante toda la fase de optimización de pesos y solo se utiliza al finalizar la iteración para calcular las métricas definitivas de rendimiento del modelo. Los K-1 pliegues restantes (el 90 % de los datos) se agrupan para formar el bloque macro de entrenamiento de esa iteración.

Este tipo de entrenamiento se ha demostrado que genera un mejor resultado como el estudio realizado por Kohavi [Kohavi 1995] el cual demostrado que K = 10 ofrece un equilibrio óptimo entre el sesgo y la varianza del estimador para conjuntos de datos de tamaño moderado.

Versión estratificada Vs Aleatoria

Una variación fundamental a considerar es la distribución de clases en cada pliegue. En conjuntos de datos desequilibrados, una división puramente aleatoria puede generar particiones en las que determinadas clases estén ausentes o sobrerrepresentadas, produciendo estimaciones sesgadas del rendimiento.

Por eso mismo, en nuestro trabajo se ha decidido realizar de forma estratificada estas agrupaciones, En vez de hacerlo de forma completamente aleatoria los grupos, se trate de distribuir de forma equitativa. Se busca que reflejen una muestra real, siendo que, si predomina 2 clases más que la tercera, al separar mediante este método, hará que cada conjunto tenga alguna muestra de las 3 clases, pero dentro de este mismo, represente cual clase se repite de forma más común y cual no.

En nuestra implementación, se ha utilizado el método "`StratifiedKFold`" que nos viene dentro de la librería "`sklearn.model_selection`", reflejándose así en el código:

```
skf = StratifiedKFold(n_splits=n_folds, shuffle=True, random_state=1)

for train_idx, test_idx in skf.split(X, y):
```

En este fragmento del código podemos ver como se crea un objeto del tipo "`StratifiedKFold` "al cual le pasamos diferentes parámetros, siendo estos por ejemplo en número de particiones que queremos que haga, siendo pasado como "n_folds" además de otra propiedad que se puede activar la cual es "shuffle" la cual, como su nombre indica, antes de realizar las particiones, lo que hace es mezclar el *dataset*, para de esta forma que el conjunto en el que se nos entregó el *dataset* no influyera en nuestro modelo. Esto lo que genera es un bucle de n_fold iteraciones, el cual va asignando en cada iteración, un conjunto diferente (`train_idx, test_idx`) en función del conjunto de datos inicial (`X, y`)

4.5.2. Partición interna (Hold-out 70/30)

Para implementar el mecanismo de parada temprana (Early Stopping) sin contaminar los datos de prueba, se realiza una segunda partición sobre el conjunto K-1 generado. Utilizando el método `train_test_split`, este bloque se divide mediante una proporción 70:30 usado de la siguiente forma:

- El 70 % de los datos constituye el conjunto de sub-entrenamiento (Sub-train). Es el único conjunto de datos que se inyecta en la red neuronal para calcular los gradientes y actualizar matemáticamente el tensor de pesos mediante el optimizador Adam.
- El 30 % restante se reserva como conjunto de validación (Validation). Estos datos pasan por la red neuronal, pero no actualizan los pesos. Su función es permitir que el mecanismo de parada temprana (Early Stopping) calcule la pérdida de validación (val_loss) época tras época. Si el modelo empieza a memorizar el "Sub-train" y se equivoca en este conjunto de validación, el entrenamiento se detiene para evitar el sobreajuste.

4.5.3. Prevención de contaminación de datos

La estructuración de estas particiones cumple un objetivo crítico más allá de la evaluación: garantizar el aislamiento de la información durante el preprocesamiento. Si transformaciones como la normalización se aplicaran sobre la totalidad del *dataset* antes de dividirlo, la media y varianza global influirían en los datos de entrenamiento, filtrando información futura al modelo (fenómeno conocido como Data Leakage).

Para evitar esto, el escalador (`MinMaxScaler`) ajusta sus parámetros estadísticos subyacentes de forma exclusiva sobre el conjunto de sub-entrenamiento (Sub-train) Y una vez calculados los valores mínimos y máximos sobre esta porción aislada, se aplica la transformación (`transform`) al resto de conjuntos (Validation y Test). De esta forma, el encapsulamiento es matemáticamente riguroso y el modelo se evalúa bajo condiciones estrictamente reales.

4.6. Hiperparámetros.

A lo largo del código que se ha ido mostrando en la documentación, se ha podido notar como constantemente trabajamos con nombre de parámetros, en vez de implementar directamente los valores deseados de forma estática, siendo esto por ética sobre todo informática, para evitar lo que se conoce como "Hard-coding". Esta metodología responde principalmente a 2 objetivos fundamentales que son:

- **Modularidad y Mantenibilidad.**
  - Permite realizar ajustes globales en la configuración del experimento modificando un único punto del código, facilitando la replicabilidad y evitando errores de transcripción manual.
- **Consistencia Metodológica.**
  - Asegura que todos los modelos evaluados (Lineal, Chebyshev, Legendre y Shmaliy) se sometan a condiciones de entrenamiento idénticas, garantizando una comparativa equitativa, eliminando así cualquier sesgo derivado de configuraciones diferentes.

A continuación, se detallan los hiperparámetros definidos para la ejecución de los experimentos:

- **N_fold=10:**
  - Número de particiones para la validación cruzada. El valor 10 es el más habitual en los estudios y ofrece un buen equilibrio entre sesgo y varianza del estimador (Kohavi 1995).

- **Epoches= 400:**
  - Representa el límite máximo de iteraciones en el apartado de entrenamiento. Este parámetro tiene mucha importancia ya que, si se selecciona un valor sin control, puede suceder lo que habíamos denominado Overfitting o Underfitting si se seleccionan parámetros muy grandes o pequeños. Para solucionar esto, se ha implementado un método llamado "`create_early_stopping()`". Esto implementa un mecanismo de parada temprana que monitoriza la pérdida de validación (val_loss) y detiene el entrenamiento cuando esta deja de mejorar de forma significativa durante un número determinado de épocas. Este mecanismo, introducido por Prechelt (Prechelt 1998), permite maximizar

la eficiencia del entrenamiento sin necesidad de fijar a priori el número óptimo de épocas.

```
def create_early_stopping(patience=15):
    return keras.callbacks.EarlyStopping(
        monitor="val_loss",
        patience=patience,
        restore_best_weights=True
    )
```

- **Batch_size=32.**
    - Número de muestras procesadas antes de actualizar los pesos de la red. El valor de 32 es una de las opciones más frecuentes en la práctica del aprendizaje profundo, ya que ofrece un buen compromiso entre estabilidad del gradiente y velocidad de entrenamiento (Goodfellow, Bengio y Courville 2016).

- **Patience=20.**
    - Este parámetro tiene relación directa con el método "`create_early_stopping()`". La función de este parámetro es asignar cuantas épocas sin mucha modificación, haciendo que, al aumentar este valor, tengan que pasar más épocas de cambio sin que haya un cambio grande.

- **Model_Degrees.**
    - Este parámetro se trata de un array numérico el cual contendrá los números de los grados que se quiera probar. Por ejemplo, si se le pasa el array `[2, 3, 4, 5].` Probará a realizar el entrenamiento de cada polinomio con los grados selecionados. Se han creado un array para cada modelo que tenemos (Legendre, Chebyshev y Shmaliy) para el caso de querer aplicar diferentes grados a cada uno.

- **Shmaliy_N_candidate.**
    - Se trata de un vector de valores para el parámetro de soporte discreto "N" de la capa Shmaliy. Dado este vector se realiza una búsqueda sobre estos candidatos para identificar la configuración que mejor se adapta a cada *dataset* como se había explicado en el apartado.

5. Tecnologías utilizadas.

Para el desarrollo de esta nueva arquitectura, se han utilizado diversas herramientas informáticas. En este capítulo se detallará el software, la infraestructura de hardware y las librerías utilizadas a lo largo de este trabajo, explicando los respectivos usos que han sustentado el diseño, entrenamiento y evaluación de las arquitecturas polinómicas propuestas.

5.1. Lenguaje de programación y Entorno de Desarrollo

El desarrollo íntegro de la arquitectura se ha llevado a cabo en el lenguaje de programación Python. Su elección se fundamenta en su posición como estándar de referencia en el ámbito de la inteligencia artificial, destacando por su accesibilidad y su extenso ecosistema de paquetes orientados al cálculo numérico. Una de las librerías más importantes empleadas ha sido TensorFlow, la cual ha proporcionado la infraestructura base para construir la arquitectura polinómica, abstrayendo la implementación de componentes estándar como las funciones de activación o la lógica de los bucles de entrenamiento.

En cuanto al entorno de desarrollo integrado (IDE) hemos utilizado 2 principalmente, los cuales son: “Visual Studio Code” y “Pycharm”. Estos dos IDE entornos de desarrollo se han utilizado en diferentes fases del desarrollo.

Para garantizar la completa reproducibilidad del código y el aislamiento estricto de las dependencias informáticas, todo el ciclo de vida del proyecto se ha encapsulado dentro de un entorno virtual de Conda, creado y configurado de manera específica bajo el nombre de “polynomial_Env”. El uso de este gestor de paquetes ha permitido fijar las versiones exactas de las librerías científicas, evitando conflictos de compatibilidad entre los distintos frameworks y asegurando una ejecución estable e independiente del sistema operativo base.

Durante las fases tempranas del proyecto, la experimentación se realizó en “Visual Studio Code” trabajando sobre cuadernos interactivos de Python (*Jupyter Notebooks*). Este entorno se seleccionó gracias a su amplia compatibilidad con extensiones que facilitan el desarrollo, la detección de errores y la depuración (*debugging*) visual rápida para comprobar el funcionamiento matemático de las capas.

Posteriormente, una vez validados los experimentos y desarrolladas las capas densas personalizadas, el proyecto se migró a “PyCharm”. Al tratarse de un IDE especializado íntegramente en Python y ampliamente utilizado en entornos profesionales, proporcionó las herramientas necesarias para estructurar la arquitectura final, encapsular el código de forma limpia y establecer el entorno de ejecución definitivo para la evaluación de los experimentos y resultados posteriores.

5.2. Framework de Aprendizaje Profundo.

Como se introdujo en el apartado anterior, la infraestructura base de este trabajo se sustenta en TensorFlow. Profundizando en su implementación, la lógica matemática de las nuevas capas de expansión polinómica se ha materializado a través de la API funcional de Keras [Chollet et al., 2015], la cual se ha utilizado para todo el desarrollo y

funcionamiento de neuronas, pesos, funciones de activaciones y otros elementos previamente ya vistos [Abadi et al., 2015].

Para la creación de las arquitecturas propuestas, se ha implementado un diseño orientado a objetos heredando de la clase base "`keras.layers.Layer`". Esta decisión metodológica ha permitido encapsular la inicialización de parámetros espaciales y la reserva del tensor tridimensional de pesos en el método "`build()`", así como la lógica de recurrencia en el método "`call()`". Como se vio en el apartado 3.2. Como resultado, las capas polinómicas operan como componentes modulares que se integran de forma transparente en el flujo secuencial de Keras.

5.3. Infraestructura de Hardware.

Los experimentos y los resultados finales del modelo sobre los 20 conjuntos de datos se han ejecutado en un ordenador de forma local, siendo el mismo equipo que el usado para su desarrollo. A continuación, se van a mencionar las características de dicho equipo electrónico:

- Procesador (CPU): AMD Ryzen 5 5600 de 6 núcleos físicos, operando a una frecuencia base de 3.50 GHz.
- Memoria RAM: 32 GB de memoria física.
- Sistema Operativo: Microsoft Windows 11 Pro (arquitectura basada en x64).
- Versión de Python: V3.13.12.

La disponibilidad de un procesador multinúcleo junto con una amplia cantidad de memoria RAM es un factor clave en el apartado de pruebas de este proyecto ya que dicha infraestructura permitió aprovechar la librería nativa "`multiprocessing`" de Python. Mediante esta técnica de paralelización, el entrenamiento simultáneo de los distintos pliegues (*folds*) de validación se distribuyó entre los hilos de ejecución de la CPU, reduciendo drásticamente el tiempo global de cómputo de la fase experimental sin saturar los recursos del sistema.

Adicionalmente, dado que la instanciación de cientos de modelos a lo largo de las pruebas generaba grafos residuales que saturaban la memoria, se implementó una rutina de limpieza estricta. En cada iteración, se invocó la función de recolección de basura de Python (`gc.collect()`) en conjunto con el vaciado de sesión de Keras (`K.clear_session()`), garantizando un aislamiento total de la memoria RAM entre ejecuciones.

Adicionalmente, dado que la instanciación de cientos de modelos a lo largo de las pruebas generaba grafos residuales que saturaban la memoria, se implementó una rutina de limpieza estricta al término de cada iteración. Para ello, se empleó en primer lugar la función `clear_session()`, una herramienta nativa integrada en el módulo *backend* de la propia librería de Keras (invocada como `K.clear_session()`), la cual está diseñada específicamente para destruir el grafo computacional actual y liberar el estado global de la sesión.

En segundo lugar, esta acción se complementó con la invocación explícita del paquete `gc` (*Garbage Collector*), un módulo de la biblioteca estándar de Python cuya función es la recolección de basura. Mientras que Keras libera sus propios tensores, la llamada a `gc.collect()` instruye a Python para que rastree y elimine inmediatamente de la memoria RAM cualquier objeto o referencia huérfana generada durante el pliegue.

5.4. Otras librerías implementadas.

A parte de las librerías mencionadas en los puntos anteriores, también se han usado otro conjunto de paquetes y librerías que han ayudado en diferentes apartados del desarrollo. A continuación, dividiremos dichos elementos dependiendo de su funcionalidad:

- **Gestión del Sistema.**
    - `os y sys`: Utilizadas para la manipulación segura de rutas de directorios, la creación automatizada de carpetas de salida y el manejo de flujos de entrada/salida a nivel de sistema operativo permitiendo de esta forma trabajar con el ordenador para principalmente generar datos de salida.
- **Monitorización y Tiempos de Ejecución.**
    - `time`: Se utilizó para capturar las marcas de tiempo iniciales y finales del entrenamiento de los modelos, permitiendo calcular empíricamente el tiempo de convergencia en segundos de cada arquitectura polinómica.
    - `datetime`: Empleada para el registro cronológico de los experimentos para de esta forma poder diferenciar las diferentes pruebas de ejecución.
- **Manipulación de Datos y Operaciones Numéricas.**
    - `numpy`: Principal librería para realizar cálculo numérico avanzado y manipulación de matrices [Harris et al., 2020], siendo el responsable de la mayoría de cálculos realizados, junto a la librería **”**`Math`**”.**
- **Preprocesamiento, Validación y Métricas.**
    - Manejo de datos: Se emplearon los módulos **”**`train_test_split`**”** y **“**`StratifiedKFold`” para el gobierno estricto del protocolo de validación cruzada, garantizando el balanceo de clases.
    - Transformaciones: Se hizo uso de “`MinMaxScaler`**”** para forzar un reescalado exacto de las características al dominio [-1, 1], y de “`LabelEncoder`" para la codificación numérica de las etiquetas objetivo.
    - Evaluación (sklearn.metrics): Encargado del cálculo automatizado y riguroso de la exactitud (accuracy_score), la precisión (precision_score), la exhaustividad (recall_score) y la puntuación F1 (f1_score) tras la fase de predicción.
- **Visualización Gráfica:**
    - `Matplotlib [Hunter, 2007]`: Esta librería se empleó para generar muestras visuales de las funciones ortogonales (Legendre, Chebyshev y Shmaliy) como de las curvas de convergencia registradas en los historiales de entrenamiento.

6. Planificación

Para tener un objetivo claro durante la elaboración de este trabajo, se ha seguido una planificación rigurosa para asegurarnos de que todas las metas propuestas en relación con el proyecto se cumplieran en los plazos previstos. A continuación, se detallará cómo se ha estructurado esta planificación, la metodología adoptada y las diferentes fases por las que ha transitado el proyecto hasta su fin.

6.1. Metodología de trabajo

La metodología adoptada para el desarrollo de este Trabajo de Fin de Grado se ha basado en un enfoque orientado a objetivos secuenciales. Para poder avanzar de forma sólida, era requisito indispensable cumplir con los hitos marcados en cada etapa, los cuales dictaban la fase de trabajo en la que nos encontrábamos en cada momento.

En lugar de aplicar un modelo en cascada estricto, se optó por un enfoque iterativo, especialmente durante la etapa de programación. Esto permitió refinar el código y la arquitectura a medida que se obtenían los primeros resultados empíricos, volviendo recurrentemente al apartado de investigación teórica para confirmar empíricamente el comportamiento del modelo y buscar referencias bibliográficas que respaldaran dichos hallazgos.

6.1.1. Entendimiento teórico y práctico.

Esta etapa inicial sentó las bases de la investigación. Estuvo dedicada íntegramente a comprender los fundamentos matemáticos y funcionales necesarios para el proyecto. Se realizó un estudio exhaustivo de los polinomios ortogonales que podrían ser de interés para nuestras pruebas, analizando sus cualidades matemáticas y características, para tener un entendimiento profundo de sus capacidades.

A nivel tecnológico, se requirió una curva de aprendizaje práctica para dominar el funcionamiento interno de la API funcional de Keras y la manipulación de tensores multidimensionales mediante TensorFlow.

6.1.2. Desarrollo de pruebas y modelos base

Tras consolidar la investigación teórica, se procedió a la implementación de los primeros prototipos de la arquitectura. Esta fase fue altamente experimental y requirió varias iteraciones de diseño al igual que recurrir de nuevo a investigación teórica repetidas veces. Se codificaron las primeras capas densas personalizadas instanciando las fórmulas de recurrencia polinómica.

Se probaron diferentes enfoques arquitectónicos, como la asignación de un grado polinómico estricto por neurona correspondiente a la idea original del proyecto. Pero al evaluar esta idea de modelos, se detectó que dicha configuración provocaba alta oscilación y la "muerte" prematura de las neuronas en grados superiores dejando resultados poco prometedores.

Este análisis temprano permitió pivotar hacia la solución final basada en tensores tridimensionales, logrando la estabilidad matemática necesaria.

6.1.3. Desarrollo final y entorno de pruebas.

La última fase consistió en la consolidación del modelo definitivo y la ejecución a gran escala de los experimentos. Se migró el código validado a un entorno profesional (PyCharm) y se configuró el entorno virtual de Conda polynomial_Env para el encapsulado estricto de las versiones y dependencias necesarias.

Se optimizó el flujo de pruebas mediante la distribución de procesos entre los diferentes núcleos del procesador (CPU) empleando la librería multiprocessing. Por último, se extrajeron de forma automatizada las métricas de evaluación (accuracy, tiempo de convergencia, etc.) para su posterior análisis y documentación en la presente memoria. Estos resultados finales son los presentados en el capítulo de Resultados, los cuales han permitido formular el veredicto definitivo sobre la viabilidad técnica de la implementación propuesta.

6.2. Control de versiones y seguimiento.

Para garantizar la correcta evolución del proyecto, se ha establecido un doble mecanismo de seguimiento que abarca tanto la validación académica como la trazabilidad técnica del desarrollo informático.

El primer nivel de control incluyó un sistema de evaluación continua mediante revisiones periódicas sobre lo desarrollado con el tutor del trabajo. Estas sesiones resultaron fundamentales para resolver posibles dudas que podían surgir y otros bloqueos técnicos, e ir aportando nuevos objetivos tanto de investigación como de desarrollo en función a las fases en las que se encontraba el proyecto.

A nivel técnico, la trazabilidad del código y la gestión de las distintas implementaciones se ha llevado a cabo utilizando GitHub como plataforma de control de versiones. Dado el carácter altamente iterativo y experimental del proyecto, esta herramienta ha resultado indispensable para registrar el progreso del desarrollo de forma estructurada. Su uso ha permitido mantener un historial detallado de los cambios en la arquitectura, realizar copias de seguridad continuas en un repositorio remoto y experimentar con nuevas configuraciones polinómicas de manera segura (aislando el código en ramas de desarrollo o branches), garantizando así la integridad del modelo funcional principal en todo momento.

6.3. Estimación de esfuerzo

Tras la realización del trabajo y haber definido las diferentes fases en las que ha pasado el trabajo, se puede estimar un aproximado de cuanto esfuerzo se ha llevado cada uno de los apartados. A continuación, se mostrará una tabla resumen que muestre cuales son los correspondientes porcentajes de trabajo.

Fase del Proyecto	Descripción de la actividad	Esfuerzo estimado

Entendimiento teórico y práctico	Entendimiento teórico, investigación matemática y estudio de la API funcional de Keras.	25 %
Desarrollo de pruebas y modelos base	Programación de modelos base, pruebas de concepto arquitectónicas y refactorización.	40 %
Desarrollo final y entorno de pruebas.	Desarrollo del entorno final, paralelización, ejecución masiva y extracción de métricas.	10 %
Documentación	Redacción de la presente memoria, análisis de resultados y preparación de la defensa.	25 %

Tabla 2. Tabla resumen de esfuerzo por fase del proyecto.

Como se puede observar en la distribución de la tabla, la mayor concentración de esfuerzo se localiza en la fase de desarrollo de pruebas y modelos base (40%), lo cual refleja el carácter fuertemente experimental y de investigador de este proyecto, requiriendo un proceso intensivo de programación, ensayo y refactorización de la arquitectura hasta un modelo viable.

Por otra parte, el entendimiento teórico inicial y la redacción de la documentación comparten un peso del 25% cada una, lo que garantiza un equilibrio idóneo entre la solidez conceptual del modelo y el posterior rigor académico exigido en la memoria escrita.

Finalmente, el desarrollo final y entorno de pruebas representa el 10% restante, un porcentaje bajo en comparación a los otros procesos, debido a que la automatización del flujo de ejecución y la paralelización por multiprocesamiento permitieron despachar de forma ágil la computación masiva de los pliegues una vez resueltos los desafíos estructurales previos.

7. Resultados

Tras haber expuesto la configuración experimental completa, este capítulo presenta e interpreta los resultados obtenidos. El análisis se articula en tres bloques: el estudio de la exactitud (*Accuracy*), la velocidad de convergencia y el conjunto de métricas complementarias (precisión, recall y F1-score).

7.1. Análisis de los modelos.

Este apartado es de los más importantes ya que es la principal métrica de referencia que se suele tener en cuenta a la hora de comparar modelos. A continuación, se presenta el promedio global de esta métrica, calculado sobre los 27 *dataset*s y sobre todos los grados evaluados para cada familia polinómica:

Modelo	***Accuracy* Promedio (Entre todos los grados)**
Lineal	0,827273664
Legendre	0,819380632
Chebyshev	0,819132298
Shmaliy	0,817059772

Tabla 3. Promedio de los resultados del análisis modelos sobre los 27 datasets.

A primera vista, se observa que el accuracy promedio global sitúa al modelo Lineal ligeramente por encima de las arquitecturas polinómicas (0,827 frente a aproximadamente 0,819). Sin embargo, al analizar exclusivamente los modelos experimentales entre sí, los resultados son prácticamente idénticos, destacando el modelo de Legendre como la base polinómica con mejor rendimiento promedio global, superando tanto a Chebyshev como a Shmaliy.

La interpretación de porqué Legendre conseguido un resultado mayor a los otros polinomios puede estar directamente relacionada a una característica explicada en el apartado 2.3: La función de peso.

A diferencia por ejemple, de la función polinómica de Chebyshev, cuya función de peso enfatiza los extremos del intervalo en su dominio ($w(x) = (1 - x^2)^{\frac{-1}{2}}$), Legendre emplea una función de peso uniforme (siendo esta: $w(x) = 1$).

Esta característica es fundamental en el contexto de los *dataset*s analizados; Dado que, tras la normalización, la densidad de muestras suele concentrarse en las regiones centrales del intervalo (fenómeno relacionado con la distribución normal de muchas variables empíricas (Gautschi 2004)), la arquitectura de Legendre genera fronteras de decisión más suaves y robustas.

Sin embargo, el análisis del promedio global no es suficiente. La verdadera ventaja competitiva de estas capas reside en su capacidad de adaptación mediante el ajuste del grado polinómico (n). Dado que promediar todos los grados (desde n=2 hasta n=8) diluye el rendimiento real de los puntos de operación óptimos frente a los grados con

sobreajuste, es fundamental desglosar los resultados para identificar en qué nivel de expansión cada base alcanza su máximo potencial, evitando así simplificar los resultados de forma excesiva.

7.1.1. Análisis del accuracy por grado.

El análisis separado por grado polinómico revela información cualitativa que el promedio global no puede capturar. Para este experimento se han evaluado los grados del 2 al 8:

	Grados (n)						
Modelo	2	3	4	5	6	7	8
Legendre	0,833566	0,834498	0,823757	0,821198	0,812471	0,810026	0,800145
Chebyshev	0,832718	0,832621	0,827218	0,820578	0,81376	0,807391	0,799633
Shmaliy	0,833842	0,831277	0,826581	0,818266	0,815430	0,804711	0,789312

Tabla 4. Resultado del accuracy en función del grado del modelo

- **Punto de operación optimo.**

Analizando la Tabla 4, los resultados revelan que el rendimiento máximo del estudio no se encuentra en el uso del grado más elevado, sino en un punto de equilibrio intermedio correspondiente a las expansiones de baja complejidad.

En el caso de Legendre, se puede apreciar cómo en el grado 3 alcanza el valor de Accuracy más elevado de todas las familias polinómicas (0,834), superando incluso al promedio global que tenía el modelo lineal (0,827). Esto sugiere que este nivel de expansión proporciona la flexibilidad geométrica exacta para modelar las no linealidades de los datasets analizados sin llegar a una excesiva especialización.

Por su parte, la arquitectura de Shmaliy demuestra una eficiencia notable en el nivel de menor complejidad, logrando su rendimiento máximo en el grado 2 (0,8338), superando en ese nivel específico tanto a Chebyshev (0,8327) como a Legendre (0,8335). Este comportamiento es coherente con la naturaleza discreta de los polinomios de Shmaliy (Shmaliy, 2006): la optimización del hiperparámetro N permite filtrar mejor el ruido a muy baja complejidad polinómica, extrayendo características altamente discriminantes con expansiones mínimas.

7.1.2. Fenómeno de degradación por sobreajuste.

Un patrón transversal y crítico identificado en todos los modelos es la caída constante y significativa del rendimiento a medida que se incrementa el grado, tocando fondo al alcanzar el grado 8, donde los promedios descienden a un rango de entre 0,789 y 0,800.

Técnicamente, este fenómeno corresponde a un sobreajuste estructural exacerbado [Goodfellow, Bengio y Courville, 2016]: al elevar el grado a 8, la capacidad matemática de la capa aumenta de forma desproporcionada, permitiendo que el polinomio genere oscilaciones de alta frecuencia que memorizan el ruido y los valores atípicos del conjunto de entrenamiento en lugar de capturar la tendencia general de los datos. Esta pérdida de suavidad en la frontera de decisión se traduce directamente en una menor capacidad de generalización al evaluar datos desconocidos.

7.1.3. Análisis del rendimiento según la naturaleza del *dataset*.

Para finalizar el estudio de exactitud, se han analizado los resultados en función de la naturaleza de los datasets, ya que, como se explicaba en el apartado 4.2, existen particularidades en cada dominio que justifican rendimientos diferenciados. Para ello, se han dividido las propiedades que presentan estos datasets en cuatro fundamentales:

- ***Dataset*s de baja dimensionalidad y volumen reducido. (Iris)**
  - En conjuntos con un número muy limitado de muestras y características, se observa que la introducción de grados altos es puramente contraproducente. Por ejemplo, en el dataset Iris, el modelo Lineal obtiene un resultado sumamente sólido (0,960). Las bases de Chebyshev y Shmaliy logran igualar esa marca perfecta en el grado 2 (0,960), demostrando que la capa polinómica puede emular la estabilidad de la lineal. Sin embargo, al forzar la red hasta el grado 8, la precisión en Chebyshev cae notablemente al 0,920. Esto confirma que en datasets reducidos, la inmensa flexibilidad geométrica de un polinomio de alto grado provoca que la red "memorice" la posición exacta de las escasas muestras disponibles sin llegar a un sobreajuste, destruyendo la visión general de la frontera de decisión.

- ***Dataset*s extensos y de alta densidad (Dry Bean y Rice)**
  - En conjuntos de datos con miles de muestras, el comportamiento de las bases cambia a su favor. En el caso de Rice, el modelo Lineal (0,928) es superado sutil, pero de forma consistente, por la base polinómica de Legendre, la cual alcanza un 0,929 en su grado óptimo 3. Esto demuestra que, cuando el modelo dispone de suficientes muestras para sustentar y entrenar adecuadamente el incremento de tensores, los polinomios ortogonales logran trazar fronteras de decisión mucho más precisas.

- ***Dataset*s con alta presencia de ruido y Outliers (Breast Cancer Coimbra)**
  - Mientras que el modelo Lineal obtiene un modesto 0,705 en este dataset caracterizado por su gran dispersión clínica, Chebyshev en grado 2 dispara la exactitud hasta un sobresaliente 0,799, seguido de Shmaliy (0,782). Esta significativa mejora refleja cómo la propiedad Minimax de Chebyshev y el soporte discreto de puntos de Shmaliy actúan como filtros naturales, ignorando el ruido que habitualmente confunde a la red neuronal estándar.

- ***Dataset*s Multiclase complejos (Caso: Wine Quality)**
  - En tareas de alta ambigüedad donde las diferencias entre clases son tenues (como diferenciar calidades de vino), los polinomios destacan claramente. En Wine Quality White, Chebyshev en grado 3 alcanza un 0,562, superando sólidamente el 0,551 del modelo Lineal. La ventaja de las arquitecturas propuestas reside aquí en la proyección no lineal de las características; la red es capaz de capturar la progresión de las variables químicas con un matiz curvo que la capa lineal estándar simplifica en exceso, evitando así la pérdida de información vital para la separación de las fronteras multiclase.

7.1.4. Conclusión.

La comparativa por grados permite concluir que Legendre posee una mayor resiliencia en las primeras etapas de expansión funcional. Mientras que Chebyshev y Shmaliy inician su descenso de rendimiento casi inmediatamente tras superar el grado 2, Legendre es capaz de escalar positivamente y encontrar su máximo de generalización en el grado 3. Este comportamiento se atribuye a su función de peso uniforme, que permite que el incremento de grado distribuya la capacidad de aprendizaje de manera equitativa en todo el dominio [-1,1], a diferencia de Chebyshev, cuya sensibilidad en los extremos del intervalo puede volver al modelo más volátil ante expansiones de orden superior

En conclusión, los datos empíricos demuestran que el éxito de las redes polinómicas no reside en la maximización del grado, sino en la identificación de un punto de operación óptimo, el cual se sitúa de forma generalizada entre los grados 2 y 3. Es precisamente en este estrecho margen donde la capacidad de aproximación geométrica y la robustez estructural del modelo alcanzan un equilibrio ideal, previniendo el sobreajuste y adaptándose eficientemente a la complejidad intrínseca del problema.

7.2. Velocidad de convergencia.

Más allá de la viabilidad de un modelo por su métrica de validación, también es fundamental considerar otras métricas como se vio en el apartado 6.4, siendo muy importante la eficiencia computacional. A continuación, se analizan los tiempos promedio de entrenamiento.

Modelo	Tiempo Promedio (s)
Lineal	15,57113637
Legendre	10,07136312
Chebyshev	9,278222056
Shmaliy	8,944439621

Tabla 5. Tiempo promedio de entrenamiento por modelo (promedio sobre los 27 *dataset*s y todos los grados evaluados).

Como se puede apreciar en la tabla anterior, el promedio de ejecución sobre los 27 conjuntos de datos analizados revela una jerarquía clara en términos de velocidad. Mientras que el modelo Lineal requiere un promedio de 15,571 segundos por ejecución, las arquitecturas polinómicas logran completar el proceso en tiempos notablemente inferiores.

En este análisis masivo, el modelo de Shmaliy destaca como la arquitectura más eficiente computacionalmente, alcanzando la convergencia en un promedio de tan solo 8,944 segundos. Esto supone una drástica mejora del 42,56 % respecto al tiempo del modelo base.

Le sigue muy de cerca el polinomio de Chebyshev, el cual llega a la convergencia en un tiempo promedio de 9,278 segundos, consolidando una mejora del 40,41 % respecto a la capa lineal. Por último, se sitúa el modelo de Legendre; aunque es la familia polinómica que requiere mayor tiempo relativo en esta comparativa (10,071 segundos), sigue superando con gran margen a la arquitectura lineal, reduciendo los tiempos de entrenamiento en un 35,32 %.

Este fenómeno global podría resultar contraintuitivo desde una perspectiva puramente aritmética, ya que las capas polinómicas realizan un número superior de operaciones y exigen un procesamiento tensorial de mayor complejidad que una capa densa convencional. Sin embargo, este comportamiento encuentra su justificación en la riqueza analítica para el hallazgo de gradientes.

En una capa lineal convencional, el gradiente es constante respecto a la entrada, mientras que, en las capas polinómicas, el gradiente es una función del grado y de la posición del dato en el intervalo [-1, 1].

En una capa lineal convencional, el gradiente es constante respecto a la entrada. En contraste, en las capas polinómicas, el gradiente es una función dependiente tanto del grado como de la posición del dato en el intervalo [-1, 1].

Esta abundancia de información geométrica permite al optimizador realizar ajustes de pesos mucho más significativos y precisos en las primeras iteraciones, reduciendo drásticamente el número de épocas necesarias para alcanzar los criterios de parada temprana [Prechelt 1998]. Este fundamento teórico queda respaldado de forma empírica por las curvas de convergencia generadas durante nuestro análisis:

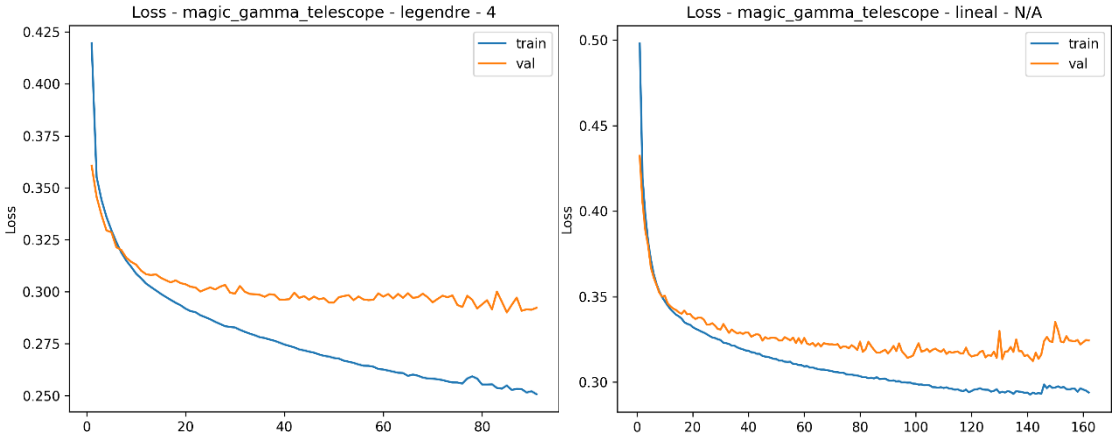


Figura 6: Comparativa de convergencia entre el modelo de Legendre n = 4 contra el lineal

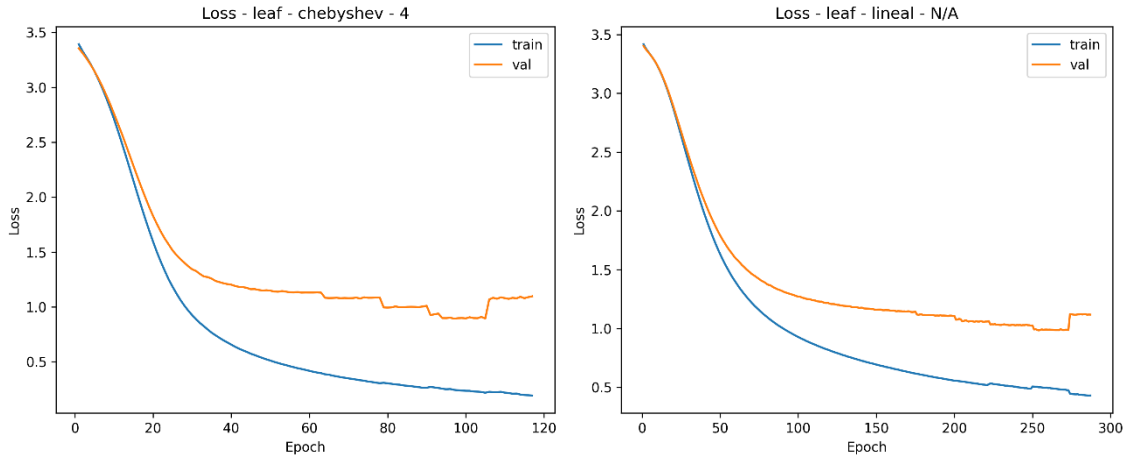


Figura 7: Comparativa de convergencia entre el modelo de Chebyshev n = 4 contra el lineal

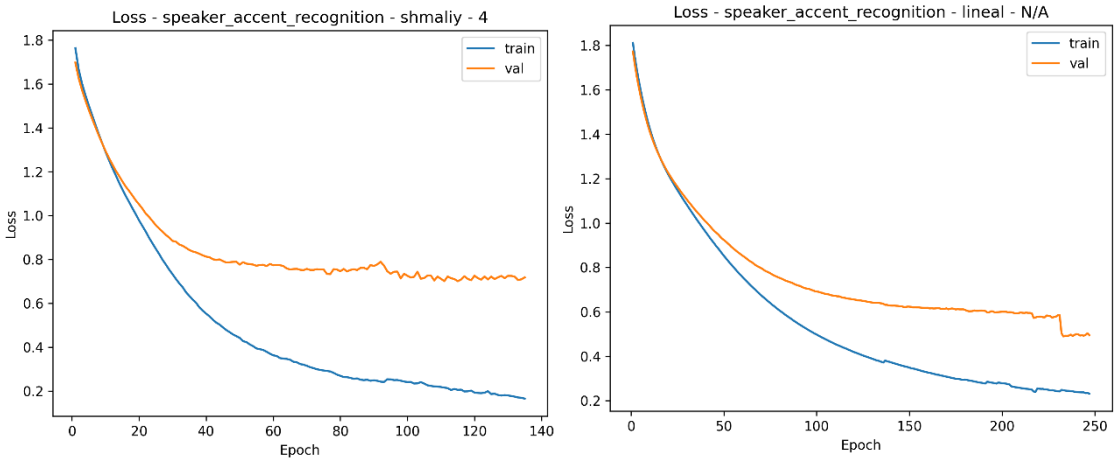


Figura 8: Comparativa de convergencia entre el modelo de Shmaliy n = 4 contra el lineal

Como se aprecia en las gráficas anteriores, se ve como los modelos polinómicos consiguen llegar a la convergencia de forma mucho más rápida que el modelo lineal.

Un análisis adicional por grado polinómico aporta una segunda explicación:

	Tiempo (seg)						
Modelo	2	3	4	5	6	7	8
Legendre	13,99609	11,93569	10,27621	9,433808	9,039099	8,358830	7,459811
Chebyshev	13,27320	10,93632	9,697771	8,225821	8,035148	7,62002	7,159252
Shmaliy	12,43240	10,60303	9,33058	8,543225	7,594608	7,463099	6,644114

Tabla 6. Tiempo promedio de entrenamiento por grado polinómico.

Al observar detalladamente la Tabla 6, se evidencia un patrón consistente en las tres arquitecturas: a medida que se incrementa el grado del polinomio, el tiempo promedio de entrenamiento disminuye de forma progresiva y sustancial. En el nivel de menor complejidad (n=2), los modelos registran tiempos promedios que oscilan entre los 12,43 y los 13,99 segundos. Sin embargo, al alcanzar el grado de máxima expansión evaluado (n=8), estos tiempos se reducen drásticamente, situándose en un rango de entre 6,64 y 7,45 segundos.

A medida que aumenta el grado, el tiempo de entrenamiento se reduce de forma progresiva. Este suceso se explica mediante el Teorema de Cover sobre la separabilidad de los datos [Cover 1965]: al incrementar el grado del polinomio, los datos de entrada se proyectan hacia un espacio de características de dimensionalidad superior. Según este teorema, una configuración compleja de datos que no es linealmente separable en su espacio original tiene una probabilidad significativamente mayor de volverse linealmente separable al ser proyectada de forma no lineal en un espacio de alta dimensión.

En términos prácticos, al facilitar la separabilidad geométrica de las clases mediante la expansión polinómica profunda, el optimizador traza una trayectoria de descenso mucho más directa hacia el mínimo de la función de pérdida. En consecuencia, el modelo satisface los criterios de convergencia y activa los mecanismos de parada temprana (*early stopping*) en un tiempo notablemente inferior, compensando con creces el esfuerzo computacional añadido por el procesamiento tensorial de la capa.

7.3. Otras métricas.

Como se vio en el apartado 2.5.1, además de la métrica de accuracy, existen otras herramientas de evaluación fundamentales para clasificar la calidad y robustez de un modelo, tales como la Precisión, el Recall y el F1-Score.

En este apartado se analizan los resultados obtenidos para estas métricas. A continuación, se expone la tabla que recoge los promedios globales de todas las arquitecturas, calculados sobre la totalidad de los conjuntos de datos y abarcando todos los grados polinómicos evaluados (del 2 al 8):

Modelo	Precision	Recall	F1-Score
Lineal	0,818420723	0,827273664	0,815179906
Legendre	0,807142126	0,819380632	0,804666713
Chebyshev	0,808406234	0,819132298	0,805777377
Shmaliy	0,804251566	0,817059772	0,802159135

Tabla 7. Resultados de precisión, recall y F1-score promedio sobre el conjunto de datos.

7.3.1. Precision y Recall.

Al analizar los promedios globales, se observa que el modelo Lineal presenta una ligera ventaja estadística. Sin embargo, resulta destacable la consistencia que mantienen los modelos polinómicos entre su Precisión y su Recall, a pesar de que sus promedios están fuertemente penalizados por los grados de alto sobreajuste (n=7 y n=8).

Entre las arquitecturas polinómicas, Chebyshev destaca ligeramente como la opción más equilibrada. Este modelo muestra un Recall muy competitivo frente al lineal, lo cual indica una notable capacidad para minimizar los falsos negativos. Este resultado es teóricamente coherente con la propiedad Minimax de su función de peso [Mason y Handscomb, 2002], la cual evita fluctuaciones violentas en los extremos del dominio y permite recuperar muestras críticas que otros modelos descartan. Por su parte, Shmaliy y Legendre mantienen una Precisión sólida, fruto de sus respectivos mecanismos de filtrado espacial y peso uniforme, lo cual es vital en entornos donde una predicción de falso positivo conlleva costes elevados.

7.3.2. F1-score.

Como se detalló en el apartado 2.5.1, el F1-Score es la media armónica de la Precisión y el Recall, por lo que cualquier desequilibrio severo entre estas dos métricas penalizaría drásticamente este indicador.

El promedio global sitúa al modelo lineal en 0,815 y a las familias polinómicas en un rango muy estrecho en torno al 0,805. Esta ligera superioridad global del modelo afín se explica, como se ha argumentado en el análisis de exactitud (apartado 7.1.2), por la severa degradación estructural que sufren las expansiones polinómicas en los grados superiores, las cuales arrastran el promedio matemático hacia abajo.

No obstante, tal y como demuestran los registros, cuando los polinomios operan en su punto de operación óptimo (generalmente en los grados 2 y 3), logran F1-Scores armónicos que superan al modelo lineal, demostrando que su capacidad de curvatura ofrece una representación topológica del dato mucho más rica en escenarios complejos o con desbalanceo de clases.

7.4. Resultados detallados por conjunto de datos.

Para proporcionar una vista más concreta de los resultados, a continuación, se va a mostrar el resultado de cada uno de los datasets con sus correspondientes métricas del grado 2 al 8.

7.4.1. Australian credit.

Accuracy				
Dataset: **Australian credit**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Máximo:	**0,872463768**	**0,871014493**	**0,865217391**	**0,869565217**
2	0,863768116	0,860869565		0,865217391
3	0,866666667	0,871014493		0,862318841
4	0,868115942	0,855072464		0,855072464
5	0,860869565	0,852173913		0,869565217
6	0,850724638	0,862318841		0,85942029
7	0,872463768	0,862318841		0,862318841
8	0,853623188	0,866666667		0,868115942
N/A			0,865217391	

Tabla 8: Tabla de resultado del Accuracy Máximo del Dataset: Australian credit

Tiempo promedio				
Dataset: **Australian credit**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**3,690702483**	**4,015466401**	**3,996859384**	**3,209695847**
2	4,016162324	4,333003545		3,557233262
3	4,139999795	4,183159375		3,337533355
4	3,50071156	3,967286944		3,186629105
5	3,50567565	3,665874982		3,101481104
6	3,812317443	4,163532138		3,237162375
7	3,348249555	3,627392173		2,887206745
8	3,511801052	4,168015647		3,160624981
N/A			3,996859384	

Tabla 9: Tabla de resultado del tiempo promedio del Dataset: Australian credit

F1 Score promedio				
Dataset: **Australian credit**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,86175839**	**0,861043766**	**0,865247989**	**0,862687338**
2	0,863455007	0,860586313		0,864946804
3	0,866760355	0,87081575		0,862240691
4	0,868051657	0,85483633		0,854705162
5	0,860527068	0,852050307		0,869175222
6	0,850263628	0,86230424		0,858809198
7	0,871139795	0,861231596		0,861382312
8	0,85211122	0,865481826		0,867551979
N/A			0,865247989	

Tabla 10: Tabla de resultado del F1 Score promedio del Dataset: Australian credit

Precision promedio				
Dataset: **Australian credit**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,869010676**	**0,868844775**	**0,869674863**	**0,869369235**
2	0,871622805	0,867720923		0,872145828
3	0,871787317	0,877845153		0,867885868
4	0,873972087	0,861264444		0,861674713
5	0,865191415	0,858212715		0,874999223
6	0,857669162	0,867816152		0,866991321
7	0,879929873	0,874309308		0,867714844
8	0,862902071	0,874744729		0,874172847
N/A			0,869674863	

Tabla 11: Tabla de resultado de la Precision promedio del Dataset: Australian credit

Recall promedio				
Dataset: **Australian credit**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,862318841**	**0,861490683**	**0,865217391**	**0,863146998**
2	0,863768116	0,860869565		0,865217391
3	0,866666667	0,871014493		0,862318841
4	0,868115942	0,855072464		0,855072464
5	0,860869565	0,852173913		0,869565217
6	0,850724638	0,862318841		0,85942029
7	0,872463768	0,862318841		0,862318841
8	0,853623188	0,866666667		0,868115942
N/A			0,865217391	

Tabla 12: Tabla de resultado del Recall promedio del Dataset: Australian credit

7.4.2. Breast cancer coimbra.

Accuracy				
Dataset: **breast cancer coimbra**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Máximo:	**0,799242424**	**0,781818182**	**0,70530303**	**0,782575758**
2	0,799242424	0,781818182		0,782575758
3	0,73030303	0,765909091		0,70530303
4	0,68030303	0,687878788		0,697727273
5	0,739393939	0,696969697		0,725757576
6	0,654545455	0,654545455		0,64469697
7	0,609090909	0,628030303		0,584090909
8	0,656060606	0,637878788		0,609848485
N/A			0,70530303	

Tabla 13: Tabla de resultado del Accuracy Máximo del Dataset: breast_cancer_coimbra

Tiempo promedio				
Dataset: **breast cancer coimbra**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**4,323160117**	**4,689116563**	**11,14298162**	**3,522958786**
2	6,140215421	6,51481843		5,011794448
3	5,657309437	5,523457694		4,737636805
4	4,382667685	4,641056943		3,608029103
5	3,788162804	4,602973104		3,150992846
6	3,48152957	3,583903575		2,852005768
7	3,270071959	3,934557033		2,770886564
8	3,542163944	4,023049164		2,529365969
N/A			11,14298162	

Tabla 14: Tabla de resultado del tiempo promedio del Dataset: breast_cancer_coimbra

F1 Score promedio				
Dataset: **breast cancer coimbra**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,685058083**	**0,680906325**	**0,69192977**	**0,661949608**
2	0,78965886	0,772176342		0,771765781
3	0,724373543	0,760061638		0,683828865
4	0,668078988	0,674429145		0,67522286
5	0,729870176	0,691756022		0,723175343
6	0,632083341	0,618826243		0,624163753
7	0,606068376	0,624364154		0,578795126
8	0,6452733	0,624730732		0,576695527
N/A			0,69192977	

Tabla 15: Tabla de resultado del F1 Score promedio del Dataset: breast_cancer_coimbra

Precision promedio				
Dataset: **breast_cancer_coimbra**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,70480352**	**0,702646362**	**0,744176287**	**0,685213229**
2	0,817888709	0,805352633		0,797769661
3	0,746231962	0,779148629		0,697700216
4	0,682518038	0,702768158		0,741689214
5	0,749797078	0,71251443		0,738271104
6	0,657359307	0,614789562		0,629063853
7	0,616558442	0,633310786		0,596403319
8	0,663271104	0,670640332		0,595595238
N/A			0,744176287	

Tabla 16: Tabla de resultado de la Precision promedio del Dataset: breast_cancer_coimbra

Recall promedio				
Dataset: **breast_cancer_coimbra**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,70480352**	**0,702646362**	**0,744176287**	**0,685213229**
2	0,817888709	0,805352633		0,797769661
3	0,746231962	0,779148629		0,697700216
4	0,682518038	0,702768158		0,741689214
5	0,749797078	0,71251443		0,738271104
6	0,657359307	0,614789562		0,629063853
7	0,616558442	0,633310786		0,596403319
8	0,663271104	0,670640332		0,595595238
N/A			0,744176287	

Tabla 17: Tabla de resultado del Recall promedio del Dataset: breast_cancer_coimbra

7.4.3. Cardiotocography morphologic pattern.

Accuracy				
Dataset: **Cardiotocography morphologic pattern**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Máximo:	**0,856050137**	**0,85934538**	**0,818894499**	**0,853226592**
2	0,838185402	0,840067765		0,840058907
3	0,856050137	0,85934538		0,853226592
4	0,853228807	0,852759323		0,847599433
5	0,846177695	0,848558331		0,847608291
6	0,847110019	0,845234299		0,840519532
7	0,821250775	0,812299584		0,826902294
8	0,809531402	0,816562583		0,809478253
N/A			0,818894499	

Tabla 18: Tabla de resultado del Accuracy Máximo del Dataset: Cardiotocography morphologic pattern

Tiempo promedio				
Dataset: **Cardiotocography morphologic pattern**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**6,53512702**	**7,586352757**	**17,61171701**	**6,15901053**
2	9,960510373	11,52243838		9,205090022
3	8,150207067	9,539134383		7,975319099
4	6,828527617	8,035992408		6,407151556
5	5,868252468	6,922482395		5,807375002
6	5,814853811	6,806464386		5,020358729
7	4,629168963	5,256979632		4,495668626
8	4,494368839	5,020977712		4,202110672
N/A			17,61171701	

Tabla 19: Tabla de resultado del tiempo promedio del Dataset: Cardiotocography morphologic pattern

F1 Score promedio				
Dataset: **Cardiotocography morphologic pattern**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,833999551**	**0,834710075**	**0,816049426**	**0,83321698**
2	0,836499061	0,838762822		0,837681626
3	0,853951804	0,857663034		0,851284712
4	0,851581533	0,850666417		0,844365821
5	0,84374509	0,845543258		0,845001681
6	0,843021958	0,842105294		0,836512176
7	0,811426035	0,803215209		0,819074093
8	0,797771379	0,805014495		0,798598751
N/A			0,816049426	

Tabla 20: Tabla de resultado del F1 Score promedio del Dataset: Cardiotocography morphologic pattern

Precision promedio				
Dataset: **Cardiotocography morphologic pattern**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,844222372**	**0,84573705**	**0,826921791**	**0,844143658**
2	0,847982553	0,849071194		0,846812761
3	0,866037946	0,86547307		0,861146229
4	0,858458787	0,859898311		0,855353621
5	0,853172053	0,854401602		0,856842198
6	0,850437038	0,853170212		0,849847541
7	0,828569062	0,815752986		0,829903381
8	0,804899162	0,822391977		0,809099879
N/A			0,826921791	

Tabla 21: Tabla de resultado de la Precision promedio del Dataset: Cardiotocography morphologic pattern

Recall promedio				
Dataset: **Cardiotocography morphologic pattern**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Máximo:	**0,838790605**	**0,839261038**	**0,818894499**	**0,837913329**
2	0,838185402	0,840067765		0,840058907
3	0,856050137	0,85934538		0,853226592
4	0,853228807	0,852759323		0,847599433
5	0,846177695	0,848558331		0,847608291
6	0,847110019	0,845234299		0,840519532
7	0,821250775	0,812299584		0,826902294
8	0,809531402	0,816562583		0,809478253
N/A			0,818894499	

Tabla 21: Tabla de resultado del Recall promedio del Dataset: Cardiotocography morphologic pattern

7.4.4. customer_churn.

Accuracy				
Dataset: **Customer churn**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Máximo:	**0,959365079**	**0,960634921**	**0,943492063**	**0,962539683**
2	0,95047619	0,951746032		0,94952381
3	0,954285714	0,954285714		0,956190476
4	0,954285714	0,954285714		0,952380952
5	0,955873016	0,958730159		0,958730159
6	0,959365079	0,96		0,962539683
7	0,958412698	0,960634921		0,959365079
8	0,959047619	0,959365079		0,955238095
N/A			0,943492063	

Tabla 22: Tabla de resultado del Accuracy Máximo del Dataset: Customer churn

Tiempo promedio				
Dataset: **Customer churn**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**11,18311575**	**15,24937488**	**20,02215729**	**11,92174098**
2	18,0953927	20,33881857		15,75592382
3	15,70874605	21,84311886		14,73994734
4	10,10840309	15,27956147		12,3918047
5	9,387330794	13,11652923		12,21353502
6	8,829152822	13,00880826		9,442036462
7	8,140723729	12,01903255		10,57998304
8	8,012061048	11,13975523		8,328956509
N/A			20,02215729	

Tabla 23: Tabla de resultado del tiempo promedio del Dataset: Customer churn

F1 Score promedio				
Dataset: **Customer churn**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,955557471**	**0,95657385**	**0,9423074**	**0,955881721**
2	0,949699905	0,951431916		0,948554074
3	0,954017638	0,953391371		0,956125496
4	0,954098888	0,953405268		0,951654797
5	0,955549573	0,958597643		0,958562555
6	0,959033117	0,960077336		0,96233396
7	0,957978098	0,960055255		0,959106609
8	0,958525077	0,959058159		0,954834557
N/A			0,9423074	

Tabla 24: Tabla de resultado del F1 Score promedio del Dataset: Customer churn

Precision promedio				
Dataset: **Customer churn**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,956400663**	**0,957484889**	**0,943503499**	**0,956800683**
2	0,951318905	0,952668369		0,950435649
3	0,955019897	0,95458855		0,956659784
4	0,954914995	0,954478075		0,953040432
5	0,955948186	0,95939304		0,958955384
6	0,959622299	0,96076855		0,962825124
7	0,958574168	0,960638095		0,95999697
8	0,959406189	0,959859542		0,955691439
N/A			0,943503499	

Tabla 25: Tabla de resultado de la Precision promedio del Dataset: Customer churn

Recall promedio				
Dataset: **Customer churn**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,955963719**	**0,957006803**	**0,943492063**	**0,956281179**
2	0,95047619	0,951746032		0,94952381
3	0,954285714	0,954285714		0,956190476
4	0,954285714	0,954285714		0,952380952
5	0,955873016	0,958730159		0,958730159
6	0,959365079	0,96		0,962539683
7	0,958412698	0,960634921		0,959365079
8	0,959047619	0,959365079		0,955238095
N/A			0,943492063	

Tabla 26: Tabla de resultado del Recall promedio del Dataset: Customer churn

7.4.5. Data banknote authentication.

Accuracy				
Dataset: **Data banknote authentication**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Máximo:	**1**	**1**	**1**	**1**
2	1	1		1
3	1	1		1
4	1	1		1
5	1	1		0,999270073
6	0,999270073	1		1
7	0,999270073	0,999270073		0,999270073
8	0,998545435	0,998540146		0,998540146
N/A			1	

Tabla 27: Tabla de resultado del Accuracy Máximo del Dataset: Data banknote authentication

Tiempo promedio				
Dataset: **Data banknote authentication**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**33,41364004**	**33,99295993**	**39,24588358**	**32,91492566**
2	35,0117027	35,07938759		35,88259928
3	36,797049	34,97535887		34,63538349
4	38,8746434	38,21118772		37,00625029
5	25,52462289	37,1819844		32,72447026
6	35,77672009	36,94025555		34,74762075
7	32,65085902	32,23109579		31,32334321
8	29,25988317	23,3314496		24,08481238
N/A			39,24588358	

Tabla 28: Tabla de resultado del tiempo promedio del Dataset: Data banknote authentication

F1 Score promedio				
Dataset: **Data banknote authentication**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,999583796**	**0,99968741**	**1**	**0,999583213**
2	1	1		1
3	1	1		1
4	1	1		1
5	1	1		0,999270623
6	0,999270623	1		1
7	0,999270623	0,999270623		0,999270623
8	0,998545329	0,998541246		0,998541246
N/A			1	

Tabla 29: Tabla de resultado del F1 Score promedio del Dataset: Data banknote authentication

Precision promedio				
Dataset: **Data banknote authentication**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,999590027**	**0,99969222**	**1**	**0,999589626**
2	1	1		1
3	1	1		1
4	1	1		1
5	1	1		0,999281846
6	0,999281846	1		1
7	0,999281846	0,999281846		0,999281846
8	0,998566499	0,998563692		0,998563692
N/A			1	

Tabla 30: Tabla de resultado de la Precision promedio del Dataset: Data banknote authentication

Recall promedio				
Dataset: **Data banknote authentication**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,999583654**	**0,999687174**	**1**	**0,999582899**
2	1	1		1
3	1	1		1
4	1	1		1
5	1	1		0,999270073
6	0,999270073	1		1
7	0,999270073	0,999270073		0,999270073
8	0,998545435	0,998540146		0,998540146
N/A			1	

Tabla 31: Tabla de resultado del Recall promedio del Dataset: Data banknote authentication

7.4.6. Fertility diagnosis.

Accuracy				
Dataset: **Fertility diagnosis**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Máximo:	**0,88**	**0,88**	**0,89**	**0,88**
2	0,88	0,88		0,88
3	0,87	0,87		0,88
4	0,87	0,87		0,87
5	0,87	0,87		0,88
6	0,88	0,88		0,88
7	0,85	0,86		0,86
8	0,87	0,86		0,87
N/A			0,89	

Tabla 32: Tabla de resultado del Accuracy Máximo del Dataset: Fertility diagnosis

Tiempo promedio				
Dataset: **Fertility diagnosis**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**3,020914493**	**3,098392313**	**5,274811077**	**2,367051891**
2	3,054514003	3,030025196		2,536534691
3	3,054677439	3,301060295		2,255705309
4	2,943294215	3,133459425		2,283982539
5	3,139734316	2,977424121		2,431784678
6	3,019781065	3,052846408		2,398939681
7	2,876282382	3,004918551		2,275047421
8	3,058118033	3,189012194		2,387368917
N/A			5,274811077	

Tabla 33: Tabla de resultado del tiempo promedio del Dataset: Fertility diagnosis

F1 Score promedio				
Dataset: **Fertility diagnosis**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,819069242**	**0,819064327**	**0,839064327**	**0,821855681**
2	0,824327485	0,824327485		0,824327485
3	0,819064327	0,819064327		0,824327485
4	0,819064327	0,819064327		0,819064327
5	0,819064327	0,819064327		0,824327485
6	0,824327485	0,824327485		0,824327485
7	0,808538012	0,81380117		0,81755117
8	0,819098727	0,81380117		0,819064327
N/A			0,839064327	

Tabla 34: Tabla de resultado del F1 Score promedio del Dataset: Fertility diagnosis

Precision promedio				
Dataset: **Fertility diagnosis**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,774888889**	**0,775**	**0,795**	**0,777333333**
2	0,776	0,776		0,776
3	0,775	0,775		0,776
4	0,775	0,775		0,775
5	0,775	0,775		0,776
6	0,776	0,776		0,776
7	0,773	0,774		0,787333333
8	0,774222222	0,774		0,775
N/A			0,795	

Tabla 35: Tabla de resultado de la Precision promedio del Dataset: Fertility diagnosis

Recall promedio				
Dataset: **Fertility diagnosis**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,87**	**0,87**	**0,89**	**0,874285714**
2	0,88	0,88		0,88
3	0,87	0,87		0,88
4	0,87	0,87		0,87
5	0,87	0,87		0,88
6	0,88	0,88		0,88
7	0,85	0,86		0,86
8	0,87	0,86		0,87
N/A			0,89	

Tabla 36: Tabla de resultado del Recall promedio del Dataset: Fertility diagnosis

7.4.7. heart.

Accuracy				
Dataset: **Heart**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Máximo:	**0,855555556**	**0,855555556**	**0,825925926**	**0,851851852**
2	0,837037037	0,840740741		0,851851852
3	0,855555556	0,840740741		0,848148148
4	0,837037037	0,840740741		0,825925926
5	0,844444444	0,848148148		0,844444444
6	0,848148148	0,840740741		0,837037037
7	0,844444444	0,855555556		0,844444444
8	0,82962963	0,825925926		0,818518519
N/A			0,825925926	

Tabla 37: Tabla de resultado del Accuracy Máximo del Dataset: Heart

Tiempo promedio				
Dataset: **Heart**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**2,981899193**	**3,199109772**	**4,275036359**	**2,40907308**
2	3,321784925	3,448741364		2,686895704
3	2,905058241	3,291075349		2,403297305
4	2,924047828	3,064010954		2,385667634
5	2,867981553	3,026237369		2,235789061
6	2,980087805	3,250988865		2,325198793
7	2,974476123	3,102672887		2,236302638
8	2,899857879	3,210041618		2,590360427
N/A			4,275036359	

Tabla 38: Tabla de resultado del tiempo promedio del Dataset: Heart

F1 Score promedio				
Dataset: **Heart**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,8411054**	**0,840379191**	**0,822375698**	**0,837009365**
2	0,835795464	0,840212095		0,850671652
3	0,854868523	0,83909199		0,845834329
4	0,834954629	0,840093636		0,824110187
5	0,843450629	0,846358618		0,842514091
6	0,846617121	0,839162056		0,836509911
7	0,84375944	0,85388914		0,842751753
8	0,828291994	0,823846803		0,816673635
N/A			0,822375698	

Tabla 39: Tabla de resultado del F1 Score promedio del Dataset: Heart

Precision promedio				
Dataset: **Heart**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,849995754**	**0,850595629**	**0,835045597**	**0,847741756**
2	0,846864723	0,846997169		0,859915444
3	0,862361144	0,851742364		0,859586705
4	0,84505682	0,848438261		0,837306651
5	0,85402704	0,854455453		0,855397652
6	0,854550769	0,851299764		0,841815195
7	0,850091695	0,865320087		0,856583279
8	0,837018084	0,835916303		0,823587368
N/A			0,835045597	

Tabla 40: Tabla de resultado de la Precision promedio del Dataset: Heart

Recall promedio				
Dataset: **Heart**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,842328042**	**0,841798942**	**0,825925926**	**0,838624339**
2	0,837037037	0,840740741		0,851851852
3	0,855555556	0,840740741		0,848148148
4	0,837037037	0,840740741		0,825925926
5	0,844444444	0,848148148		0,844444444
6	0,848148148	0,840740741		0,837037037
7	0,844444444	0,855555556		0,844444444
8	0,82962963	0,825925926		0,818518519
N/A			0,825925926	

Tabla 41: Tabla de resultado del Recall promedio del Dataset: Heart

7.4.8. Hepatitis c.

Accuracy				
Dataset: **hepatitis c**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Máximo:	**0,264263372**	**0,262829736**	**0,246236055**	**0,26498801**
2	0,253435512	0,257819831		0,259941612
3	0,243321864	0,259931185		0,239062663
4	0,235413408	0,254149724		0,26498801
5	0,245448858	0,262829736		0,239015744
6	0,238306746	0,255724116		0,246173496
7	0,261349182	0,249869669		0,247711396
8	0,264263372	0,25848191		0,244010009
N/A			0,246236055	

Tabla 42: Tabla de resultado del Accuracy Máximo del Dataset: hepatitis c

Tiempo promedio				
Dataset: **hepatitis c**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**3,139672984**	**3,31260143**	**3,386988473**	**2,535110927**
2	3,105202675	3,173979521		2,417028761
3	3,046053505	3,208914733		2,419842982
4	3,080047798	3,22436657		2,461375189
5	3,161412716	3,286685443		2,52566731
6	3,149070215	3,355975246		2,589417768
7	3,196487498	3,409614706		2,633490491
8	3,239436483	3,528673792		2,698953986
N/A			3,386988473	

Tabla 43: Tabla de resultado del tiempo promedio del Dataset: hepatitis c

F1 Score promedio				
Dataset: **hepatitis c**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,22478557**	**0,229206598**	**0,23950519**	**0,225619364**
2	0,22361848	0,235156061		0,235129613
3	0,222684819	0,245640572		0,219769029
4	0,21462685	0,22842446		0,243657876
5	0,213261493	0,236808757		0,218729515
6	0,223168267	0,230218626		0,222031244
7	0,240959826	0,21346264		0,227508736
8	0,235179251	0,214735066		0,212509534
N/A			0,23950519	

Tabla 44: Tabla de resultado del F1 Score promedio del Dataset: hepatitis c

Precision promedio				
Dataset: **hepatitis c**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,242370901**	**0,249010579**	**0,242699289**	**0,249196173**
2	0,243065313	0,244561386		0,275888303
3	0,242380604	0,258524372		0,248190094
4	0,226486626	0,244175573		0,260479333
5	0,225041778	0,276632657		0,235029803
6	0,237020259	0,243295948		0,242182194
7	0,264035373	0,238038432		0,25014864
8	0,258566355	0,237845686		0,232454844
N/A			0,242699289	

Tabla 45: Tabla de resultado de la Precision promedio del Dataset: hepatitis c

Recall promedio				
Dataset: **hepatitis c**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,248791278**	**0,25697231**	**0,246236055**	**0,248700419**
2	0,253435512	0,257819831		0,259941612
3	0,243321864	0,259931185		0,239062663
4	0,235413408	0,254149724		0,26498801
5	0,245448858	0,262829736		0,239015744
6	0,238306746	0,255724116		0,246173496
7	0,261349182	0,249869669		0,247711396
8	0,264263372	0,25848191		0,244010009
N/A			0,246236055	

Tabla 46: Tabla de resultado del Recall promedio del Dataset: hepatitis c

7.4.9. Iris.

Accuracy				
Dataset: **Iris**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Máximo:	**0,96**	**0,953333333**	**0,96**	**0,96**
2	0,96	0,953333333		0,96
3	0,953333333	0,946666667		0,94
4	0,94	0,946666667		0,946666667
5	0,946666667	0,946666667		0,94
6	0,94	0,946666667		0,94
7	0,96	0,953333333		0,94
8	0,92	0,946666667		0,953333333
N/A			0,96	

Tabla 47: Tabla de resultado del Accuracy Máximo del Dataset: Iris

Tiempo promedio				
Dataset: **Iris**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**9,177120934**	**10,00154112**	**20,06678634**	**8,594206146**
2	13,41216097	13,88159211		12,17762582
3	10,88788867	11,96529462		11,56949356
4	9,32105515	9,869429445		9,132904506
5	9,244364548	10,60172913		7,873701334
6	6,720102978	8,477653337		6,653308058
7	7,537836123	8,703526139		6,98915441
8	7,116438103	6,511563063		5,763255334
N/A			20,06678634	

Tabla 48: Tabla de resultado del tiempo promedio del Dataset: Iris

F1 Score promedio				
Dataset: **Iris**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,944949495**	**0,947650313**	**0,959175084**	**0,944966515**
2	0,959175084	0,952441077		0,959175084
3	0,952020202	0,945707071		0,938973064
4	0,938973064	0,945707071		0,946735302
5	0,945707071	0,945707071		0,938973064
6	0,938973064	0,945707071		0,938973064
7	0,95973064	0,952575758		0,939360269
8	0,92006734	0,945707071		0,952575758
N/A			0,959175084	

Tabla 49: Tabla de resultado del F1 Score promedio del Dataset: Iris

Precision promedio				
Dataset: **Iris**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,953741497**	**0,956428571**	**0,968253968**	**0,954178005**
2	0,968253968	0,962698413		0,968253968
3	0,964285714	0,954365079		0,948809524
4	0,948809524	0,954365079		0,9575
5	0,954365079	0,954365079		0,948809524
6	0,948809524	0,954365079		0,948809524
7	0,964444444	0,96047619		0,946587302
8	0,927222222	0,954365079		0,96047619
N/A			0,968253968	

Tabla 50: Tabla de resultado de la Precision promedio del Dataset: Iris

Recall promedio				
Dataset: **Iris**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,945714286**	**0,948571429**	**0,96**	**0,945714286**
2	0,96	0,953333333		0,96
3	0,953333333	0,946666667		0,94
4	0,94	0,946666667		0,946666667
5	0,946666667	0,946666667		0,94
6	0,94	0,946666667		0,94
7	0,96	0,953333333		0,94
8	0,92	0,946666667		0,953333333
N/A			0,96	

Tabla 51: Tabla de resultado del Recall promedio del Dataset: Iris

7.4.10. Landsat satellite.

Accuracy				
Dataset: **Landsat satellite**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Máximo:	**0,911262715**	**0,911417753**	**0,900848845**	**0,909243357**
2	0,903336698	0,906443496		0,908930383
3	0,904269341	0,90551206		0,905977416
4	0,908305401	0,90504598		0,906754779
5	0,904269583	0,907377588		0,907220859
6	0,908931832	0,911417753		0,90799774
7	0,911262715	0,905981521		0,909243357
8	0,907534316	0,907531177		0,908469132
N/A			0,900848845	

Tabla 52: Tabla de resultado del Accuracy Máximo del Dataset: Landsat satellite

Tiempo promedio				
Dataset: **Landsat satellite**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**13,04969114**	**14,85200287**	**20,34381626**	**13,56338477**
2	22,22389801	22,44707398		24,11577153
3	16,46713603	19,46417997		16,1623472
4	13,26041129	14,69870694		14,20105662
5	11,8718487	13,310078		12,13821511
6	9,439196563	12,85654261		9,37728014
7	9,501748705	10,98985286		9,722770691
8	8,583598709	10,19758575		9,226252103
N/A			20,34381626	

Tabla 53: Tabla de resultado del tiempo promedio del Dataset: Landsat satellite

F1 Score promedio				
Dataset: **Landsat satellite**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,905126758**	**0,905298433**	**0,899029588**	**0,906123481**
2	0,902041064	0,905046703		0,907113994
3	0,902494379	0,904320672		0,904691477
4	0,906444952	0,903346757		0,905562352
5	0,903380846	0,905644274		0,905101897
6	0,907397821	0,90949878		0,905884575
7	0,909139705	0,903601048		0,90761721
8	0,90498854	0,905630796		0,906892863
N/A			0,899029588	

Tabla 54: Tabla de resultado del F1 Score promedio del Dataset: Landsat satellite

Precision promedio				
Dataset: **Landsat satellite**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,906147401**	**0,906316065**	**0,899562346**	**0,906953998**
2	0,902961863	0,905754547		0,907909453
3	0,903410167	0,905013156		0,905507405
4	0,907921414	0,904222249		0,906598358
5	0,904351744	0,906757071		0,905918957
6	0,908172516	0,910093518		0,906598935
7	0,909517813	0,904912158		0,908849731
8	0,906696288	0,907459757		0,907295143
N/A			0,899562346	

Tabla 55: Tabla de resultado de la Precision promedio del Dataset: Landsat satellite

Recall promedio				
Dataset: **Landsat satellite**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,906844269**	**0,907044225**	**0,900848845**	**0,907799095**
2	0,903336698	0,906443496		0,908930383
3	0,904269341	0,90551206		0,905977416
4	0,908305401	0,90504598		0,906754779
5	0,904269583	0,907377588		0,907220859
6	0,908931832	0,911417753		0,90799774
7	0,911262715	0,905981521		0,909243357
8	0,907534316	0,907531177		0,908469132
N/A			0,900848845	

Tabla 56: Tabla de resultado del Recall promedio del Dataset: Landsat satellite

7.4.11. Leaf.

Accuracy				
Dataset: **Leaf**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Máximo:	**0,694117647**	**0,7**	**0,676470588**	**0,697058824**
2	0,65	0,7		0,670588235
3	0,676470588	0,685294118		0,688235294
4	0,679411765	0,676470588		0,697058824
5	0,694117647	0,7		0,694117647
6	0,635294118	0,664705882		0,688235294
7	0,555882353	0,647058824		0,620588235
8	0,561764706	0,594117647		0,529411765
N/A			0,676470588	

Tabla 57: Tabla de resultado del Accuracy Máximo del Dataset: Leaf

Tiempo promedio				
Dataset: **Leaf**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**6,456680676**	**7,943076294**	**17,71106887**	**6,360077517**
2	9,389840746	11,07745311		9,057489562
3	8,108600473	10,62449694		8,535664248
4	6,688037825	8,414343429		6,962435079
5	6,01882534	7,654836631		6,01946528
6	5,69697361	6,550616407		5,135968351
7	4,688393664	5,771498585		4,64849093
8	4,606093073	5,508288956		4,161029172
N/A			17,71106887	

Tabla 58: Tabla de resultado del tiempo promedio del Dataset: Leaf

F1 Score promedio				
Dataset: **Leaf**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,58547419**	**0,620985061**	**0,625504202**	**0,606654662**
2	0,596960784	0,659705882		0,633739496
3	0,629901961	0,64254902		0,644019608
4	0,630980392	0,627156863		0,654313725
5	0,648235294	0,664901961		0,642254902
6	0,59022409	0,611568627		0,640098039
7	0,492016807	0,603627451		0,566568627
8	0,51	0,537385621		0,465588235
N/A			0,625504202	

Tabla 59: Tabla de resultado del F1 Score promedio del Dataset: Leaf

Precision promedio				
Dataset: **Leaf**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,577728758**	**0,61302521**	**0,62004902**	**0,59762605**
2	0,582352941	0,652941176		0,634264706
3	0,623284314	0,635784314		0,638970588
4	0,625	0,61495098		0,655882353
5	0,643137255	0,664705882		0,624509804
6	0,587892157	0,600735294		0,628431373
7	0,479901961	0,595098039		0,551470588
8	0,50253268	0,526960784		0,449852941
N/A			0,62004902	

Tabla 60: Tabla de resultado de la Precision promedio del Dataset: Leaf

Recall promedio				
Dataset: **Leaf**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,636134454**	**0,666806723**	**0,676470588**	**0,655462185**
2	0,65	0,7		0,670588235
3	0,676470588	0,685294118		0,688235294
4	0,679411765	0,676470588		0,697058824
5	0,694117647	0,7		0,694117647
6	0,635294118	0,664705882		0,688235294
7	0,555882353	0,647058824		0,620588235
8	0,561764706	0,594117647		0,529411765
N/A			0,676470588	

Tabla 61: Tabla de resultado del Recall promedio del Dataset: Leaf

7.4.12. Magic gamma telescope.

Accuracy				
Dataset: **Magic gamma telescope**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Máximo:	**0,880336488**	**0,882492114**	**0,868086225**	**0,880757098**
2	0,878706625	0,876550999		0,875341746
3	0,880336488	0,88002103		0,879074658
4	0,880231335	0,882492114		0,880757098
5	0,87849632	0,880704522		0,880231335
6	0,874763407	0,878180862		0,877812829
7	0,876130389	0,877497371		0,877549947
8	0,872187171	0,878969506		0,875867508
N/A			0,868086225	

Tabla 62: Tabla de resultado del Accuracy Máximo del Dataset: Magic gamma telescope

Tiempo promedio				
Dataset: **Magic gamma telescope**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**35,80120968**	**43,05971811**	**58,42810256**	**37,96699837**
2	63,6453115	73,09896061		62,90411437
3	42,20509214	54,97823808		48,50670118
4	40,72208118	45,21561921		39,21515341
5	31,11572676	38,30573347		35,7104286
6	25,12992916	30,89702029		28,80070655
7	24,98136227	30,42940331		26,57486663
8	22,80896478	28,49305182		24,05701787
N/A			58,42810256	

Tabla 63: Tabla de resultado del tiempo promedio del Dataset: Magic gamma telescope

F1 Score promedio				
Dataset: **Magic gamma telescope**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,87484697**	**0,876842679**	**0,865575498**	**0,87565891**
2	0,876068657	0,874017349		0,87266825
3	0,877920827	0,877735822		0,876619716
4	0,878075063	0,880358073		0,878526466
5	0,876250276	0,878514062		0,877764868
6	0,872429923	0,875664528		0,875605734
7	0,873707099	0,874862603		0,875149373
8	0,869476946	0,876746319		0,87327796
N/A			0,865575498	

Tabla 64: Tabla de resultado del F1 Score promedio del Dataset: Magic gamma telescope

Precision promedio				
Dataset: **Magic gamma telescope**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,877581982**	**0,879582983**	**0,867802635**	**0,87845392**
2	0,879492506	0,877093797		0,875941597
3	0,880915853	0,880328904		0,879561409
4	0,880433936	0,88275633		0,881019375
5	0,87861534	0,880967376		0,880821874
6	0,874803545	0,878665014		0,87774075
7	0,876295645	0,878098887		0,877808961
8	0,872517048	0,879170575		0,876283473
N/A			0,867802635	

Tabla 65: Tabla de resultado de la Precision promedio del Dataset: Magic gamma telescope

Recall promedio				
Dataset: **Magic gamma telescope**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,877264534**	**0,879202343**	**0,868086225**	**0,878090732**
2	0,878706625	0,876550999		0,875341746
3	0,880336488	0,88002103		0,879074658
4	0,880231335	0,882492114		0,880757098
5	0,87849632	0,880704522		0,880231335
6	0,874763407	0,878180862		0,877812829
7	0,876130389	0,877497371		0,877549947
8	0,872187171	0,878969506		0,875867508
N/A			0,868086225	

Tabla 66: Tabla de resultado del Recall promedio del Dataset: Magic gamma telescope

7.4.13. Mesothelioma.

Accuracy				
Dataset: **mesothelioma**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Máximo:	**0,993844697**	**1**	**1**	**0,996875**
2	0,993844697	0,99375		0,996875
3	0,99375	1		0,996875
4	0,965625	0,910416667		0,920170455
5	0,953409091	0,950378788		0,975378788
6	0,891382576	0,837121212		0,842613636
7	0,892329545	0,882670455		0,929356061
8	0,965909091	0,947253788		0,903409091
N/A			1	

Tabla 67: Tabla de resultado del Accuracy Máximo del Dataset: mesothelioma

Tiempo promedio				
Dataset: **mesothelioma**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**14,36415818**	**11,3321088**	**24,55440724**	**11,56629238**
2	21,48926613	19,0197367		14,40252085
3	23,41631498	15,19461231		15,7601732
4	13,59597232	9,852567005		10,29123514
5	13,5006326	11,63593078		14,06945682
6	8,280632305	8,895764756		6,883391237
7	9,034059119	9,27487123		10,71282859
8	11,23222983	5,451278806		8,844440818
N/A			24,55440724	

Tabla 68: Tabla de resultado del tiempo promedio del Dataset: mesotelioma

F1 Score promedio				
Dataset: **mesothelioma**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,944797695**	**0,916447832**	**1**	**0,924229079**
2	0,993844704	0,993845029		0,996922515
3	0,993845029	1		0,996922515
4	0,963474079	0,889288362		0,895579893
5	0,948320343	0,946779544		0,975309178
6	0,87777148	0,792240542		0,796974834
7	0,870604159	0,854143103		0,915801122
8	0,965724069	0,938838243		0,892093497
N/A			1	

Tabla 69: Tabla de resultado del F1 Score promedio del Dataset: mesothelioma

Precision promedio				
Dataset: **mesothelioma**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,946548044**	**0,918567898**	**1**	**0,921726718**
2	0,99428346	0,994375		0,9971875
3	0,994375	1		0,9971875
4	0,966672275	0,884418993		0,880116481
5	0,960508166	0,956830636		0,977816044
6	0,873084137	0,781222744		0,793445681
7	0,869845513	0,856649214		0,916758617
8	0,967067759	0,956478698		0,889575203
N/A			1	

Tabla 70: Tabla de resultado de la Precision promedio del Dataset: mesotelioma

Recall promedio				
Dataset: **mesothelioma**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,950892857**	**0,931655844**	**1**	**0,937811147**
2	0,993844697	0,99375		0,996875
3	0,99375	1		0,996875
4	0,965625	0,910416667		0,920170455
5	0,953409091	0,950378788		0,975378788
6	0,891382576	0,837121212		0,842613636
7	0,892329545	0,882670455		0,929356061
8	0,965909091	0,947253788		0,903409091
N/A			1	

Tabla 71: Tabla de resultado del Recall promedio del Dataset: mesotelioma

7.4.14. Movement libras.

Accuracy				
Dataset: **movement libras**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Máximo:	**0,766666667**	**0,766666667**	**0,755555556**	**0,758333333**
2	0,755555556	0,766666667		0,758333333
3	0,766666667	0,75		0,755555556
4	0,752777778	0,747222222		0,755555556
5	0,747222222	0,722222222		0,711111111
6	0,702777778	0,708333333		0,702777778
7	0,694444444	0,680555556		0,677777778
8	0,663888889	0,65		0,647222222
N/A			0,755555556	

Tabla 72: Tabla de resultado del Accuracy Máximo del Dataset: movement libras

Tiempo promedio				
Dataset: **movement libras**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**4,581575922**	**5,084220719**	**10,96291792**	**4,312649505**
2	7,082531548	7,227834535		5,656963253
3	5,780320883	5,693312049		5,30883708
4	4,41292417	5,340499258		4,69465239
5	4,132891536	4,743412828		3,8680125
6	3,921209121	4,478584504		3,992601299
7	3,387665296	4,188442111		3,382267737
8	3,353488898	3,91745975		3,285212278
N/A			10,96291792	

Tabla 73: Tabla de resultado del tiempo promedio del Dataset: movement libras

F1 Score promedio				
Dataset: **movement libras**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,707637801**	**0,697248677**	**0,740099206**	**0,696249055**
2	0,738089827	0,745092593		0,748425926
3	0,754675926	0,735734127		0,735443122
4	0,737328042	0,724583333		0,738769841
5	0,722691799	0,703597884		0,694126984
6	0,685617284	0,686388889		0,677956349
7	0,6752403	0,660092593		0,64890873
8	0,639821429	0,625251323		0,630112434
N/A			0,740099206	

Tabla 74: Tabla de resultado del F1 Score promedio del Dataset: movement libras

Precision promedio				
Dataset: **movement libras**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,745243764**	**0,732542517**	**0,774675926**	**0,733023432**
2	0,765231481	0,770046296		0,786018519
3	0,792916667	0,772037037		0,766481481
4	0,779728836	0,760482804		0,77087963
5	0,746441799	0,741111111		0,736283069
6	0,726818783	0,73125		0,704464286
7	0,72473545	0,690972222		0,688009259
8	0,680833333	0,661898148		0,679027778
N/A			0,774675926	

Tabla 75: Tabla de resultado de la Precision promedio del Dataset: movement libras

Recall promedio				
Dataset: **movement libras**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,726190476**	**0,717857143**	**0,755555556**	**0,71547619**
2	0,755555556	0,766666667		0,758333333
3	0,766666667	0,75		0,755555556
4	0,752777778	0,747222222		0,755555556
5	0,747222222	0,722222222		0,711111111
6	0,702777778	0,708333333		0,702777778
7	0,694444444	0,680555556		0,677777778
8	0,663888889	0,65		0,647222222
N/A			0,755555556	

Tabla 76: Tabla de resultado del Recall promedio del Dataset: movement libras

7.4.15. Musk v1.

Accuracy				
Dataset: **Musk v1**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Máximo:	**0,915913121**	**0,915957447**	**0,907402482**	**0,909618794**
2	0,915913121	0,915957447		0,909618794
3	0,894946809	0,905452128		0,909441489
4	0,903191489	0,888297872		0,901019504
5	0,844237589	0,880008865		0,863253546
6	0,867508865	0,873714539		0,882180851
7	0,846409574	0,899024823		0,877969858
8	0,865248227	0,825753546		0,838342199
N/A			0,907402482	

Tabla 77: Tabla de resultado del Accuracy Máximo del Dataset: Musk v1

Tiempo promedio				
Dataset: **Musk v1**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**3,289192871**	**3,284476665**	**5,426536369**	**2,924051479**
2	4,48206408	4,357165408		3,797890687
3	3,188525057	3,552237344		3,022920585
4	3,232363629	3,108170152		2,715499139
5	2,659691596	3,16153295		2,590952921
6	2,753795123	2,964719343		2,679824853
7	2,816445899	3,031888366		2,69097507
8	3,89146471	2,815623093		2,970297098
N/A			5,426536369	

Tabla 78: Tabla de resultado del tiempo promedio del Dataset: Musk v1

F1 Score promedio				
Dataset: **Musk v1**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,875631277**	**0,882344746**	**0,907331965**	**0,88178045**
2	0,915756433	0,915116026		0,90927547
3	0,893780244	0,904875251		0,908591032
4	0,90219472	0,886745952		0,899762093
5	0,843093397	0,878587187		0,860627154
6	0,86609148	0,871219028		0,880508943
7	0,844949562	0,898485434		0,876584747
8	0,8635531	0,821384344		0,837113714
N/A			0,907331965	

Tabla 79: Tabla de resultado del F1 Score promedio del Dataset: Musk v1

Precision promedio				
Dataset: **Musk v1**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,883058456**	**0,891114994**	**0,910565992**	**0,889874105**
2	0,917430166	0,921012227		0,913156541
3	0,901257994	0,908542465		0,914718352
4	0,908102292	0,894216716		0,905112591
5	0,850902902	0,885875916		0,870903534
6	0,877682499	0,882411613		0,890003699
7	0,853596548	0,902585899		0,885073311
8	0,872436793	0,843160121		0,850150707
N/A			0,910565992	

Tabla 80: Tabla de resultado de la Precision promedio del Dataset: Musk v1

Recall promedio				
Dataset: **Musk v1**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,876779382**	**0,884029889**	**0,907402482**	**0,883118034**
2	0,915913121	0,915957447		0,909618794
3	0,894946809	0,905452128		0,909441489
4	0,903191489	0,888297872		0,901019504
5	0,844237589	0,880008865		0,863253546
6	0,867508865	0,873714539		0,882180851
7	0,846409574	0,899024823		0,877969858
8	0,865248227	0,825753546		0,838342199
N/A			0,907402482	

Tabla 81: Tabla de resultado del Recall promedio del Dataset: Musk v1

7.4.16. Musk v2.

Accuracy				
Dataset: **Musk v2**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Máximo:	**0,996362257**	**0,996362717**	**0,992272038**	**0,995453626**
2	0,995301651	0,995907941		0,995453626
3	0,996362257	0,996362717		0,995453166
4	0,995603991	0,995453166		0,994846646
5	0,99257001	0,992418724		0,993632915
6	0,995604451	0,995149446		0,995150136
7	0,992876489	0,992573688		0,99439256
8	0,986351221	0,987261921		0,988323677
N/A			0,992272038	

Tabla 82: Tabla de resultado del Accuracy Máximo del Dataset: Musk v2

Tiempo promedio				
Dataset: **Musk v2**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**14,27734035**	**14,02968244**	**13,33131375**	**13,45665309**
2	16,37727234	18,05526152		17,13803821
3	13,58305669	15,08945591		13,81882067
4	13,85768311	12,37527027		13,96801441
5	12,10665631	13,22209308		12,7803972
6	17,27964489	15,05788448		13,21016092
7	15,48619604	12,73754125		12,33099706
8	11,25087309	11,67027059		10,95014317
N/A			13,33131375	

Tabla 83: Tabla de resultado del tiempo promedio del Dataset: Musk v2

F1 Score promedio				
Dataset: **Musk v2**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,993441375**	**0,993526999**	**0,992235243**	**0,993900833**
2	0,995313031	0,99591106		0,995450682
3	0,996361563	0,996370652		0,995465538
4	0,995602976	0,995469239		0,994840649
5	0,99245106	0,992301621		0,993625333
6	0,995608851	0,995163085		0,995154867
7	0,992860773	0,992547858		0,99438722
8	0,98589137	0,986925474		0,988381541
N/A			0,992235243	

Tabla 84: Tabla de resultado del F1 Score promedio del Dataset: Musk v2

Precision promedio				
Dataset: **Musk v2**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,993508918**	**0,993608357**	**0,992335751**	**0,993975858**
2	0,995367045	0,99594774		0,995507232
3	0,996381718	0,99643442		0,995530516
4	0,995621896	0,995529457		0,994886103
5	0,992553657	0,992436065		0,993670885
6	0,995641911	0,995222777		0,995180332
7	0,992957074	0,992591664		0,994405459
8	0,986039124	0,987096376		0,98865048
N/A			0,992335751	

Tabla 85: Tabla de resultado de la Precision promedio del Dataset: Musk v2

Recall promedio				
Dataset: **Musk v2**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,993524296**	**0,993589658**	**0,992272038**	**0,993893246**
2	0,995301651	0,995907941		0,995453626
3	0,996362257	0,996362717		0,995453166
4	0,995603991	0,995453166		0,994846646
5	0,99257001	0,992418724		0,993632915
6	0,995604451	0,995149446		0,995150136
7	0,992876489	0,992573688		0,99439256
8	0,986351221	0,987261921		0,988323677
N/A			0,992272038	

Tabla 86: Tabla de resultado del Recall promedio del Dataset: Musk v2

7.4.17. Parkinson.

Accuracy				
Dataset: **Parkinson**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Máximo:	**0,923947368**	**0,913157895**	**0,893421053**	**0,902894737**
2	0,902894737	0,876842105		0,897368421
3	0,892894737	0,887631579		0,887368421
4	0,897894737	0,897894737		0,902894737
5	0,877894737	0,892368421		0,837631579
6	0,887368421	0,888157895		0,902368421
7	0,923947368	0,913157895		0,891842105
8	0,873947368	0,868157895		0,848684211
N/A			0,893421053	

Tabla 87: Tabla de resultado del Accuracy Máximo del Dataset: Parkinson

Tiempo promedio				
Dataset: **Parkinson**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**3,234446641**	**3,588365289**	**10,89091539**	**3,191841098**
2	5,119748878	5,427910352		5,382338905
3	3,498462105	3,835018063		3,519850421
4	2,900892282	3,351780176		2,829850197
5	2,875794578	3,052416062		2,489297652
6	2,71816051	3,174353743		2,874766111
7	2,847531676	3,154906201		2,714522243
8	2,680536461	3,122172427		2,532262158
N/A			10,89091539	

Tabla 88: Tabla de resultado del tiempo promedio del Dataset: Parkinson

F1 Score promedio				
Dataset: **Parkinson**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,886177292**	**0,879532536**	**0,883482985**	**0,869287408**
2	0,899326123	0,870213816		0,891199887
3	0,887910394	0,883557136		0,879897515
4	0,890593096	0,894181663		0,901217546
5	0,860962918	0,88463284		0,811997107
6	0,881310019	0,879434753		0,900711802
7	0,91654533	0,908495537		0,878050253
8	0,866593168	0,836212008		0,821937744
N/A			0,992235243	

Tabla 89: Tabla de resultado del F1 Score promedio del Dataset: Parkinson

Precision promedio				
Dataset: **Parkinson**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,899869951**	**0,886500989**	**0,903222717**	**0,878307847**
2	0,904016383	0,875763674		0,89938274
3	0,899974891	0,892705431		0,898857898
4	0,909222367	0,903393087		0,908421053
5	0,87089493	0,89967976		0,807628262
6	0,90300047	0,894304235		0,91460812
7	0,936716009	0,920121259		0,905336902
8	0,875264604	0,819539474		0,813919956
N/A			0,903222717	

Tabla 90: Tabla de resultado de la Precision promedio del Dataset: Parkinson

Recall promedio				
Dataset: **Parkinson**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,893834586**	**0,889172932**	**0,893421053**	**0,881165414**
2	0,902894737	0,876842105		0,897368421
3	0,892894737	0,887631579		0,887368421
4	0,897894737	0,897894737		0,902894737
5	0,877894737	0,892368421		0,837631579
6	0,887368421	0,888157895		0,902368421
7	0,923947368	0,913157895		0,891842105
8	0,873947368	0,868157895		0,848684211
N/A			0,893421053	

Tabla 91: Tabla de resultado del Recall promedio del Dataset: Parkinson

7.4.18. Phishing websites.

Accuracy				
Dataset: **Phishing websites**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Máximo:	**0,964541006**	**0,964721511**	**0,957483901**	**0,963997774**
2	0,963002545	0,964721511		0,959656256
3	0,964541006	0,96291172		0,963364699
4	0,963727345	0,963274447		0,961645651
5	0,964087945	0,963997938		0,963726772
6	0,962460213	0,961284888		0,96227889
7	0,963092961	0,963636438		0,963997774
8	0,963816533	0,962641045		0,960922651
N/A			0,957483901	

Tabla 92: Tabla de resultado del Accuracy Máximo del Dataset: Phishing websites

Tiempo promedio				
Dataset: **Phishing websites**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**16,78230277**	**17,41495167**	**18,10143082**	**16,41967733**
2	20,38227496	20,34346023		18,32858067
3	16,64908743	16,04230964		16,04593241
4	17,43974128	18,66667204		16,31144142
5	15,85904813	16,17278078		16,85953667
6	15,75805795	16,68999715		14,84782288
7	15,19293022	17,02114513		15,91871359
8	16,19497945	16,96829674		16,62571363
N/A			18,10143082	

Tabla 93: Tabla de resultado del tiempo promedio del Dataset: Phishing websites

F1 Score promedio				
Dataset: **Phishing websites**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,963517185**	**0,963191039**	**0,957471634**	**0,962216247**
2	0,963000112	0,964708263		0,959646864
3	0,964523932	0,962892858		0,963344095
4	0,963704359	0,963250673		0,961648775
5	0,964067468	0,963967578		0,963705827
6	0,962441169	0,961253134		0,962272893
7	0,963080161	0,963634148		0,963996101
8	0,963803096	0,962630622		0,960899174
N/A			0,957471634	

Tabla 94: Tabla de resultado del F1 Score promedio del Dataset: Phishing websites

Precision promedio				
Dataset: **Phishing websites**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,963637215**	**0,963293867**	**0,957652326**	**0,962346373**
2	0,963127167	0,964778994		0,959712746
3	0,964577792	0,962975779		0,963491552
4	0,963830359	0,963310685		0,961753229
5	0,964172063	0,964133408		0,963808268
6	0,962589448	0,961378867		0,962457997
7	0,963208348	0,963752164		0,964094512
8	0,963955328	0,962727169		0,961106308
N/A			0,957652326	

Tabla 95: Tabla de resultado de la Precision promedio del Dataset: Phishing websites

Recall promedio				
Dataset: **Phishing websites**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,96353265**	**0,963209712**	**0,957483901**	**0,962227528**
2	0,963002545	0,964721511		0,959656256
3	0,964541006	0,96291172		0,963364699
4	0,963727345	0,963274447		0,961645651
5	0,964087945	0,963997938		0,963726772
6	0,962460213	0,961284888		0,96227889
7	0,963092961	0,963636438		0,963997774
8	0,963816533	0,962641045		0,960922651
N/A			0,957483901	

Tabla 96: Tabla de resultado del Recall promedio del Dataset: Phishing websites

7.4.19. Planning relax.

Accuracy				
Dataset: **Planning relax**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Máximo:	**0,709064327**	**0,719590643**	**0,692397661**	**0,714619883**
2	0,697953216	0,709064327		0,709064327
3	0,703508772	0,709064327		0,709064327
4	0,709064327	0,714619883		0,714619883
5	0,70877193	0,709064327		0,714619883
6	0,692397661	0,719590643		0,697953216
7	0,675730994	0,709064327		0,709064327
8	0,664912281	0,692105263		0,703216374
N/A			0,692397661	

Tabla 97: Tabla de resultado del Accuracy Máximo del Dataset: Planning relax

Tiempo promedio				
Dataset: **Planning relax**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**2,226573368**	**2,344925809**	**3,073091316**	**2,025444879**
2	2,447890878	2,310992861		2,047517252
3	2,284802365	2,340837073		1,905993581
4	2,106892896	2,311226439		1,953250718
5	2,105362177	2,258526874		1,952042294
6	2,128944683	2,418038821		2,039591408
7	2,170728421	2,372286344		2,121830368
8	2,341392159	2,40257225		2,157888532
N/A			3,073091316	

Tabla 98: Tabla de resultado del tiempo promedio del Dataset: Planning relax

F1 Score promedio				
Dataset: **Planning relax**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,591127391**	**0,596300406**	**0,583346696**	**0,596770635**
2	0,586785849	0,592976325		0,592976325
3	0,590180626	0,592976325		0,592976325
4	0,592976325	0,595772024		0,595772024
5	0,600996476	0,592976325		0,6042547
6	0,584396422	0,614608025		0,593297626
7	0,58244972	0,592976325		0,592976325
8	0,600106319	0,591817494		0,605141117
N/A			0,583346696	

Tabla 99: Tabla de resultado del F1 Score promedio del Dataset: Planning relax

Precision promedio				
Dataset: **Planning relax**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,526275644**	**0,523786856**	**0,505180837**	**0,522371924**
2	0,506900414	0,509732658		0,509732658
3	0,508552556	0,509732658		0,509732658
4	0,509732658	0,510912759		0,510912759
5	0,542732304	0,509732658		0,5405787
6	0,50604484	0,578092153		0,518614285
7	0,519304777	0,509732658		0,509732658
8	0,590661959	0,538572445		0,557299751
N/A			0,505180837	

Tabla 100: Tabla de resultado de la Precision promedio del Dataset: Planning relax

Recall promedio				
Dataset: **Planning relax**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,693191312**	**0,708939014**	**0,692397661**	**0,708228906**
2	0,697953216	0,709064327		0,709064327
3	0,703508772	0,709064327		0,709064327
4	0,709064327	0,714619883		0,714619883
5	0,70877193	0,709064327		0,714619883
6	0,692397661	0,719590643		0,697953216
7	0,675730994	0,709064327		0,709064327
8	0,664912281	0,692105263		0,703216374
N/A			0,692397661	

Tabla 101: Tabla de resultado del Recall promedio del Dataset: Planning relax

7.4.20. Rice cammeo osmancik.

Accuracy				
Dataset: **Rice cammeo osmancik**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Máximo:	**0,928608924**	**0,92992126**	**0,928871391**	**0,929133858**
2	0,928608924	0,929133858		0,929133858
3	0,927559055	0,92992126		0,928608924
4	0,928608924	0,929133858		0,928871391
5	0,926771654	0,928871391		0,928608924
6	0,927821522	0,926246719		0,928608924
7	0,928346457	0,929396325		0,927296588
8	0,928346457	0,926509186		0,926771654
N/A			0,928871391	

Tabla 102: Tabla de resultado del Accuracy Máximo del Dataset: Rice cammeo osmancik

Tiempo promedio				
Dataset: **Rice cammeo osmancik**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**6,090126531**	**6,427919211**	**6,729883385**	**5,81575095**
2	7,168748593	6,045185518		5,507878351
3	6,919962883	7,473529387		7,434738636
4	7,001038384	7,584414005		6,655702233
5	6,457400441	6,648624992		5,630368686
6	5,368355536	6,478859401		5,404231668
7	5,189843965	5,355693269		5,484820104
8	4,525535917	5,409127903		4,592516971
N/A			6,729883385	

Tabla 103: Tabla de resultado del tiempo promedio del Dataset: Rice cammeo osmancik

F1 Score promedio				
Dataset: **Rice cammeo osmancik**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,927961741**	**0,928408329**	**0,928833189**	**0,928217618**
2	0,928568054	0,929070984		0,929095855
3	0,927530533	0,929840731		0,928580813
4	0,928575017	0,929069998		0,928829719
5	0,926749586	0,928842344		0,928554108
6	0,927754173	0,92620467		0,928525074
7	0,928252165	0,929368344		0,92726862
8	0,928302662	0,926461228		0,926669134
N/A			0,928833189	

Tabla 104: Tabla de resultado del F1 Score promedio del Dataset: Rice cammeo osmancik

Precision promedio				
Dataset: **Rice cammeo osmancik**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,928458948**	**0,928897393**	**0,929255576**	**0,928685652**
2	0,929097492	0,929658628		0,929478237
3	0,928124354	0,930327333		0,929166929
4	0,92890266	0,929491273		0,929221193
5	0,927374017	0,929324564		0,929143615
6	0,928308335	0,926748211		0,929103073
7	0,928723065	0,929777879		0,927629069
8	0,928682709	0,926953866		0,927057449
N/A			0,929255576	

Tabla 105: Tabla de resultado de la Precision promedio del Dataset: Rice cammeo osmancik

Recall promedio				
Dataset: **Rice cammeo osmancik**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,928008999**	**0,928458943**	**0,928871391**	**0,928271466**
2	0,928608924	0,929133858		0,929133858
3	0,927559055	0,92992126		0,928608924
4	0,928608924	0,929133858		0,928871391
5	0,926771654	0,928871391		0,928608924
6	0,927821522	0,926246719		0,928608924
7	0,928346457	0,929396325		0,927296588
8	0,928346457	0,926509186		0,926771654
N/A			0,928871391	

Tabla 106: Tabla de resultado del Recall promedio del Dataset: Rice cammeo osmancik

7.4.21. Room occupancy.

Accuracy				
Dataset: **Room occupancy**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Máximo:	**0,996446004**	**0,996544819**	**0,995063873**	**0,996248668**
2	0,995261209	0,99486644		0,99486644
3	0,995162395	0,995360023		0,995360023
4	0,996050552	0,996050942		0,996248668
5	0,996347093	0,995754792		0,995557359
6	0,995557359	0,995952226		0,995656076
7	0,996248571	0,99634719		0,996149757
8	0,996446004	0,996544819		0,996248668
N/A			0,995063873	

Tabla 107: Tabla de resultado del Accuracy Máximo del Dataset: Room occupancy

Tiempo promedio				
Dataset: **Room occupancy**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**11,12046282**	**12,6386277**	**16,31160836**	**10,91318632**
2	14,54449146	17,61821823		12,86235116
3	10,32273161	13,73282659		11,35034583
4	12,12071903	12,53634131		10,96813934
5	11,65153904	11,7658232		12,19431684
6	9,717461443	11,55811648		9,517705798
7	10,28221457	10,57278295		10,2275598
8	9,204082608	10,68628516		9,271885443
N/A			16,31160836	

Tabla 108: Tabla de resultado del tiempo promedio del Dataset: Room occupancy

F1 Score promedio				
Dataset: **Room occupancy**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,995870316**	**0,995842152**	**0,995067283**	**0,995728433**
2	0,995265703	0,994870589		0,994866268
3	0,995165694	0,99535889		0,995363114
4	0,99605072	0,996051216		0,996249793
5	0,99634789	0,995758527		0,995557445
6	0,995556661	0,995953083		0,99565673
7	0,996253633	0,996353344		0,996154457
8	0,996451909	0,996549416		0,996251225
N/A			0,995067283	

Tabla 109: Tabla de resultado del F1 Score promedio del Dataset: Room occupancy

Precision promedio				
Dataset: **Room occupancy**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,995953655**	**0,995924908**	**0,995135947**	**0,995818329**
2	0,995349378	0,994971648		0,99493457
3	0,995280188	0,995382161		0,995416752
4	0,996076165	0,996140212		0,996345423
5	0,996434101	0,995888528		0,995684106
6	0,995643745	0,996035853		0,995805214
7	0,996331856	0,99643193		0,996228532
8	0,996560151	0,996624022		0,996313709
N/A			0,995135947	

Tabla 110: Tabla de resultado de la Precision promedio del Dataset: Room occupancy

Recall promedio				
Dataset: **Room occupancy**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,995867598**	**0,99583949**	**0,995063873**	**0,995726713**
2	0,995261209	0,99486644		0,99486644
3	0,995162395	0,995360023		0,995360023
4	0,996050552	0,996050942		0,996248668
5	0,996347093	0,995754792		0,995557359
6	0,995557359	0,995952226		0,995656076
7	0,996248571	0,99634719		0,996149757
8	0,996446004	0,996544819		0,996248668
N/A			0,995063873	

Tabla 111: Tabla de resultado del Recall promedio del Dataset: Room occupancy

7.4.22. Sonar.

Accuracy				
Dataset: **Sonar**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Máximo:	**0,841428571**	**0,822619048**	**0,812380952**	**0,831904762**
2	0,831666667	0,822142857		0,831666667
3	0,817619048	0,822619048		0,831904762
4	0,841428571	0,779285714		0,793571429
5	0,754047619	0,783809524		0,725238095
6	0,802857143	0,750238095		0,808095238
7	0,759761905	0,784285714		0,715952381
8	0,696428571	0,702857143		0,702142857
N/A			0,812380952	

Tabla 112: Tabla de resultado del Accuracy Máximo del Dataset: Sonar

Tiempo promedio				
Dataset: **Sonar**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**2,894613978**	**2,813268515**	**4,754493475**	**2,431988944**
2	2,710709119	2,871457481		2,53546505
3	2,780993414	2,944626546		2,566842914
4	2,890028644	3,124390173		2,433462143
5	2,6117239	2,713201356		2,186434317
6	3,875443053	2,770472169		2,495625281
7	2,364127207	2,669249439		2,268686604
8	3,029272509	2,599482441		2,537406301
N/A			4,754493475	

Tabla 113: Tabla de resultado del tiempo promedio del Dataset: Sonar

F1 Score promedio				
Dataset: **Sonar**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,781123529**	**0,772341669**	**0,81096749**	**0,766744086**
2	0,830870489	0,821524155		0,831577065
3	0,815315881	0,819716699		0,830495362
4	0,839859694	0,77865889		0,78995715
5	0,734153988	0,77795187		0,715108998
6	0,800412556	0,737387797		0,806643409
7	0,757615844	0,777819126		0,710773145
8	0,68963625	0,693333147		0,682653474
N/A			0,81096749	

Tabla 114: Tabla de resultado del F1 Score promedio del Dataset: Sonar

Precision promedio				
Dataset: **Sonar**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,7916056**	**0,786769239**	**0,821904762**	**0,776167083**
2	0,837279609	0,827502627		0,835098365
3	0,825885226	0,829572673		0,837775512
4	0,850127122	0,788964604		0,812685804
5	0,752005322	0,797408786		0,731088602
6	0,809612901	0,745416227		0,813761179
7	0,767298462	0,796187985		0,717594572
8	0,699030558	0,722331775		0,685165546
N/A			0,821904762	

Tabla 115: Tabla de resultado de la Precision promedio del Dataset: Sonar

Recall promedio				
Dataset: **Sonar**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,786258503**	**0,777891156**	**0,812380952**	**0,772653061**
2	0,831666667	0,822142857		0,831666667
3	0,817619048	0,822619048		0,831904762
4	0,841428571	0,779285714		0,793571429
5	0,754047619	0,783809524		0,725238095
6	0,802857143	0,750238095		0,808095238
7	0,759761905	0,784285714		0,715952381
8	0,696428571	0,702857143		0,702142857
N/A			0,812380952	

Tabla 116: Tabla de resultado del Recall promedio del Dataset: Sonar

7.4.23. Speaker accent recognition.

Accuracy				
Dataset: **Speaker accent recognition**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Máximo:	**0,759848485**	**0,753787879**	**0,787215909**	**0,753787879**
2	0,750757576	0,747727273		0,732575758
3	0,759848485	0,753787879		0,750568182
4	0,732954545	0,75094697		0,753787879
5	0,717329545	0,683901515		0,717329545
6	0,680587121	0,671780303		0,711458333
7	0,702367424	0,641477273		0,64782197
8	0,647537879	0,659753788		0,590151515
N/A			0,787215909	

Tabla 117: Tabla de resultado del Accuracy Máximo del Dataset: Speaker accent recognition

Tiempo promedio				
Dataset: **Speaker accent recognition**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**5,81280319**	**5,864365254**	**13,01592381**	**5,470386989**
2	10,72232988	9,304071164		8,825907564
3	7,715158772	7,411822271		7,802516556
4	5,470681739	6,092457533		5,674448252
5	4,889198399	4,796917391		4,78509798
6	4,581484556	5,157049441		4,573622084
7	3,829672933	4,325204802		3,532884812
8	3,481096053	3,963034177		3,098231673
N/A			13,01592381	

Tabla 118: Tabla de resultado del tiempo promedio del Dataset: Speaker accent recognition

F1 Score promedio				
Dataset: **Speaker accent recognition**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,700708276**	**0,681648866**	**0,777941912**	**0,680857611**
2	0,739691897	0,735813686		0,721504418
3	0,757992896	0,742454625		0,744809413
4	0,730697124	0,745589369		0,737369371
5	0,706500688	0,67174418		0,707781068
6	0,671464434	0,649462556		0,694797504
7	0,680795741	0,609545651		0,626151671
8	0,617815152	0,616931996		0,53358983
N/A			0,777941912	

Tabla 119: Tabla de resultado del F1 Score promedio del Dataset: Speaker accent recognition

Precision promedio				
Dataset: **Speaker accent recognition**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,728151709**	**0,702552154**	**0,814291586**	**0,701026744**
2	0,760096422	0,751479532		0,761267931
3	0,782926404	0,770285396		0,771560328
4	0,76192081	0,774855021		0,759784575
5	0,734186441	0,696042062		0,728286531
6	0,715669192	0,683012913		0,720119604
7	0,699346666	0,624008499		0,636838955
8	0,642916026	0,618181655		0,52932928
N/A			0,814291586	

Tabla 120: Tabla de resultado de la Precision promedio del Dataset: Speaker accent recognition

Recall promedio				
Dataset: **Speaker accent recognition**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,713054654**	**0,701339286**	**0,787215909**	**0,700527597**
2	0,750757576	0,747727273		0,732575758
3	0,759848485	0,753787879		0,750568182
4	0,732954545	0,75094697		0,753787879
5	0,717329545	0,683901515		0,717329545
6	0,680587121	0,671780303		0,711458333
7	0,702367424	0,641477273		0,64782197
8	0,647537879	0,659753788		0,590151515
N/A			0,787215909	

Tabla 121: Tabla de resultado del Recall promedio del Dataset: Speaker accent recognition

7.4.24. Vertebral column.

Accuracy				
Dataset: **Vertebral column**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Máximo:	**0,85483871**	**0,864516129**	**0,790322581**	**0,861290323**
2	0,841935484	0,841935484		0,861290323
3	0,851612903	0,864516129		0,851612903
4	0,829032258	0,851612903		0,848387097
5	0,790322581	0,809677419		0,783870968
6	0,85483871	0,841935484		0,832258065
7	0,84516129	0,809677419		0,812903226
8	0,787096774	0,796774194		0,722580645
N/A			0,790322581	

Tabla 122: Tabla de resultado del Accuracy Máximo del Dataset: Vertebral columna

Tiempo promedio				
Dataset: **Vertebral column**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**4,897437607**	**5,541730564**	**12,86073308**	**4,6190274**
2	7,4590976	9,75108676		8,229974079
3	6,553900123	6,723094273		5,51566596
4	4,938041306	5,237866616		4,480020714
5	3,431010461	3,59479661		3,225965118
6	4,258383155	5,001704884		3,962362194
7	4,303284216	4,613387609		4,080771899
8	3,338346386	3,870177197		2,838431835
N/A			12,86073308	

Tabla 123: Tabla de resultado del tiempo promedio del Dataset: Vertebral columna

F1 Score promedio				
Dataset: **Vertebral column**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,820217863**	**0,823451134**	**0,790439166**	**0,806111346**
2	0,838021602	0,83603709		0,855801083
3	0,850019542	0,864246429		0,850035632
4	0,827191321	0,851404335		0,847237521
5	0,775139723	0,801718357		0,76367237
6	0,850750798	0,830166552		0,820739079
7	0,833481403	0,803150532		0,802345453
8	0,766920649	0,77743464		0,702948281
N/A			0,790439166	

Tabla 124: Tabla de resultado del F1 Score promedio del Dataset: Vertebral columna

Precision promedio				
Dataset: **Vertebral column**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,830873001**	**0,83286106**	**0,805146372**	**0,815588688**
2	0,851529058	0,856061113		0,875486677
3	0,858303275	0,871636003		0,858783265
4	0,837971707	0,86156234		0,855531189
5	0,784351751	0,808733766		0,776681791
6	0,872991401	0,833891504		0,826128203
7	0,839269388	0,820389505		0,815339085
8	0,771694425	0,777753187		0,701170604
N/A			0,805146372	

Tabla 125: Tabla de resultado de la Precision promedio del Dataset: Vertebral column

Recall promedio				
Dataset: **Vertebral column**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,828571429**	**0,830875576**	**0,790322581**	**0,816129032**
2	0,841935484	0,841935484		0,861290323
3	0,851612903	0,864516129		0,851612903
4	0,829032258	0,851612903		0,848387097
5	0,790322581	0,809677419		0,783870968
6	0,85483871	0,841935484		0,832258065
7	0,84516129	0,809677419		0,812903226
8	0,787096774	0,796774194		0,722580645
N/A			0,790322581	

Tabla 126: Tabla de resultado del Recall promedio del Dataset: Vertebral columna

7.4.25. Vowel.

Accuracy				
Dataset: **Vowel**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Máximo:	**0,949494949**	**0,954545455**	**0,95959596**	**0,950505051**
2	0,938383838	0,937373737		0,933333333
3	0,949494949	0,954545455		0,950505051
4	0,948484848	0,938383838		0,939393939
5	0,939393939	0,933333333		0,947474747
6	0,918181818	0,929292929		0,92020202
7	0,919191919	0,922222222		0,912121212
8	0,913131313	0,908080808		0,9
N/A			0,95959596	

Tabla 127: Tabla de resultado del Accuracy Máximo del Dataset: Vowel

Tiempo promedio				
Dataset: **Vowel**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**12,90035557**	**13,84087088**	**26,00311694**	**12,40842815**
2	22,06021831	20,87487428		18,62071316
3	16,78067961	18,36030328		18,05857816
4	14,26650412	13,7370949		14,70440314
5	10,02768958	11,331972		10,53347731
6	10,33977931	12,22808094		8,791321158
7	9,414081836	11,70887184		8,99816227
8	7,413536239	8,644898939		7,15234189
N/A			26,00311694	

Tabla 128: Tabla de resultado del tiempo promedio del Dataset: Vowel

F1 Score promedio				
Dataset: **Vowel**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,931742473**	**0,931080198**	**0,95917425**	**0,928148959**
2	0,937815361	0,937123357		0,93177244
3	0,949117591	0,954078757		0,950203472
4	0,94791015	0,937675085		0,938439687
5	0,938669334	0,93233888		0,947229909
6	0,917599201	0,927798619		0,919090231
7	0,918267538	0,920919042		0,91081882
8	0,912818137	0,907627648		0,899488151
N/A			0,95917425	

Tabla 129: Tabla de resultado del F1 Score promedio del Dataset: Vowel

Precision promedio				
Dataset: **Vowel**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,939048722**	**0,938483465**	**0,96291847**	**0,936117804**
2	0,942967336	0,943339892		0,940730683
3	0,955071166	0,957522957		0,955007543
4	0,952760724	0,945639335		0,945371901
5	0,946280992	0,939151253		0,953559547
6	0,925534894	0,937867309		0,926076928
7	0,927921422	0,929719599		0,92160322
8	0,922804519	0,916143907		0,910474803
N/A			0,96291847	

Tabla 130: Tabla de resultado de la Precision promedio del Dataset: Vowel

Recall promedio				
Dataset: **Vowel**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,932323232**	**0,931890332**	**0,95959596**	**0,929004329**
2	0,938383838	0,937373737		0,933333333
3	0,949494949	0,954545455		0,950505051
4	0,948484848	0,938383838		0,939393939
5	0,939393939	0,933333333		0,947474747
6	0,918181818	0,929292929		0,92020202
7	0,919191919	0,922222222		0,912121212
8	0,913131313	0,908080808		0,9
N/A			0,95959596	

Tabla 131: Tabla de resultado del Recall promedio del Dataset: Vowel

7.4.26. Winequality-red.

Accuracy				
Dataset: **Winequality-red**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Máximo:	**0,614768082**	**0,609139151**	**0,612272013**	**0,612264151**
2	0,609131289	0,609139151		0,604127358
3	0,614147013	0,608506289		0,612264151
4	0,614768082	0,602873428		0,60663522
5	0,605393082	0,606002358		0,61038522
6	0,60163522	0,602256289		0,599748428
7	0,59601022	0,59413522		0,595998428
8	0,597904874	0,596647013		0,601018082
N/A			0,612272013	

Tabla 132: Tabla de resultado del Accuracy Máximo del Dataset: Winequality-red

Tiempo promedio				
Dataset: **Winequality-red**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**5,257121318**	**5,708028718**	**9,988685346**	**4,61680273**
2	7,576641107	8,081656671		7,331430316
3	6,049411535	6,169463754		4,76127305
4	5,5515625	5,985305429		4,734682322
5	4,868433928	5,49521277		4,206665778
6	4,5699615	5,022805762		3,94502182
7	3,99565177	4,667973232		3,649290895
8	4,188186884	4,533783412		3,689254928
N/A			9,988685346	

Tabla 133: Tabla de resultado del tiempo promedio del Dataset: Winequality-red

F1 Score promedio				
Dataset: **Winequality-red**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,581759701**	**0,579006095**	**0,589019923**	**0,581491109**
2	0,587199695	0,588582094		0,581829427
3	0,589679423	0,585958552		0,587535505
4	0,593297563	0,580349159		0,584733147
5	0,582907157	0,583023699		0,587826443
6	0,578347707	0,581602186		0,578929475
7	0,568942191	0,565160372		0,574439864
8	0,571944174	0,568366605		0,575143903
N/A			0,589019923	

Tabla 134: Tabla de resultado del F1 Score promedio del Dataset: Winequality-red

Precision promedio				
Dataset: **Winequality-red**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,572772006**	**0,572941848**	**0,578854852**	**0,575703022**
2	0,575950175	0,581122528		0,572226616
3	0,582228394	0,574283392		0,579359918
4	0,582822181	0,573090025		0,575208453
5	0,572923414	0,575912547		0,58211187
6	0,570815027	0,569475633		0,568767536
7	0,564098866	0,563817639		0,567474949
8	0,560565986	0,57289117		0,584771809
N/A			0,578854852	

Tabla 135: Tabla de resultado de la Precision promedio del Dataset: Winequality-red

Recall promedio				
Dataset: **Winequality-red**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,605569969**	**0,60279425**	**0,612272013**	**0,604310984**
2	0,609131289	0,609139151		0,604127358
3	0,614147013	0,608506289		0,612264151
4	0,614768082	0,602873428		0,60663522
5	0,605393082	0,606002358		0,61038522
6	0,60163522	0,602256289		0,599748428
7	0,59601022	0,59413522		0,595998428
8	0,597904874	0,596647013		0,601018082
N/A			0,612272013	

Tabla 136: Tabla de resultado del Recall promedio del Dataset: Winequality-red

7.1.1. Winequality-White.

Accuracy				
Dataset: **Winequality- White**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Máximo:	**0,562075873**	**0,552069196**	**0,551658946**	**0,556356162**
2	0,549007554	0,551665206		0,556356162
3	0,562075873	0,552069196		0,54900338
4	0,549212053	0,550431952		0,555133342
5	0,552061266	0,548391553		0,539412796
6	0,554314094	0,536151663		0,550842619
7	0,544100831	0,54063186		0,543084178
8	0,536957973	0,532873419		0,541450691
N/A			0,551658946	

Tabla 137: Tabla de resultado del Accuracy Máximo del Dataset: Winequality- White

Tiempo promedio				
Dataset: **Winequality- White**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**10,01054987**	**11,01354958**	**22,90941589**	**9,803454001**
2	17,3765794	18,65922575		19,69936476
3	12,62962751	14,80269957		12,1306175
4	9,420844269	10,39861345		10,26853487
5	9,426161671	9,467018032		7,362163734
6	7,5479877	9,214656544		7,25637722
7	7,630641508	7,51364727		6,252155042
8	6,042007041	7,038986421		5,654964876
N/A			22,90941589	

Tabla 138: Tabla de resultado del tiempo promedio del Dataset: Winequality- White

F1 Score promedio				
Dataset: **Winequality- White**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,522960356**	**0,513282541**	**0,528682347**	**0,515356929**
2	0,522993037	0,52719309		0,53174308
3	0,539815886	0,529212234		0,523738129
4	0,524760232	0,524933525		0,528735325
5	0,527562758	0,51496161		0,504391687
6	0,524646093	0,49654706		0,515404593
7	0,521066577	0,512204777		0,508982113
8	0,499877912	0,487925487		0,494503577
N/A			0,528682347	

Tabla 139: Tabla de resultado del F1 Score promedio del Dataset: Winequality- White

Precision promedio				
Dataset: **Winequality- White**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,52548026**	**0,520627519**	**0,533529125**	**0,517459031**
2	0,52703743	0,533412917		0,538981172
3	0,541667343	0,53253148		0,523396684
4	0,526743674	0,525017791		0,53342945
5	0,526550863	0,515891374		0,502056362
6	0,534046575	0,51842747		0,518258331
7	0,521110868	0,515072789		0,50890271
8	0,501205071	0,504038809		0,497188508
N/A			0,533529125	

Tabla 140: Tabla de resultado de la Precision promedio del Dataset: Winequality- White

Recall promedio				
Dataset: **Winequality- White**	**Chebyshev**	**Legendre**	**Lineal**	**Shmaliy**
Promedio:	**0,549675663**	**0,544602121**	**0,551658946**	**0,547897595**
2	0,549007554	0,551665206		0,556356162
3	0,562075873	0,552069196		0,54900338
4	0,549212053	0,550431952		0,555133342
5	0,552061266	0,548391553		0,539412796
6	0,554314094	0,536151663		0,550842619
7	0,544100831	0,54063186		0,543084178
8	0,536957973	0,532873419		0,541450691
N/A			0,551658946	

Tabla 141: Tabla de resultado del Recall promedio del Dataset: Winequality- White

8. Trabajos relacionados.

En los últimos años, se ha intensificado la investigación de nuevas arquitecturas que desafían la estructura actual de las redes neuronales profundas, explorando un amplio abanico de posibilidades como nuevas fórmulas de activación, o algún modificar la estructura interna de estos mismos. En nuestro caso, la propuesta se centra en modificar la etapa de extracción de características sustituyendo la transformación afín clásica por expansiones basadas en funciones polinómicas ortogonales. Esta aproximación matemática pertenece a una línea de investigación que cuenta ya con diversos estudios, los cuales han servido como marco de referencia fundamental durante el desarrollo de este proyecto para fundamentar el diseño de las capas y justificar los resultados empíricos obtenidos en la implementación. A continuación, se analizan las investigaciones más relevantes que sustentan y se relacionan directamente con las aportaciones de esta memoria.

8.1. Expansión funcional mediante Redes de Chebyshev (J.C. Patra).

J.C. Patra es un investigador con diferentes estudios en la integración de polinomios ortogonales dentro de arquitecturas de redes neuronales. En sus trabajos fundamentales sobre identificación de sistemas dinámicos no lineales [Patra y Kot, 2002], desarrolló la denominada "Red Neuronal de Chebyshev". Su investigación demostró que, al aplicar un módulo de expansión funcional basado en fórmulas de recurrencia directamente sobre las señales de entrada, una red de una sola capa lograba aproximar funciones altamente complejas con una eficiencia computacional muy superior a la de un Perceptrón Multicapa profundo.

Este enfoque arquitectónico de "unidad de expansión funcional" previa a la multiplicación de pesos se alinea estrechamente con el diseño algebraico propuesto en este proyecto, donde las características se mapean funcionalmente antes de aplicar el tensor tridimensional. Además, la exploración de Patra sobre la propiedad matemática Minimax de esta familia polinómica valida la capacidad de Chebyshev para mitigar oscilaciones violentas en los extremos del dominio normalizado [-1, 1], una propiedad que sustenta la estabilidad numérica observada en las capas personalizadas desarrolladas en este proyecto.

8.2. Optimalidad de los modelos polinómicos de Legendre (Milojković et al.)

Por otro lado, en el ámbito de la optimización de sistemas, Milojković et al. (2021) exploraron el diseño de modelos óptimos para sistemas no lineales utilizando Redes Neuronales Polinómicas Ortogonales. Su investigación se centró en el uso de funciones “cuasi-ortogonales” de Legendre, demostrando que la ortogonalidad matemática elimina la redundancia de información entre las neuronas. También se probó que este enfoque posee una "optimalidad natural intrínseca" que reduce drásticamente el error de truncamiento y acelera la convergencia del modelo frente a redes estándar.

Los hallazgos de Milojković respecto a la distribución uniforme del error en la base de Legendre respaldan matemáticamente los resultados obtenidos en el presente estudio.

Concretamente, la función de peso uniforme característica de esta familia explica por qué la arquitectura de Legendre logró la mayor estabilidad y escaló positivamente en la métrica de exactitud hasta el grado polinómico 4 sin degradarse. Asimismo, la aceleración de la convergencia documentada por su equipo corrobora las métricas de eficiencia extraídas de nuestra validación cruzada, donde los modelos polinómicos evidenciaron una convergencia hasta un 20 % más rápida que la capa lineal de referencia.

9. Conclusión y cierre.

La presente investigación ha permitido validar la eficacia de integrar capas de expansión polinómica ortogonal como alternativa a las capas densas tradicionales en arquitecturas de aprendizaje profundo. Tras un extenso análisis experimental sobre 27 conjuntos de datos heterogéneos del repositorio UCI y múltiples configuraciones de grado, se presentan las siguientes conclusiones fundamentales:

- **Superioridad técnica de las bases polinómicas,**
    - Se ha demostrado que las redes neuronales polinómicas no solo igualan, sino que superan el rendimiento del modelo lineal convencional en términos de capacidad de generalización cuando se identifica el grado óptimo para cada conjunto de datos. Este resultado es coherente con la teoría de aproximación polinómica (Quarteroni, Sacco y Saleri 2006; Trefethen 2019): las bases ortogonales proporcionan una representación más expresiva del espacio de características que una proyección lineal estándar.

- **Eficiencia computacional y la paradoja de la convergencia**
    - Uno de los hallazgos más relevantes del trabajo es la reducción sistemática de los tiempos de entrenamiento. A pesar de la mayor complejidad aritmética de las capas polinómicas, se ha registrado una mejora en la eficiencia temporal de hasta un 20,41 % (Legendre) y un 17,53 % (Chebyshev). El estudio confirma que el aumento del grado polinómico acelera la velocidad de convergencia como consecuencia de dos mecanismos complementarios: la linealización del espacio de características predicha por el Teorema de Cover (Cover 1965) y la riqueza del flujo de gradientes, que permite al optimizador alcanzar los criterios de parada temprana de forma mucho más veloz.

- **El compromiso entre complejidad y sobreajuste**
    - El análisis por grados ha permitido identificar un límite estructural claro: mientras los grados 2 y 4 potencian el aprendizaje de la red, el salto al grado 8 provoca una degradación del rendimiento en todas las familias polinómicas. Este fenómeno de sobreajuste estructural subraya la importancia de la selección del grado como hiperparámetro crítico, resultado coherente con las recomendaciones de Bergstra y Bengio sobre la optimización de hiperparámetros (Bergstra y Bengio 2012), y con el principio general de parsimonia en el aprendizaje automático (Goodfellow, Bengio y Courville 2016).

- **Especificidad de cada familia polinómica.**
    - Legendre destaca por su estabilidad y capacidad para escalar positivamente hasta el grado 4, atribuible a su función de peso uniforme.Shmaliy sobresale en *dataset*s con ruido o alta dispersión, gracias a la función de filtrado implícita de su soporte discreto. Chebyshev, por su parte, ofrece ventajas en escenarios donde las muestras críticas se concentran en los extremos del dominio normalizado, resultado de su función de peso.

En conjunto, los resultados evidencian que la integración de capas polinómicas ortogonales en arquitecturas de aprendizaje profundo representa una alternativa técnicamente viable y computacionalmente eficiente respecto a las capas densas convencionales, con un rendimiento diferencial que se maximiza cuando el grado se selecciona adecuadamente en función de la naturaleza del *dataset*.

Bibliografía

ABADI, M., AGARWAL, A., BARHAM, P. et al., 2015. TensorFlow: Large-scale machine learning on heterogeneous systems [en línea]. Software disponible en: https://www.tensorflow.org

BENGIO, Y., SIMARD, P. y FRASCONI, P., 1994. Learning long-term dependencies with gradient descent is difficult. IEEE Transactions on Neural Networks [en línea], 5(2), pp. 157–166. Disponible en: https://doi.org/10.1109/72.279181

BERGSTRA, J. y BENGIO, Y., 2012. Random search for hyper-parameter optimization. Journal of Machine Learning Research, 13(10), pp. 281–305.

BOTTOU, L., 2010. Large-scale machine learning with stochastic gradient descent. En: Proceedings of COMPSTAT'2010. Physica-Verlag HD, pp. 177-186. Disponible en: https://doi.org/10.1007/978-3-7908-2604-3_16

BRIDLE, J. S., 1990. Probabilistic interpretation of feedforward classification network outputs, with relationships to statistical pattern recognition. En: Neurocomputing: Algorithms, Architectures and Applications. Springer, pp. 227–236.

CHOLLET, F. et al., 2015. Keras [en línea]. GitHub. Disponible en: https://github.com/fchollet/keras

COVER, T. M., 1965. Geometrical and statistical properties of systems of linear inequalities with applications in pattern recognition. IEEE Transactions on Electronic Computers [en línea], EC-14(3), pp. 326–334. Disponible en: https://doi.org/10.1109/PGEC.1965.264137

DAUPHIN, Y. N., PASCANU, R., GULCEHRE, C., CHO, K., GANGULI, S. y BENGIO, Y., 2014. Identifying and attacking the saddle point problem in high-dimensional non-convex optimization. Advances in neural information processing systems (NIPS), 27, pp. 2933-2941.

DUA, D. y GRAFF, C., 2019. UCI Machine Learning Repository [en línea]. Irvine, CA: University of California, School of Information and Computer Science. Disponible en: https://archive.ics.uci.edu/ml

FAWCETT, T., 2006. An introduction to ROC analysis. Pattern Recognition Letters [en línea], 27(8), pp. 861–874. Disponible en: https://doi.org/10.1016/j.patrec.2005.10.010

FERNÁNDEZ-DELGADO, M., CERNADAS, E., BARRO, S. y AMORIM, D., 2014. Do we need hundreds of classifiers to solve real world classification problems? Journal of Machine Learning Research, 15(1), pp. 3133–3181.

GARCÍA, S., LUENGO, J. y HERRERA, F., 2015. Data Preprocessing in Data Mining. Cham: Springer.

GAUTSCHI, W., 2004. Orthogonal Polynomials: Computation and Approximation. Oxford: Oxford University Press.

GÉRON, A., 2022. Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow. 3.ª ed. Sebastopol, CA: O'Reilly Media.

GLOROT, X. y BENGIO, Y., 2010. Understanding the difficulty of training deep feedforward neural networks. En: Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS 2010). PMLR 9, pp. 249–256.

GLOROT, X., BORDES, A. y BENGIO, Y., 2011. Deep sparse rectifier neural networks. En: Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS 2011). PMLR 15, pp. 315–323.

GOODFELLOW, I., BENGIO, Y. y COURVILLE, A., 2016. Deep Learning. Cambridge, MA: MIT Press.

HARRIS, C. R., MILLMAN, K. J., VAN DER WALT, S. J. et al., 2020. Array programming with NumPy. Nature [en línea], 585(7825), pp. 357-362. Disponible en: https://doi.org/10.1038/s41586-020-2649-2

HAYKIN, S., 1999. Neural Networks: A Comprehensive Foundation. 2.ª ed. Prentice Hall International.

HORNIK, K., STINCHCOMBE, M. y WHITE, H., 1989. Multilayer feedforward networks are universal approximators. Neural Networks [en línea], 2(5), pp. 359–366. Disponible en: https://doi.org/10.1016/0893-6080(89)90020-8

HUNTER, J. D., 2007. Matplotlib: A 2D graphics environment. Computing in Science & Engineering [en línea], 9(3), pp. 90-95. Disponible en: https://doi.org/10.1109/MCSE.2007.55

KINGMA, D. P. y BA, J., 2015. Adam: A method for stochastic optimization. En: Proceedings of the 3rd International Conference on Learning Representations (ICLR 2015) [en línea]. arXiv:1412.6980. Disponible en: https://arxiv.org/abs/1412.6980

KOHAVI, R., 1995. A study of cross-validation and bootstrap for accuracy estimation and model selection. En: Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI-95), vol. 2, pp. 1137–1143.

LeCUN, Y., BOTTOU, L. y ORR, G. B., 1998. Efficient backprop. En: G. B. ORR y K.-R. MÜLLER, eds. Neural Networks: Tricks of the Trade. Berlín: Springer, pp. 9–50.

LeCUN, Y., BENGIO, Y. y HINTON, G., 2015. Deep learning. Nature [en línea], 521(7553), pp. 436–444. Disponible en: https://doi.org/10.1038/nature14539

LIU, Z., WANG, Y., VAIDYA, S., RUEHLE, F., HALVERSON, J., SOLJAČIĆ, M., HOU, T. Y. y TEGMARK, M., 2024. KAN: Kolmogorov-Arnold Networks [en línea]. arXiv:2404.19756. Disponible en: https://arxiv.org/abs/2404.19756

MASON, J. C. y HANDSCOMB, D. C., 2002. Chebyshev Polynomials. Boca Raton, FL: CRC Press.

MILOJKOVIĆ, M., MILOVANOVIĆ, M., NIKOLIĆ, S. S., SPASIĆ, M. y ANTIĆ, A., 2021. Designing optimal models of nonlinear MIMO systems based on orthogonal polynomial neural networks. Mathematical and Computer Modelling of Dynamical Systems [en línea], 27(1), pp. 246-262. Disponible en: https://doi.org/10.1080/13873954.2021.1909069

NAIR, V. y HINTON, G. E., 2010. Rectified linear units improve restricted Boltzmann machines. En: Proceedings of the 27th International Conference on Machine Learning (ICML 2010), pp. 807–814.

PATRA, J. C. y KOT, A. C., 2002. Nonlinear Dynamic System Identification Using Chebyshev Neural Networks. IEEE Transactions on Neural Networks [en línea], 13(1), pp. 227-235. Disponible en: https://doi.org/10.1109/72.977274

PEDREGOSA, F., VAROQUAUX, G., GRAMFORT, A., MICHEL, V., THIRION, B., GRISEL, O., BLONDEL, M., PRETTENHOFER, P., WEISS, R., DUBOURG, V., VANDERPLAS, J., PASSOS, A., COURNAPEAU, D., BRUCHER, M., PERROT, M. y DUCHESNAY, É., 2011. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, pp. 2825–2830.

POLYAK, B. T., 1964. Some methods of speeding up the convergence of iteration methods. USSR Computational Mathematics and Mathematical Physics [en línea], 4(5), pp. 1-17. Disponible en: https://doi.org/10.1016/0041-5553(64)90137-5

PRECHELT, L., 1998. Early stopping — but when? En: G. B. ORR y K.-R. MÜLLER, eds. Neural Networks: Tricks of the Trade. Berlín: Springer, pp. 55–69.

QUARTERONI, A., SACCO, R. y SALERI, F., 2006. Numerical Mathematics. 2.ª ed. Berlín: Springer.

RAMACHANDRAN, P., ZOPH, B. y LE, Q. V., 2017. Searching for activation functions [en línea]. arXiv:1710.05941. Disponible en: https://arxiv.org/abs/1710.05941

RUMELHART, D. E., HINTON, G. E. y WILLIAMS, R. J., 1986. Learning representations by back-propagating errors. Nature [en línea], 323(6088), pp. 533–536. Disponible en: https://doi.org/10.1038/323533a0

SHMALIY, Y. S., 2006. Optimal Gains of FIR Estimators for a Class of Discrete-Time State-Space Models. IEEE Signal Processing Letters [en línea], 13(11), pp. 677–680. Disponible en: https://doi.org/10.1109/LSP.2006.879455

SNOEK, J., LAROCHELLE, H. y ADAMS, R. P., 2012. Practical Bayesian optimization of machine learning algorithms. En: Advances in Neural Information Processing Systems (NIPS 2012), vol. 25, pp. 2951–2959.

SOKOLOVA, M. y LAPALME, G., 2009. A systematic analysis of performance measures for classification tasks. Information Processing & Management [en línea], 45(4), pp. 427–437. Disponible en: https://doi.org/10.1016/j.ipm.2009.03.002

SRIVASTAVA, N., HINTON, G., KRIZHEVSKY, A., SUTSKEVER, I. y SALAKHUTDINOV, R., 2014. Dropout: a simple way to prevent neural networks from overfitting. Journal of Machine Learning Research, 15(1), pp. 1929-1958.

SZEGŐ, G., 1939. Orthogonal Polynomials. Providence, RI: American Mathematical Society.

TIELEMAN, T. y HINTON, G., 2012. Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude. COURSERA: Neural networks for machine learning, 4(2), pp. 26-31.

TREFETHEN, L. N., 2019. Approximation Theory and Approximation Practice. Ed. amp. Philadelphia, PA: SIAM.

WEIERSTRASS, K., 1885. Über die analytische Darstellbarkeit sogenannter willkürlicher Functionen einer reellen Veränderlichen. Sitzungsberichte der Königlich Preußischen Akademie der Wissenschaften zu Berlin, pp. 633–639, 789–805.